% Senior iOS Interview Prep — Revision % Muhammed Shahid Siddiqui

Revision Track

Short-form **active recall** notes. Use these **after** you study the full chapters in `../weeks/` .

How to use

1. Study the full day folder: `../weeks/week-XX/day-YY/README.md`
2. Close it
3. Open the matching file here: `weeks/week-XX/day-YY.md`
4. Run flashcards + timed drills from the revision file
5. Only peek at full chapters for gaps

Layout

```
revision/
  README.md          ← you are here
  weeks/week-XX/day-YY.md   ← compact drill notes (frozen outline)
```

Full self-contained teaching lives under `roadmap/weeks/week-XX/day-YY/` (modular chapters).

Rules

- Do not expand revision files into textbooks — growth goes to full chapters only
- Metrics and company claims must stay resume-true (see `../provenance/`)

Revision Week 01

Day 01 — Value vs Reference, COW, Enums, Actors Intro

> *Week 1 · Phase: Swift mastery · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- When to choose `struct` vs `class` vs `actor` vs `enum`
- What copy-on-write means for Array/String/Dictionary and when a copy actually happens
- How recursive enums and associated values model domain state
- A 30s pitch tying value semantics to the BMS ads / listing models

2. Concept deep dive

2.1 Value vs reference

Type	Semantics	Shared mutation?	Typical use
`struct` / `enum`	Value	No (independent copies)	Models, state, DTOs
`class`	Reference	Yes	Identity, shared services, UIKit objects
`actor`	Reference + isolation	Mutate only inside actor	Shared mutable state across concurrency

Senior rule: Prefer value types for data. Introduce classes when you need identity, inheritance (rare), or Objective-C interop. Introduce actors when you need safe shared mutability across tasks.

2.2 Copy-on-write (COW)

Swift's `Array`, `Dictionary`, `String`, `Set` use COW: assignment is cheap (share buffer); mutation of a uniquely referenced buffer happens in place; if `isKnownUniquelyReferenced` is false, the buffer is copied first.

Trap: Holding two variables to the "same" array and mutating one — the other may or may not see changes depending on uniqueness. Don't rely on reference-like sharing for value collections.

2.3 Enums as state machines

Associated-value enums excel for UI/network/payment states:

```
enum LoadState<T> {
    case idle
    case loading
    case loaded(T)
    case failed(Error)
}
```

Payment popup at BMS maps cleanly: `processing` / `success` / `failure` / `timeout`.

2.4 Actors (intro only — Day 05 deepens)

An `actor` is a reference type that serializes access to its mutable state. Soft pitch today: "Where we used a serial queue around a dictionary at BMS, a Swift `actor` is the modern language-native equivalent for new code."

2.5 Trade-offs

Choice	When	Cost
Large struct with many copies	Small models, frequent pass-by-value	Copy cost if not COW / large stored properties
Class for everything	UIKit legacy	Shared mutation bugs, retain cycles
Actor for UI state	Rarely — prefer `@MainActor` ViewModel	Over-isolation complexity

3. Read these

Priority	Resource	Why
Must	Classes and Structures	Canonical value vs reference
Must	Enumerations	Associated values, recursive enums
Deepen	Concurrency — Actors (actors section)	Prep for Day 05
Repo	social-feed.md — requirements/scope only today	Week 1 SD spine

4. Map to your work

Company / feature: BookMyShow — Ads / listing models; Payment processing popup states
What you did: Kept render models as value-friendly DTOs in a type-safe ads pipeline; modeled processing UI as explicit states rather than boolean flags.
Interview line (≤20s): "I default to structs for ad and listing models so accidental shared mutation can't corrupt revenue UI; classes/actors only at identity or concurrency boundaries."

→ Full STAR: story-bank S1, S7

5. Normal questions

Q1. Struct vs class? `(30–45s)`

Skeleton: Value vs reference → mutation sharing → prefer struct for data, class for identity.
Follow-up: When is a class mandatory?
Story: Ads models / HeroWidget ownership.

Q2. What is COW? `(30–45s)`

Skeleton: Shared buffer until mutation; unique ref → in-place.
Follow-up: Does assigning an array always copy elements?
Story: Large listing arrays — avoid defensive copies in hot paths.

Q3. Why prefer enums over booleans for UI state? `(30–45s)`

Skeleton: Impossible states; associated payloads; switch exhaustiveness.
Follow-up: How do you version new cases for SDUI?
Story: Payment processing popup.

Q4. What is an actor at a high level? `(30–45s)`

Skeleton: Isolated reference type; await hops; prevents data races.
Follow-up: Actor vs serial queue?
Story: Bridge to synchronised dictionaries (S2).

Q5. Value type containing a class — what happens on copy? `(45s)`

Skeleton: Struct copied; nested class reference shared.
Follow-up: How do you deep-copy if needed?
Story: Mixed model graphs in UIKit cells.

Q6. `===` vs `==`? `(30s)`

Skeleton: Identity vs equality; classes; Equatable.
Follow-up: Should view models use identity?

Q7. When would you use a class for a model? `(45s)`

Skeleton: Shared mutable cache entry with identity; ObjC; KVO legacy.
Follow-up: Could an actor replace it?

Q8. Recursive enums — why `indirect`? `(45s)`

Skeleton: Fixed size layout; indirection for recursive associated values.
Follow-up: Example AST / nested comment thread.

Q9. Are actors value or reference types? `(30s)`

Skeleton: Reference types with isolation.

Q10. How do you explain COW in one sentence to a junior? `(30s)`

Skeleton: “Cheap share until someone writes.”

6. Tricky questions

T1. You mutate `arrayB` after `let arrayA = arrayB` — does A change? `(90s)`

Trap: Assuming always shared or always copied.
Senior answer: Depends on uniqueness at mutation time; COW may copy first. Demonstrate mental model with

`isKnownUniquelyReferenced`.
Follow-up: What if both are class-wrapped?

T2. Large struct passed through many functions — performance? `(90–120s)`

Trap: "Structs are always faster."
Senior answer: Small structs win; large stored properties may copy; use COW wrappers, `inout`, or class/actor at boundary.
Follow-up: How Instruments Allocations would show this.

T3. Enum with associated `Error` vs generic `Result` for ViewModel state `(90s)`

Trap: One true way.
Senior answer: Domain enum communicates UI; `Result` is fine internally; map at boundary. Payment popup example.

T4. Why not make every ViewModel an actor? `(90s)`

Trap: Actors solve all threading.
Senior answer: UI needs `@MainActor`; overusing actors adds hop latency and API friction; isolate the shared mutable store, not every VM.

T5. Class in a Set — what's required? `(90s)`

Trap: Only Hashable on fields.
Senior answer: `Hashable` / `Equatable` usually by identity (`ObjectIdentifier`) or stable ID; mutating hashed fields while in Set is undefined behavior.

7. Flashcards for today

Front	Back
struct vs class	Value vs reference · Trap: class for all models · Prod: BMS ad DTOs as structs
COW	Share buffer until write · Trap: assume A sees B's mutation · Prod: listing arrays
actor (1-liner)	Isolated reference type · Trap: use for all UI · Prod: modernize sync dicts
enum state machine	Exhaustive states · Trap: boolean flags · Prod: payment popup
===	Referential identity · Trap: use for structs · Prod: VC identity
indirect enum	Heap box for recursion · Trap: forget indirect · Prod: nested domain trees
Prefer values when	No identity needed · Trap: premature class · Prod: Clean models
Nested class in struct copy	Reference shared · Trap: think deep copy · Prod: mixed graphs
`@MainActor` vs actor	UI affinity vs general isolation · Trap: conflate · Prod: VM on main
LoadState<T>	idle/loading/loaded/failed · Trap: isLoading+optional · Prod: search MVVM

8. Practice

- **Coding:** Implement `LoadState<T>` + map `Result` → `LoadState`. Write a tiny COW demo in a Playground (two array vars, mutate, print).
- **SD slice (20–30 min):** Read Social Feed **requirements/scope** only — write 5 clarifying questions you'd ask for BMS-scale listing (30L+ DAU).
- **Complexity / agenda:** For any "struct vs class" Q, say agenda in 5s: "semantics → mutation → example."

9. Timed drill

1. Record Q1, Q2, Q3, T1, T2.
2. Score with answer-timing-guide.md.

3. Practice S7 payment states in ≤90s Action segment.
4. Log gotchas.

Day 02 — Protocols, POP, Generics, Associated Types, Type Erasure

> *Week 1 · Phase: Swift mastery · Time budget today: ~4–5 hrs*

1. Outcome

Explain aloud:

- Protocol-oriented design vs inheritance for UI component pipelines
- Generics + associated types + `where` clauses
- When you need type erasure (`AnyPublisher` -style) and the cost
- How BMS Ads **HeroWidget** used POP + Generics for a type-safe renderer

2. Concept deep dive

2.1 Protocol-Oriented Programming (POP)

Design around capabilities (`Renderable` , `Trackable` , `LifecycleAware`) composed with extensions. Prefer protocol composition over deep class hierarchies for ad units, form fields, SDUI components.

```
protocol AdRenderable {
    associatedtype Content
    func render(_ content: Content) -> UIView
}
```

2.2 Generics vs protocols with associated types (PATs)

- **Generics:** Caller chooses concrete type; compiler specializes; best for pipelines.
- **PATs:** Cannot use as existential easily (`any AdRenderable` limitations historically); often pair with generics: `func install<R: AdRenderable>(_ r: R)` .
- Swift 5.7+ `any` / `some` improved existentials — still know the trade-offs.

2.3 Type erasure

When you must store heterogeneous PAT-conforming values, erase: wrapper class/struct holding `boxes` with a common interface. Cost: allocation, indirection, lost specialization.

2.4 Static vs dynamic dispatch

Protocol witness table (dynamic) vs generic specialization (static). `final` classes / generics help performance-sensitive paths (cell configure, ad bind).

2.5 Trade-offs

Choice	When	Cost
POP + generics pipeline	Many ad/SDUI variants	Learning curve; PAT friction
Inheritance hierarchy	Rare shared identity	Fragile base class
Type erasure	Heterogeneous arrays	Runtime cost
`any Protocol`	Flexibility	Existential overhead

3. Read these

Priority	Resource	Why
Must	Protocols	Extensions, composition
Must	Generics	Constraints, where
Deepen	Swift Generics Manifesto (skim)	Mental model
Repo	Skim component registry ideas in sdui-engine.md	Same POP shape as SDUI

4. Map to your work

Company / feature: BookMyShow — Ads module / HeroWidget
What you did: Refactored highest-revenue module with POP + Generics for type-safe rendering; video pause/play lifecycle on HeroWidget.
Interview line (≤20s): "We made ad rendering a generic protocol pipeline so new creatives plugged in without forking the revenue path."

→ S1 Ads · S10 Stories SDK for reusable protocol APIs

5. Normal questions

Q1. What is POP? `(30–45s)`

Skeleton: Design by composing protocols + extensions; prefer over inheritance.
Story: S1 Ads.

Q2. associatedtype vs generic parameter? `(45–60s)`

Skeleton: associatedtype belongs to conforming type; generics chosen by caller.
Follow-up: Can you put PAT in an array?

Q3. `some` vs `any`? `(45s)`

Skeleton: opaque concrete vs existential.
Follow-up: Performance difference.

Q4. Why generics in an ads pipeline? `(45s)`

Skeleton: Compile-time type safety; avoid `Any` casts in revenue code.
Story: S1.

Q5. Protocol extension default methods — dispatch catch? `(60s)`

Skeleton: Defaults are not always dynamically dispatched unless requirement declared.
Follow-up: How to force dynamic dispatch.

Q6. What is type erasure? `(45s)`

Skeleton: Wrap PAT to common type; example AnyPublisher.
Follow-up: Cost?

Q7. Protocol composition (`A & B`)? `(30s)`

Skeleton: Require multiple capabilities.
Story: Renderable & Trackable ads.

Q8. When is inheritance still OK? `(45s)`

Skeleton: UIKit subclasses; shared identity; ObjC.
Follow-up: Mixing POP + UIView subclasses.

Q9. Conditional conformance? `(45s)`

Skeleton: `Array: Equatable where Element: Equatable`.
Follow-up: Use in model layers.

Q10. Generics for repository interfaces? `(45s)`

Skeleton: `Repository` with associated `Model` / generic methods.
Story: District Clean boundaries.

Q11. Difference between `protocol P: AnyObject` and struct-friendly protocols? `(45s)`

Skeleton: Class-bound for weak refs / identity.

6. Tricky questions

T1. Why can't you write `var x: [AdRenderable]` easily with associated types? `(120s)`

Trap: "Just use the protocol name."
Senior answer: PAT + Self/associatedtype prevent simple existentials historically; use generics, type erasure, or constrained `any` carefully.

T2. Default implementation in extension vs requirement — override surprise `(90s)`

Trap: Expect polymorphic call to default.
Senior answer: If not a protocol requirement, static dispatch to default via witness of static type.

T3. Type-erasing a generic renderer — when is it worth it? `(90s)`

Trap: Erase everything.
Senior answer: Only at module boundaries / heterogeneous lists; keep generics inside hot pipeline.

T4. POP for SDUI component registry vs enums `(120s)`

Trap: Enum of all components forever.
Senior answer: Enum is closed; protocol registry is open for CMS growth — need versioning + unknown fallback (preview Day 10).

T5. Generics overkill for two ad types? `(90s)`

Trap: Always abstract.
Senior answer: YAGNI — introduce POP when third variant or test seams demand it; at BMS revenue scale, variants justified abstraction.

T6. `rethrows` + generic higher-order functions `(90s)`

Trap: Ignore error propagation.
Senior answer: Propagate only if closure throws; clean API for mappers.

7. Flashcards for today

Front	Back
POP	Compose protocols · Trap: deep inheritance · Prod: Ads pipeline
associatedtype	Conformer picks type · Trap: use as easy existential · Prod: AdRenderable
some vs any	Opaque vs existential · Trap: synonym · Prod: View returns
Type erasure	Box PAT · Trap: free · Prod: heterogeneous ads list
Protocol extension dispatch	Defaults may be static · Trap: expect override · Prod: shared track()

Generics benefit	Specialize + safety · Trap: Any everywhere · Prod: HeroWidget
AnyObject protocol	Class-bound · Trap: struct conform · Prod: weak delegate
Conditional conformance	where on extension · Trap: always free · Prod: model arrays
Composition A & B	Multi-capability · Trap: multiple inheritance myth · Prod: Render+Track
Open vs closed SDUI	Registry vs enum · Trap: enum forever · Prod: BMS header
Witness table	Dynamic protocol dispatch · Trap: always slow · Prod: cell bind
SDK public API	Protocols at boundary · Trap: expose concretes · Prod: Stories SDK

8. Practice

- **Coding:** Build a mini ads pipeline: `protocol Creative`, `struct VideoCreative`, `struct ImageCreative`, `struct Renderer<C: Creative>`. Add pause/play protocol for video only via composition.
- **SD:** Note how SDUI component registry mirrors POP (read 15 min of sdui-engine component registry section).

9. Timed drill

1. Record Q1, Q4, Q6, T1, T4.
2. Deliver S1 architecture talk in **≤5 min** using timing guide architecture template (dry run for Day 14).
3. Score; log gotchas.

Day 03 — ARC, Retain Cycles, weak/unowned, Instruments Leaks

> *Week 1 · Phase: Memory · Time budget today: ~4–5 hrs*

1. Outcome

Explain aloud:

- How ARC works (strong/weak/unowned) and when objects deallocate
- Classic retain cycle patterns (closures, delegates, Timer, NotificationCenter)
- How you'd hunt a leak with Instruments at BMS scale
- Difference between leak, abandoned memory, and high watermark

2. Concept deep dive

2.1 ARC basics

Strong references increment retain count; at zero, `deinit` runs. `weak` is optional zeroing; `unowned` is non-optional non-zeroing (crash if dangling). Use `unowned` only when lifetimes are provably tied (e.g. child owned by parent).

2.2 Cycle hotspots

Pattern	Fix
Closure captures `self` strongly	`[weak self]` + guard
Delegate `strong`	`weak var delegate` (AnyObject)
Timer targeting self	Block timer + weak / invalidate in deinit
NotificationCenter observer (older API)	Token-based / weak observer patterns
Parent→child→parent	One side weak/unowned

2.3 Instruments

- **Leaks:** true leaked objects (no pointers)
- **Allocations:** growth, abandoned memory still referenced
- **Debug Memory Graph:** visual cycles in Xcode

At 30L+ DAU, even small leaks in navigation/ad paths become memory pressure and jetsam.

2.5 Trade-offs

Choice	When	Cost
always `[weak self]`	Uncertain lifetime	Optional noise; early nil
`unowned`	Nested clearly owned	Crash if wrong
`unowned(unsafe)`	Extreme perf	Never in app code casually

3. Read these

Priority	Resource	Why
Must	Automatic Reference Counting	Canonical
Must	Xcode Memory Graph + Leaks instrument (practice once)	Muscle memory
Deepen	crash-reporting-sdk.md — breadcrumbs/OOM skim	Memory ↔ crashes
Repo	social-feed.md image/memory notes if present	Feed memory

4. Map to your work

Company / feature: BookMyShow — crash triage / Crashlytics workflows
What you did: Maintained 99.95%+ crash-free via structured crash workflows; memory/race issues were part of reliability work.
Interview line (≤20s): "At 30L+ DAU we treated retain cycles and abandoned VCs as reliability bugs — Memory Graph + Crashlytics, not guesswork."

→ S8 IMOC

5. Normal questions

Q1. How does ARC work? `(30–45s)`

Skeleton: Compile-time inserts retains/releases; zero → deinit.
Follow-up: ARC vs GC pause behavior.

Q2. weak vs unowned? `(45s)`

Skeleton: Optional zeroing vs non-optional; crash if dangling.
Story: Closure in ads networking.

Q3. Common retain cycle? `(45s)`

Skeleton: Escaping closure → self → owns closure.
Follow-up: How detect?

Q4. Should delegates be weak? `(30s)`

Skeleton: Yes for class delegates to avoid cycles.

Q5. What is a memory leak vs abandoned memory? `(60s)`

Skeleton: Unreachable vs reachable but unused.
Follow-up: Which Instruments?

Q6. Does `[weak self]` always fix cycles? `(45s)`

Skeleton: Only if that edge was the cycle; nested captures may remain.

Q7. `deinit` not called — checklist? `(60s)`

Skeleton: Cycle, singleton ownership, still on screen, timer, observation.

Q8. Value types and ARC? `(30s)`

Skeleton: Structs aren't ARC'd; nested classes are.

Q9. Autorelease pools — when? `(45s)`

Skeleton: Tight loops creating many temporaries (ObjC bridging).

Q10. How do you prevent leaks in async Task? `(60s)`

Skeleton: Task retains; cancel on disappear; weak self in task body.

6. Tricky questions

T1. `unowned self` in async callback after VC dismissed `(90s)`

Trap: unowned for convenience.
Senior answer: Use weak; unowned crashes if callback outlives VC — common in network/ad SDK callbacks.

T2. NotificationCenter + closure observer cycle `(90s)`

Trap: Forget remove/token.
Senior answer: Store token; remove on deinit; capture list.

T3. Lazy var closure capturing self `(90s)`

Trap: Lazy init cycle.
Senior answer: Lazy closure can capture self strongly during init — careful patterns / weak.

T4. Collection of closures storing self `(120s)`

Trap: Only weak the outer one.
Senior answer: Each escaping closure needs capture discipline; prefer structured concurrency cancellation.

T5. Why Memory Graph shows cycle but Leaks doesn't `(90s)`

Trap: Tools are synonyms.
Senior answer: Leaks finds unreachable; cycles are reachable → Allocations/Graph.

T6. UIKit view retain via layer delegate `(90s)`

Trap: Only Swift closures matter.
Senior answer: CALayer.delegate strong quirks historically — know UIKit ownership edges.

7. Flashcards for today

Front	Back
ARC	Retain count via compiler · Trap: GC pauses · Prod: BMS reliability
weak	Optional zeroing · Trap: overuse unowned · Prod: network callbacks

unowned	Non-optional · Trap: outlive owner · Prod: nested owned child
Closure cycle	self↔closure · Trap: forget capture list · Prod: ad completion
Leaks vs Graph	Unreachable vs cycles · Trap: same tool · Prod: triage
Timer cycle	Target retains · Trap: never invalidate · Prod: live scoreboard ticks
Task retain	Task holds locals · Trap: fire-and-forget · Prod: search debounce
Abandoned memory	Still referenced · Trap: call it leak · Prod: nav stack caches
deinit missing	Cycle checklist · Trap: blame ARC bug · Prod: Crashlytics + Graph
Autorelease	Drain temporaries · Trap: always needed · Prod: image loops
99.95% CFS	Reliability bar · Trap: invent metrics · Prod: S8
Delegate weak	Break VC↔delegate · Trap: strong delegate · Prod: UIKit

8. Practice

- **Coding:** Deliberately create a retain cycle in a Playground/sample, prove with Memory Graph, then fix with `[weak self]`.
- **Story:** Record S8 in 2:30 focusing on triage process.

9. Timed drill

1. Q1, Q2, Q5, T1, T5 — record.
2. Score timing.
3. Gotchas log: any weak/unowned confusion.

Day 04 — GCD: Queues, sync/async, Barriers, Groups; Thread-Safe Structures

> *Week 1 · Phase: Concurrency (traditional) · Time budget today: ~4–5 hrs*

1. Outcome

Explain aloud:

- Serial vs concurrent queues; main vs global QoS
- `sync` vs `async` and the main-queue deadlock
- How a serial queue implements a thread-safe dictionary
- Reader-writer pattern with barrier on concurrent queue
- Exact BMS **synchronised dictionaries** story (S2) in ≤3 min

2. Concept deep dive

2.1 Queue types

Queue	Behavior
Main (serial)	UI work only
Serial private	Mutual exclusion for a resource
Concurrent global	Parallelism; QoS userInteractive→background
Concurrent private + barrier	Reader-writer

2.2 sync vs async

- `async` : schedule and return
- `sync` : wait for completion — **never `sync` to current queue** (deadlock), especially main

2.3 Thread-safe dictionary patterns

Pattern A — serial queue:

- writes: `async` to serial queue
- reads: `sync` to serial queue returning copy/value

Pattern B — concurrent queue + barrier:

- reads: `sync` concurrent
- writes: `async` flags `.barrier`

Pattern C — modern: Swift `actor` (Day 05)

2.4 DispatchGroup / semaphore

- Group: fan-out multiple tasks, notify when done (e.g. prefetch)
- Semaphore: limit concurrency (e.g. max 4 image decodes) — easy to misuse; prefer TaskGroup later

2.5 Trade-offs

Choice	When	Cost
Serial queue wrapper	Simple shared maps	Read latency under load
RW barrier	Read-heavy	Harder correctness
NSLock	Small critical sections	Priority inversion risk if careless
Actor	New Swift code	Migration cost

3. Read these

Priority	Resource	Why
Must	Dispatch	API truth
Must	Day notes + implement sync dict in Playground	Muscle memory
Deepen	cheatsheet.md concurrency snippets if any	Interview numbers
Repo	Skim race discussion in networking/offline specs as needed	Context

4. Map to your work

Company / feature: BookMyShow — synchronised dictionaries
What you did: Introduced GCD serial queues / read-write locks to eliminate race conditions and concurrent-access crashes in shared async state.
Interview line (≤20s): "We gated shared dictionaries behind a serial queue API so call sites couldn't race the storage — crashes went away on that path."

→ S2

5. Normal questions

Q1. Serial vs concurrent queue? `(30–45s)`

Skeleton: One-at-a-time vs parallel tasks.
Story: Sync dict uses serial.

Q2. async vs sync? `(30–45s)`

Skeleton: Fire-and-forget vs wait; deadlock risk.

Q3. Why not sync to main from main? `(45s)`

Skeleton: Deadlock — main waiting for itself.

Q4. How implement thread-safe dictionary? `(90–120s)` conceptual

Skeleton: Private serial queue; sync get; async/sync set.
Story: S2.

Q5. QoS levels — why care? `(45s)`

Skeleton: Priority/energy; don't starve UI.

Q6. DispatchGroup use case? `(45s)`

Skeleton: Parallel fetches then merge.

Q7. Barrier flag purpose? `(45s)`

Skeleton: Exclusive write on concurrent queue.

Q8. Main thread rule for UI? `(30s)`

Skeleton: UIKit/SwiftUI updates on main.

Q9. Race vs deadlock? `(45s)`

Skeleton: Unsynchronized shared mut vs waiting circle.

Q10. When semaphore over group? `(45s)`

Skeleton: Limit in-flight work.

Q11. `DispatchQueue.main.async` after network? `(30s)`

Skeleton: Hop to main for UI bind.

6. Tricky questions

T1. Read with `sync` + write with `async` on same serial queue — ordering? `(90s)`

Trap: Assume writes always visible.
Senior answer: Serial queue orders tasks; sync read waits for prior tasks; design API so callers don't assume wall-clock without sync.

T2. Concurrent queue + non-barrier write `(90s)`

Trap: "Concurrent is fine for dict."
Senior answer: Data race UB — need barrier or serial.

T3. sync from serial queue to itself via nested call `(90s)`

Trap: Only main deadlocks.
Senior answer: Any serial queue can deadlock on sync re-entry.

T4. RW lock vs serial queue — when? `(120s)`

Trap: Always RW.
Senior answer: Read-heavy large maps may benefit; complexity + writer starvation; at BMS serial queue was enough for crash fix.

T5. Priority inversion with locks `(90s)`

Trap: Ignore QoS.
Senior answer: Low priority holds lock needed by high; GCD QoS / lock strategies matter.

T6. Exposing internal queue to callers `(90s)`

Trap: Let callers `async` onto your queue freely.
Senior answer: Hide queue; expose safe API methods only (S2).

7. Flashcards for today

Front	Back
Serial queue	Mutex via queue · Trap: sync re-entry · Prod: S2 dicts
Concurrent + barrier	RW pattern · Trap: write without barrier · Prod: read-heavy caches
sync deadlock	Wait on current queue · Trap: only main · Prod: UI hops use async
DispatchGroup	Join parallel work · Trap: forget leave · Prod: prefetch
QoS	Priority/energy · Trap: always userInteractive · Prod: analytics batch
Race	Unsynchronized share · Trap: intermittent ignore · Prod: BMS crashes
Safe dict API	Hide storage · Trap: return mutable ref · Prod: S2
Semaphore	Limit concurrency · Trap: deadlock wait · Prefer TaskGroup later
Main UI rule	UI on main · Trap: parse JSON on main · Prod: search bind
async write / sync read	Common serial pattern · Trap: return unsafely · Prod: S2
Reader-writer	Parallel reads · Trap: writer starve · Prod: optional upgrade
Actor vs GCD	Language isolation · Trap: rewrite all now · Prod: new code actors

8. Practice

- **Coding:** Implement `Final class SafeDict<Key: Hashable, Value>` with serial queue; write a race test that fails on plain Dictionary and passes on SafeDict.
- **Story:** Record S2 in ≤3 min with trade-off "I'd consider actor today."

9. Timed drill

1. Q4 (2 min), Q2, T3, T4 — record.
2. Full S2 STAR once.
3. Score; gotchas.

Day 05 — async/await, Structured Concurrency, Tasks, Actors, Sendable

> *Week 1 · Phase: Modern concurrency · Time budget today: ~4–5 hrs*

1. Outcome

Explain aloud:

- async/await vs GCD callbacks; structured concurrency benefits
- Task, TaskGroup, cancellation propagation
- Actor isolation, reentrancy, `@MainActor`
- Sendable and why Swift 6 cares
- How you'd migrate BMS synchronised dictionaries toward an actor

2. Concept deep dive

2.1 async/await

Suspends without blocking a thread (cooperatively). Replace callback pyramids; surface errors with `throws`.

2.2 Structured concurrency

Parent task owns children; cancellation and errors propagate. Prefer `async let` / `withTaskGroup` over detached fire-and-forget.

API	Use
`Task { }`	Unstructured bridge from sync
`Task.detached`	Rare — independent priority/lifetime
`async let`	Fixed parallel children
`withTaskGroup`	Dynamic fan-out

2.3 Actors

Serialized access to mutable state. Methods are `async` from outside. **Reentrancy:** after `await` inside actor, state may change — don't assume continuity across awaits.

2.4 Sendable

Types safe to share across concurrency domains. Value types usually Sendable; classes need care (`@unchecked Sendable` is a smell unless proven).

2.5 Trade-offs

Choice	When	Cost
Actor for shared dict	New shared mutable state	Migration; API async
Keep GCD serial	Legacy stable module	Dual models in app
`@MainActor` VM	UI state	Don't do heavy work on main
Detached tasks	Truly independent	Lose structure

3. Read these

Priority	Resource	Why
Must	Concurrency	Full mental model
Must	WWDC: Meet async/await in Swift; Protect mutable state with Swift actors	Visual intuition

Deepen	Sendable	Swift 6 readiness
Repo	Social feed HLD section — note async image/pipeline	social-feed.md

4. Map to your work

Company / feature: BookMyShow — shared async state / sync dictionaries
What you did: Fixed races with GCD; interview narrative includes modern equivalent.
Interview line (≤20s): "Production fix was a serial-queue dictionary; greenfield I'd expose an actor with the same safe API surface."

→ S2 · search debounce tasks S3

5. Normal questions

Q1. async/await vs GCD? `(45–60s)`

Skeleton: Syntax + structured cancel/errors vs queues; both schedule work.

Q2. What is structured concurrency? `(45s)`

Skeleton: Hierarchy of tasks; cancel propagates.

Q3. How does Task cancellation work? `(60s)`

Skeleton: Cooperative — check `Task.isCancelled` / `try Task.checkCancellation()`.

Q4. Actor vs class + lock? `(60s)`

Skeleton: Compiler-enforced isolation vs manual.
Story: S2 migration pitch.

Q5. What is actor reentrancy? `(60–90s)`

Skeleton: Await suspends; other calls may run; re-validate state.

Q6. `@MainActor` purpose? `(30–45s)`

Skeleton: Isolate UI-affined state to main.

Q7. Sendable in one sentence? `(30s)`

Skeleton: Safe to share across concurrency domains.

Q8. `async let` vs TaskGroup? `(45s)`

Skeleton: Fixed vs dynamic number of children.

Q9. When Task.detached? `(45s)`

Skeleton: Rare independent work; usually avoid.

Q10. Replace callback URLSession with async? `(45s)`

Skeleton: `await session.data(for:)`; cancel via Task.

Q11. Priority / QoS with Tasks? `(45s)`

Skeleton: Task priority; inherit from parent; avoid starving UI.

6. Tricky questions

T1. Race across await inside actor method `(120s)`

Trap: "Actor means no races ever inside method."
Senior answer: Reentrancy — after await, fields may have changed; use local lets / re-check invariants.

T2. `@unchecked Sendable` on a class with mutable state `(90s)`

Trap: Silence the compiler.
Senior answer: Only if you manually synchronize; prefer actor.

T3. Starting Task in ViewModel without cancellation `(90s)`

Trap: Fire-and-forget search.
Senior answer: Store task; cancel on new keystroke / deinit — BMS search debounce.

T4. Calling MainActor from actor — deadlock myths `(90s)`

Trap: Actors deadlock like queues.
Senior answer: Different model; can still create logical waits; avoid sync bridges.

T5. GCD sync inside async context `(90s)`

Trap: Mix freely.
Senior answer: Can block thread pools; prefer async primitives end-to-end.

T6. Sendable across module boundaries with public classes `(120s)`

Trap: Ignore library types.
Senior answer: Annotate carefully; send values across; isolate references inside actors.

7. Flashcards for today

Front	Back
Structured concurrency	Parent owns children · Trap: detached everywhere · Prod: search tasks
Actor	Isolated mutable state · Trap: no reentrancy · Prod: S2 future
Reentrancy	State may change after await · Trap: assume continuity · Prod: token refresh
Sendable	Cross-domain safe · Trap: @unchecked casually · Prod: Swift 6
@MainActor	UI isolation · Trap: heavy work on main · Prod: VM
Task cancel	Cooperative · Trap: kill thread · Prod: debounce
async let	Fixed parallel · Trap: unbounded fanout · Prod: dual fetch
TaskGroup	Dynamic parallel · Trap: forget await all · Prod: prefetch
async vs GCD	Await + structure · Trap: throw GCD away blindly · Prod: mixed codebase
Detached	Independent lifetime · Trap: default choice · Prod: rare
Hop to main	await MainActor · Trap: sync main · Prod: bind UI
Migration pitch	Queue dict → actor · Trap: big-bang rewrite · Prod: S2

8. Practice

- **Coding:** Implement `actor SafeDict<Key: Hashable, Value: Sendable>` with get/set. Compare API to Day 04 GCD version.

- **SD:** Social feed HLD — list 4 client layers + where TaskGroup prefetch fits.

9. Timed drill

1. Q4, Q5, T1, T3 — record.
2. 90s answer: "How would you migrate synchronised dictionaries to actors?"
3. Score; gotchas.

Day 06 — DSA: Arrays, Strings, Two Pointers, Sliding Window + Catch-up

> *Week 1 · Weekend-style volume day · Time budget today: ~5–6 hrs*

1. Outcome

- Solve Easy/Medium array-string problems in Swift while narrating
- Open every problem with a **2–3 min approach** (timing guide)
- Identify two-pointer vs sliding-window vs prefix patterns quickly
- Catch up any shaky Week 1 topic (actors / ARC / POP)

2. Concept deep dive

2.1 Pattern cheatsheet

Pattern	Signals	Complexity target
Two pointers (opposite)	Sorted pair sum, palindrome	O(n)
Fast/slow	Cycle, middle	O(n)
Sliding window fixed	Exact k length	O(n)
Sliding window variable	Longest/shortest with constraint	O(n)
Prefix sums	Range sums, subarray sum	O(n) prep
Frequency map	Anagrams, counts	O(n)

2.2 "Say this first" script (60–90s)

> *"Constraints? Sorted? Duplicates? Mutate in place?*
> *Brute force would be … O(?).*
> *I'll use … because … .*
> *Edge cases: empty, single element, all same.*
> *Coding now."*

2.3 Swift-specific tips

- Prefer `Array` indices carefully (`startIndex`)
- `Character` vs `String` indexing costs — convert to `Array` of `Character` when random access needed
- Speak complexity for time **and** space

3. Read these

Priority	Resource	Why
Must	coding/dsa-track.md Week 1 list	Canonical problem set

Must	Timing guide § Coding approach	Communication grade
Deepen	Catch-up: Day 03 or 05 notes you marked weak	Retention

4. Map to your work

DSA rarely maps 1:1 to BMS — still use product intuition for examples:

- Debounced search ≈ cancel in-flight work (concurrency story, not algorithm)
- Listing windows ≈ pagination cursors (system design Week 2+)

Interview line: "I treat DSA as structured communication under uncertainty — clarify, brute, optimize, then code."

5. Normal questions (meta + warmups)

Q1. Explain two pointers `(30–45s)`

Skeleton: Two indices moving; sorted/palindrome use cases.

Q2. Sliding window vs two pointers? `(45s)`

Skeleton: Window maintains a range invariant; pointers may be broader.

Q3. When prefix sums? `(30s)`

Skeleton: Many range sum queries.

Q4. Why narrate before coding? `(30s)`

Skeleton: Interviewer grades process; catch wrong approach early.

Q5–Q12. Problem set (solve; each gets approach script)

Use full list in dsa-track; minimum today:

1. Two Sum (map)
2. Best Time to Buy/Sell Stock
3. Valid Palindrome
4. Container With Most Water
5. Longest Substring Without Repeating Characters
6. Maximum Subarray (Kadane)
7. Product of Array Except Self
8. 3Sum (if time)

For each: 2–3 min approach → code → test edges → complexity out loud.

6. Tricky questions

T1. Interviewer asks for O(1) space after you used a map `(90s)`

Trap: Panic rewrite silently.
Senior answer: State trade-off; ask if input can be sorted/mutated; propose alternative.

T2. String indexing in Swift feels O(n) — what do you do? `(90s)`

Trap: Pretend String is random-access.
Senior answer: Convert to `[Character]` or use indices carefully; mention cost.

T3. Off-by-one in window `(90s)`

Trap: Expand/shrink wrong.
Senior answer: Invariant comment; examples dry-run.

T4. Mutating array while iterating `(90s)`

Trap: Remove during for-in.
Senior answer: Two-pointer write index pattern.

T5. They want both brute and optimal coded `(120s)`

Trap: Only optimal.
Senior answer: Sketch brute verbally; code optimal; mention when brute OK for n≤20.

7. Flashcards for today

Front	Back
Two pointers opposite	Sorted pair / palindrome · O(n)
Sliding window variable	Longest with constraint · expand/shrink
Kadane	Max subarray · running reset
Prefix sum	Range sum O(1) after O(n)
Frequency map	Anagram / counts
Say first	Clarify→brute→opt→edges
Swift String index	Not random O(1) · use Array
Write pointer	In-place filter/dedup
Two Sum map	value→index · O(n)
Container water	Ends inward · O(n)
Catch-up rule	Weak topic > new LC hard
Communication > clever	Senior signal

8. Practice

- **Coding:** Complete minimum 6 problems from list in Swift.
- **Catch-up (60–90 min):** Re-read weakest of Days 01–05; redo 5 flashcards from that day.
- **SD light:** Write Social Feed API: cursor pagination request/response JSON sketch (15 min).

9. Timed drill

1. Pick 2 problems: full timed — 3 min approach + 25 min code each.
2. One conceptual from Day 05 (actor reentrancy) at 2 min.
3. Gotchas log: any pattern mis-ID.

Day 07 — Week 1 Revision + Mock Interview #1

> *Week 1 · Mock day · Time budget today: ~5–6 hrs*

1. Outcome

- Active-recall all Week 1 flashcards (≥80% correct)
- Deliver S2 (synchronised dictionaries) in ≤3 min at score ≥4
- Complete Mock #1: concurrency + memory deep dive with timed answers
- Close Social Feed LLD notes (cache + infinite scroll bullets)

2. Concept deep dive — revision map

Topic	Day	Must-nail
struct/class/COW/enum	01	Value semantics + payment states
POP/generics/type erasure	02	Ads pipeline rationale
ARC/cycles/Instruments	03	weak vs unowned; Graph vs Leaks
GCD sync dict	04	Serial API; deadlock
async/await/actors/Sendable	05	Reentrancy; cancel
DSA patterns	06	Window + two pointers

2.1 Mock #1 format (60–90 min)

Block	Time	Content
Warm-up defs	10 min	5× 45s definitions from Weeks 1
Deep dive	25 min	Concurrency + memory (interviewer-style follow-ups)
Story	10 min	S2 full STAR + "actor migration" follow-up
Mini SD	20 min	Social feed: clarify + HLD only
Retro	10–15 min	Score with timing rubric

Partner script (or self): ask Normal then Tricky from Days 03–05 without letting notes show.

3. Read these

Priority	Resource	Why
Must	flashcards/week-01.md	Active recall
Must	timing guide	Scoring
Must	S2 story	Mock centerpiece
Repo	social-feed.md LLD sections	Finish Week 1 SD

4. Map to your work

Today's narrative spine is **reliability under concurrency at BMS scale** (30L+ DAU, crash-free culture) — S2 + S8 adjacent.

Interview line: "Week 1 theme: I don't just know GCD and actors — I've shipped race fixes on a large consumer app and can discuss migration paths."

5. Normal questions (mock warm pool)

Pull live from Days 01–05. Priority set:

1. struct vs class (45s)
2. COW (45s)
3. weak vs unowned (45s)
4. serial vs concurrent (45s)
5. async/await vs GCD (60s)
6. thread-safe dictionary design (2 min)

7. actor isolation (60s)
8. Sendable (45s)
9. retain cycle examples (60s)
10. Main-queue deadlock (45s)
11. Task cancellation (60s)
12. POP in ads pipeline (60s)

6. Tricky questions (mock deep pool)

1. Actor reentrancy after await
2. sync to serial queue re-entry deadlock
3. Memory Graph cycle vs Leaks tool
4. `@unchecked Sendable` ethics
5. Type erasure cost in renderer
6. COW uniqueness traps
7. Mixing GCD sync inside async contexts
8. Migrating SafeDict GCD → actor without big-bang

7. Flashcards for today

Use full Week 1 deck. Quick meta cards:

Front	Back
Mock agenda opener	“Defs → concurrency deep dive → story → feed HLD”
S2 time box	≤3 min STAR
Score 5 means	On time + trade-off + prod proof
Weak topic rule	Re-drill before Week 2
SD clarify first	DAU, offline, pagination
No new topics	Revision only today

8. Practice

- **Morning:** Flashcards full deck + rewrite gotchas cleanly
- **Mid:** Mock #1 with peer/self recording
- **Late:** Social feed LLD bullets: cursor pagination, image cache tiers, prefetch cancellation
- **Coding light:** Re-solve 2 misses from Day 06

9. Timed drill

1. Full Mock #1 with timer visible.
2. Score every answer 1–5.
3. List top 5 weak cards → pin to Week 2 daily warm-up.
4. Deliver S1 Ads POP talk once in 5 min (preview Mock #2) — optional if energy allows.

Mock #1 pass criteria

- Average ≥3.5 on deep-dive answers
- S2 ≥4
- No answer >2× budget without self-correction
- At least one explicit trade-off in concurrency discussion

Revision Week 02

Day 08 — MVVM / Clean / MVI · Layering · DI

> *Week 2 · Phase: Architecture, networking, SDUI, UI · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- When to pick **MVVM vs Clean vs MVI**, and what each layer owns vs must never own
- How **dependency injection** (constructor / protocol / factory) makes features testable without a DI "framework religion"
- How you migrated District patterns (**MVVM ↔ Clean**) with Context Engineering without skipping design ownership
- How BMS search sits in MVVM: debounce, cancellation, explicit UI states — and how you'd defend that in interview

2. Concept deep dive

2.1 Pattern map (say this in 45s)

Pattern	Core idea	UI binding	Best fit
MVC	VC owns too much	Imperative	Tiny screens; legacy UIKit
MVVM	View observes state; VM holds presentation logic	Bindings / Combine / Observation	Feature screens, search, forms
Clean	Use cases + entities independent of UI/framework	VM or Presenter at edge	Multi-team modules, migrations, heavy domain
MVI / unidirectional	Intent → Reducer → State → View	Single state stream	Complex multi-source UI (checkout, live)

Interview line: "I default to MVVM for feature UI. I introduce Clean boundaries when domain rules, reuse, or migration risk demand an explicit UseCase layer. I use unidirectional state when many async sources fight over one screen."

2.2 Layering that interviewers expect

```
View (SwiftUI / UIKit)
  → ViewModel / Presenter  (UI state, mapping, user intents)
    → UseCase / Interactor (optional; business rules, orchestration)
      → Repository         (cache + remote policy)
        → DataSource       (URLSession, DB, CMS)
```

Rules of thumb:

- Views: layout + forward events. No networking. No business rules.
- ViewModels: map domain → display models; hold loading/empty/error; cancel in-flight work on disappear / new query.
- UseCases: "why" of the product (e.g. Free Parking billing adjustment rules) — pure enough to unit test without UI.
- Repositories: decide cache-vs-network; hide DTO decoding; never leak URLSession to VMs.

2.3 DI without dogma

Technique	When	Cost
Constructor injection of protocols	Default for VMs / UseCases	Slightly more init boilerplate
Factory / Assembler per feature	Feature modules, SPM boundaries	Need clear ownership of graph

Service locator / shared singleton	Avoid for domain; ok for app-scoped analytics?	Hidden deps, hard tests
SwiftUI `.environment` / Observation	UI-tree plumbing	Don't put networking clients only in Environment without protocols

Senior stance: Protocols at **boundaries** (NetworkClient, SearchRepository, Analytics). Concrete types inside the module. Fake in XCTest. District: AI helped generate tests **after** protocols existed — not instead of them.

2.4 MVVM ↔ Clean migration (District playbook)

Incremental migration beats big-bang:

1. Freeze feature contracts (APIs, analytics events, nav).
2. Extract UseCase from fat VM **one flow at a time** (Free Parking billing adjustments).
3. Keep VM as adapter: still owns UI state; now calls UseCase.
4. Add XCTest on UseCase first (cheapest confidence), then VM state tests.
5. Context Engineering: feed Cursor/Claude **boundary docs + existing patterns**, review every diff; you own regressions.

2.5 Trade-offs

Choice	When	Cost
Fat MVVM only	Single team, simple screens	VMs become god objects
Clean everywhere on day 1	Rarely	Ceremony slows small features
MVI for every list	Overkill	Boilerplate; harder junior onboarding
Shared AppCoordinator singleton for all DI	Fast start	Untestable coupling at scale
Protocol + constructor DI	Default senior choice	Explicit wiring

3. Read these

Priority	Resource	Why
Must	Day 08 notes above + app-modularization.md (boundaries)	Language for layers in SD / deep-dive
Deepen	Apple: Testing · WWDC: Data Essentials in SwiftUI (state ownership)	Test + state ownership vocabulary
Repo	search-autocomplete.md	Ties MVVM search to system design
Repo	stories/story-bank.md#s9--free-parking--cleanmvvm--ai-tooling-district · S3#s3--backend-driven-header--search-bookmyshow	Production hooks

4. Map to your work

Company / feature: District — Free Parking + Clean/MVVM migration with Context Engineering
What you did: Shipped Free Parking billing adjustments; migrated patterns across MVVM and Clean; used AI inside a strict architectural envelope (reviews + XCTest/XCUITest), owned on-call fixes.
Interview line (≤20s): "At District I shipped Free Parking while migrating MVVM↔Clean incrementally — AI accelerated scaffolding inside protocols I owned, with tests as the gate."

→ Full STAR: stories/story-bank.md#s9--free-parking--cleanmvvm--ai-tooling-district

Secondary map — BMS search: Debounced MVVM search with explicit loading/empty/error; race-safer UX on high-traffic surface.
→ stories/story-bank.md#s3--backend-driven-header--search-bookmyshow

5. Normal questions

For each: answer out loud within the time. Skeleton only — expand from memory.

Q1. Explain MVVM vs MVC. `(30–45s)`

Skeleton: MVC VCs accumulate networking + state → hard tests. MVVM: View renders state, VM owns presentation + async orchestration, Model/domain separate. UIKit: bindings/closures/Combine; SwiftUI: Observation/ `@State` / `@Bindable` .
Follow-up: Where does networking live? → Repository behind protocol, not in View.
Story: S3 search VM; S9 migration away from fat layers.

Q2. When do you introduce Clean Architecture? `(45–60s)`

Skeleton: When domain rules are non-trivial, shared across screens, or you're migrating safely. Entities + UseCases independent of UIKit/SwiftUI. UI adapters stay thin. Don't Clean-ify a settings toggle.
Follow-up: How thin is the VM then? → Maps UseCase output → view state; still cancels tasks.
Story: S9 Free Parking + migration.

Q3. What is MVI / unidirectional data flow? `(45s)`

Skeleton: User Intent → reducer/processor → immutable State → View. Side effects explicit. Great when checkout/live scoreboard has many async inputs; heavier than MVVM for CRUD.
Follow-up: Relation to TCA? → Same family; discuss trade-off of framework vs hand-rolled.
Story: Mentally link S7 payment states / S11 live scoreboard as "state machine" cousins.

Q4. How do you do DI on iOS without a container? `(45–60s)`

Skeleton: Protocol boundaries + constructor injection. Feature factory builds graph. Tests inject fakes. Avoid service locators for domain. SwiftUI Environment for UI-scoped deps carefully.
Follow-up: Circular dependencies? → Split protocols; invert ownership toward UseCase.
Story: S9; S10 Stories SDK public API isolation.

Q5. How would you structure BMS search in MVVM? `(60–90s)`

Skeleton: `SearchViewModel` : query subject → debounce (~300ms) → cancel previous `Task` → repository → states: idle/loading/results/empty/error. View binds state only. Ranking stays server-side unless product needs local filter.
Follow-up: Why debounce + cancel? → Avoid out-of-order responses; save battery/network at 30L+ DAU scale.
Story: S3#s3--backend-driven-header--search-bookmyshow

Q6. Fat ViewModel — smells and fix? `(45s)`

Skeleton: Smells: networking strings, analytics dumps, routing, business rules all in VM. Fix: extract UseCase + Router/Coordinator + AnalyticsClient protocol.
Follow-up: Migration order? → Extract UseCase under test first.
Story: S9.

Q7. How do View and ViewModel communicate in UIKit vs SwiftUI? `(45s)`

Skeleton: UIKit: closures, Combine `PassthroughSubject` , delegates for one-off. SwiftUI: `@Observable` / `@Bindable` , or published state. Prefer state down, events up.
Follow-up: Retain cycles? → `[weak self]` in UIKit closures; Observation reduces classic Combine cycle footguns but still careful with long-lived tasks.
Story: S13 hybrid if asked UIKit hosting.

Q8. How did you use AI in the District migration without losing seniority? `(60s)`

Skeleton: Context Engineering: feed architecture constraints, existing module patterns, acceptance tests. AI drafts migrations/tests; you review for boundary violations, naming, race conditions. Never "AI wrote the app."
Follow-up: What did AI get wrong? → Be ready with a concrete miss (wrong layer, missing cancellation, flaky UI test).
Story: S9 — **critical interview story**.

Q9. Repository vs UseCase — difference? `(30–45s)`

Skeleton: UseCase = application policy ("adjust Free Parking billing"). Repository = data policy (cache TTL, remote fetch, mapping DTOs). VM shouldn't know URLSession; UseCase shouldn't know UIKit.
Follow-up: Can UseCase call multiple repos? → Yes, orchestration is the point.
Story: S9.

Q10. How do you keep feature modules testable? `(45–60s)`

Skeleton: Protocols at edges, pure UseCases, deterministic fakes, snapshot/UI tests sparingly for critical flows, CI gates. District: XCTest/XCUITest generation inside review loop.
Follow-up: What do you not mock? → Prefer fakes over heavy mocks; don't mock value types needlessly.
Story: S9; S1 ads protocols for testability.

Q11. Coordinator vs Router vs NavigationPath? `(45s)`

Skeleton: Coordinator/Router owns cross-feature nav side effects so VM stays testable. SwiftUI: `NavigationPath` / deep-link handlers at app edge (Grizzlies). Don't put `UINavigationController` pushes inside UseCases.
Follow-up: Deep links? → Parse at app boundary → intent → coordinator.
Story: S13.

Q12. How does architecture relate to crash-free at 30L+ DAU? `(45–60s)`

Skeleton: Clear ownership + fail-soft states + fewer god objects → fewer nil races and lifecycle bugs. Architecture doesn't replace Crashlytics/IMOC — it reduces blast radius.
Follow-up: Example fail-soft? → Search error state; SDUI unknown component skip (preview Day 10).
Story: S8 reliability culture + S3 states.

6. Tricky questions

T1. "Isn't Clean Architecture overengineering for mobile?" `(90–120s)`

Trap: Binary yes/no. Seniors scope it.
Senior answer: Overkill for every screen. Worth it when domain + multi-team + migration risk. At District we extracted UseCases where billing rules lived; left simple UI as MVVM. Measure ceremony vs regression cost.
Follow-up: Show a screen you'd *not* Clean-ify.

T2. "Where should debounce live — View, VM, or Repository?" `(90s)`

Trap: Putting it only in UIControl or only in network layer.
Senior answer: Presentation policy → **VM** (or UseCase if product rule). Repository shouldn't know keystroke timing. Network layer may still cancel tasks. BMS: debounce in search VM + cancel in-flight.
Follow-up: Different debounce for analytics vs search? → Separate pipelines.

T3. "MVVM doesn't work with SwiftUI — discuss." `(90–120s)`

Trap: Claiming SwiftUI kills VM.
Senior answer: SwiftUI wants clear state ownership; VM/ `@Observable` models still useful for async, testing, and sharing. Avoid duplicating every `@State` into VM. Prefer view-local state for pure UI; hoist async/domain.
Follow-up: `@Observable` vs `ObservableObject` — Day 12 preview.

T4. "How do you migrate a live app from MVC to Clean without a rewrite?" `(120s)`

Trap: Big-bang rewrite.
Senior answer: Strangler: new features on new boundaries; extract UseCases behind old VC; characterization tests; feature flags; District-style incremental MVVM↔Clean. Ship Free Parking value while migrating.
Follow-up: Rollback plan? → Keep old path behind flag.

T5. "Dependency injection frameworks on iOS — yes/no?" `(90s)`

Trap: Tool advocacy without trade-offs.
Senior answer: Optional. Prefer manual constructor DI + factories until graph pain is real. Frameworks help large graphs; cost is magic and compile/runtime opacity. Protocols first.
Follow-up: How Stories SDK exposes DI to host apps? → Inject networking/analytics via protocols (S10).

T6. "Who owns navigation — VM or Coordinator?" `(90s)`

Trap: Absolute rule.
Senior answer: Simple push: VM can emit `NavigationEvent`. Cross-feature, auth gates, deep links: Coordinator/Router. Test VMs by asserting events, not by pushing VCs.
Follow-up: LE Bottom Sheet — presented UI vs full nav (S6).

T7. "How do you prevent AI-generated Clean Architecture from becoming nonsense layers?" `(90–120s)`

Trap: Either ban AI or praise it blindly.
Senior answer: Envelope: documented layering, example PRs, checklist (no UIKit in UseCase, no URLSession in VM). AI proposes; human reviews boundaries; tests must fail for wrong layer. Context Engineering is the control plane.
Follow-up: Metric of success? → Regression rate + review cycle time, not LOC generated.

7. Flashcards for today

Front	Back
MVVM one-liner	View renders state; VM presentation + async; Model domain · Trap: networking in View · Prod: BMS search VM
Clean one-liner	UseCases/entities independent of UI · Trap: Clean everywhere · Prod: District Free Parking rules
MVI one-liner	Intent → reduce → State → View · Trap: boilerplate for CRUD · Prod: payment/live state cousins
Where debounce lives	VM/UseCase presentation policy · Trap: only in URLSession · Prod: BMS search
DI default	Protocol + constructor injection · Trap: service locator for domain · Prod: testable VMs
Repository duty	Cache/remote + DTO map · Trap: business rules in repo · Prod: search repository
Fat VM smell	Networking + routing + rules · Fix: UseCase + Router · Prod: S9 extract
AI in migration	Accelerate inside envelope + review · Trap: "AI wrote it" · Prod: S9 Context Engineering
Navigation ownership	Events from VM; Coordinator for cross-feature · Trap: UIKit in UseCase · Prod: S13 deeplinks
Fail-soft UI states	loading/empty/error/success · Trap: spinner forever · Prod: BMS search
When not Clean	Trivial UI, no shared domain · Trap: resume padding · Prod: judgment call
Test pyramid tip	UseCase unit tests cheapest · Trap: only UITests · Prod: District XCTest
Strangler migration	Incremental extract, flags · Trap: rewrite · Prod: District
Protocol boundary	Network/Search/Analytics · Trap: protocol every struct · Prod: S1/S10
Scale link	Clear layers reduce blast radius · Trap: architecture = crash-free alone · Prod: 30L DAU + S8

8. Practice

- **Coding / SD:** Sketch on paper: `SearchViewModel` API (state enum, `onQueryChange`, cancel). Then outline District Free Parking UseCase inputs/outputs without UIKit.
- **Complexity / agenda to say first:** "I'll define layers, where debounce/cancel live, then map to BMS search and District migration trade-offs."

- **Optional:** 15 min read of search-autocomplete.md HLD — note where VM vs repository sit.

9. Timed drill

1. Pick 3 Normal + 2 Tricky (recommended: Q5, Q8, Q2 + T1, T7). Record.
2. Score against answer-timing-guide.md.
3. Log misses in gotchas (especially AI-ownership wording and debounce placement).
4. Deliver S9 STAR in **2–3 min** once; opener only in **20s** once.

Day 09 — Networking: URLSession, Interceptors, Token Refresh, Caching, Cancellation

> *Week 2 · Phase: Architecture, networking, SDUI, UI · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- How to design a **protocol-oriented networking layer** on `URLSession` (endpoints, interceptors, decoding, errors)
- **Token refresh** under concurrency (single-flight refresh, queue/retry, failure fan-out)
- **Cancellation**, retries/backoff, and **caching** responsibilities (HTTP cache vs app cache)
- Why/how you migrated Ads networking **Alamofire → URLSession** with **HTTPS, SSL pinning, domain whitelist** at BMS

2. Concept deep dive

2.1 Layer shape (interview HLD in 60s)

```
APIEndpoint (path, method, headers, body, auth requirement)
    → RequestBuilder → URLRequest
        → Interceptors (auth inject, logging, tracing)
            → URLSession data/upload task
                → Response pipeline (status map, decode, domain errors)
                    → TokenRefresher (on 401) → retry once
```

Speak to: generics `request<T: Decodable>(_:) async throws -> T`, mockable `NetworkSession` protocol wrapping `URLSession`, and **no** Alamofire requirement for senior answers — first-party control matters for pinning and Ads.

Primary doc: networking-layer.md

2.2 Interceptors

Interceptor	Does	Must not
Auth	Attach Bearer / custom headers	Block UI thread
Logging	Safe metadata (path, status, latency)	Log tokens / PII
Tracing	Firebase Performance spans (preview Week 3)	Become business logic
Retry	Idempotent GETs with backoff	Blindly retry POSTs (payments!)

Composition: ordered pipeline; short-circuit on hard failures.

2.3 Token refresh (single-flight)

Problem: N parallel 401s → N refresh calls → thundering herd / revoked refresh tokens.

Pattern:

1. Detect 401 / auth-expired error.
2. First caller starts refresh; others **await the same Task**.
3. On success: update Keychain-backed token store; retry original requests **once**.
4. On failure: fail all waiters; force logout / re-auth path.
5. Actor or serial queue around refresh state.

Say aloud: "Refresh is a critical section; retries are bounded; payments use stricter idempotency keys."

2.4 Cancellation & task identity

- Prefer `async` + `Task` cancelled in `onDisappear` / new search query (ties to Day 08).
- `URLSessionTask.cancel()` / structured concurrency cancellation propagation.
- Ignore `CancellationError` in UI (don't show as hard error).
- Search autocomplete: cancel stale in-flight when query changes.

2.5 Caching — split responsibilities

Layer	Mechanism	Use
URLCache / HTTP Cache-Control	Transport cache	Static-ish GETs
App memory (NSCache)	Hot decoded models	Session UX
Disk (files / DB)	Offline / SDUI fallback	Day 10
"No cache"	Auth'd personalized / payments	Correctness

Trap: Caching authenticated personalized responses without keying by user.

2.6 Alamofire → URLSession (BMS Ads)

Motivations: dependency surface, first-party pinning hooks, consistent Ads pod networking.

Migration checklist you can recite:

1. Introduce `NetworkClient` protocol matching call sites.
2. Implement URLSession backend; parity tests (status, decode, error mapping).
3. Enforce HTTPS + **SSL pinning** + **domain whitelist**.
4. Document **pin rotation** / break-glass (pinning without ops = outage).
5. Shadow traffic or staged rollout on Ads module; watch Crashlytics + fill metrics.

2.7 Trade-offs

Choice	When	Cost
URLSession first-party	Security-sensitive modules (Ads)	More boilerplate than Alamofire
Alamofire	Rapid CRUD apps, team fluency	Less control; dependency risk
Aggressive HTTP cache	Public content	Stale personalized data risk
Retry all methods	Never	Duplicate charges / side effects
Pinning	High-threat / high-value traffic	Rotation + outage risk if mis-ops

3. Read these

Priority	Resource	Why
Must	URLSession	Baseline API

Must	networking-layer.md	Full HLD/LLD for interviews
Deepen	mobile-security-privacy-engine.md (pinning section)	Pair with S4
Deepen	authentication-oauth-biometric.md	Refresh/token storage
Repo	stories/story-bank.md#s4--ssl-pinning--alamofire--urlsession-bookmyshow	Production hook

4. Map to your work

Company / feature: BookMyShow — Ads networking Alamofire → URLSession + SSL pinning + domain whitelist
What you did: Migrated Ads pod networking to URLSession; enforced HTTPS, pinning, whitelist; documented rotation/failure behavior for ops.
Interview line (≤20s): "I moved Ads off Alamofire onto URLSession so we owned pinning and whitelist on a high-traffic revenue module — with an explicit pin-rotation plan."

→ Full STAR: stories/story-bank.md#s4--ssl-pinning--alamofire--urlsession-bookmyshow

Also link: Search debounce/cancel (S3); Firebase perf traces on journeys (S5) as observability on the same stack.

5. Normal questions

For each: answer out loud within the time. Skeleton only — expand from memory.

Q1. Why prefer URLSession over Alamofire in a senior design? `(30–45s)`

Skeleton: First-party control, fewer deps, direct `URLSessionDelegate` for pinning/challenges, async/await native. Alamofire fine for speed; Ads needed security ownership.
Follow-up: What did you lose? → Convenience adapters; you reimplemented needed pieces.
Story: S4.

Q2. Sketch a generic request API. `(45–60s)`

Skeleton: `protocol APIEndpoint` ; `NetworkClient.request<T: Decodable>(_:) async throws -> T` ; map HTTP status → domain `NetworkError` ; inject session for tests.
Follow-up: Upload/download? → Separate task types; progress callbacks.
Story: Point to networking-layer.md components.

Q3. How do auth interceptors work? `(45s)`

Skeleton: Before send: read token store, attach header. On 401: refresh path. Never log secrets.
Follow-up: Guest vs authed endpoints? → Endpoint flag `requiresAuth` .
Story: S4 stack; general auth design.

Q4. Explain safe token refresh under concurrency. `(60–90s)`

Skeleton: Single-flight Task/actor; waiters share result; retry once; failure → logout all; store tokens in Keychain.
Follow-up: Refresh token reuse detection? → Server revoke → force re-login.
Story: Architecture answer + security discipline from S4.

Q5. How do you cancel in-flight search requests? `(45s)`

Skeleton: New query cancels previous `Task` ; treat cancellation as non-error; debounce in VM. Prevents out-of-order apply.
Follow-up: URLSession task vs Task cancellation? → Prefer structured concurrency wrapping session calls.
Story: S3.

Q6. Retry policy — what do you retry? `(45–60s)`

Skeleton: Transient 408/429/5xx on **idempotent** GETs; exponential backoff + jitter; cap attempts. No blind POST retry for payments (use idempotency keys).

Follow-up: 429? → Honor Retry-After when present.
Story: S7 payments caution.

Q7. HTTP cache vs app cache? `(45s)`

Skeleton: URLCache honors headers; app cache for decoded models/offline. Key by user for private data. SDUI layouts often disk+TTL (Day 10).
Follow-up: Cache invalidation? → TTL + pull-to-refresh + version keys.
Story: S3 header CMS freshness.

Q8. What is SSL pinning and what goes wrong? `(60s)`

Skeleton: Pin public key/cert SPKI; reject unexpected server certs → mitigate MITM. Failure mode: expired pin → total outage. Need rotation, backup pins, monitoring.
Follow-up: ATS vs pinning? → ATS baseline; pinning extra.
Story: S4.

Q9. Domain whitelisting — why? `(30–45s)`

Skeleton: Ads/network client only talks to approved hosts; reduces SSRF-ish misconfig and unexpected third-party calls from CMS mistakes.
Follow-up: How enforced? → Central request builder validation before resume.
Story: S4.

Q10. How do you test networking without flaky CI? `(45–60s)`

Skeleton: Protocol-wrap session; return fixture Data; test decode + 401 refresh state machine with fakes; few integration smoke tests.
Follow-up: Recorded traffic? → Optional; keep secrets out of fixtures.
Story: S9 test discipline; S4 migration parity.

Q11. Error mapping you use in production? `(45s)`

Skeleton: Transport (offline/timeout), HTTP (4xx/5xx), decoding, cancelled, auth. UI maps to retryable vs fatal. Don't show raw JSON errors.
Follow-up: Partial success? → Domain-specific; don't hide in generic client.
Story: S3 empty/error states.

Q12. How does networking relate to p50/p90? `(45s)`

Skeleton: Trace request spans (Firebase Performance); optimize slow endpoints; client timeouts must align with backend SLOs. Averages lie — watch p90.
Follow-up: Client-only fixes? → Caching, prefetch, cancel waste.
Story: S5.

6. Tricky questions

T1. "N requests get 401 at once — design refresh." `(120s)`

Trap: Refresh per request.
Senior answer: Single-flight refresher; queue/await; one retry; mutex/actor; if refresh 401 → global logout; prevent refresh storm. Draw the sequence.
Follow-up: What if refresh succeeds but original POST isn't idempotent? → Don't auto-retry unsafe methods without idempotency key.

T2. "Should the networking SDK own disk cache?" `(90s)`

Trap: Kitchen-sink SDK.
Senior answer: Transport-level cache optional; domain/offline cache usually Repository/SDUI FallbackEngine. Keep SDK focused:

build, auth, execute, decode, cancel.
Follow-up: Point to networking-layer.md out-of-scope notes.

T3. "Pinning broke production after cert rotate — your fault?" `(90–120s)`

Trap: Blame-only or "pinning bad."
Senior answer: Pinning without rotation runbook is incomplete design. Backup pins, staged mobile config remote pin set if possible, monitoring of TLS failures, break-glass disable flag carefully scoped. S4 lesson: design ops path.
Follow-up: How detect early? → Canary + TLS failure metrics.

T4. "Alamofire already supports interceptors — why migrate?" `(90s)`

Trap: NIH syndrome.
Senior answer: Ads needed first-party pinning/whitelist control, dependency reduction, consistent pod ownership. Migration cost justified by security + control on revenue module — not ideology.
Follow-up: Would you migrate whole BMS app? → Risk-based; start where threat/value highest.

T5. "How do you cancel URLSession work with async/await correctly?" `(90s)`

Trap: Fire-and-forget Task.
Senior answer: Hold Task handle; cancel on lifecycle; use `withTaskCancellationHandler` / check `Task.isCancelled`; map cancellation to silent UI; ensure session task cancelled too.
Follow-up: Multiple screens sharing one request? → Shared publisher/Task with refcount or repository-level coalescing.

T6. "Retry + payment checkout." `(90–120s)`

Trap: Generic retry helper on all routes.
Senior answer: Payments: idempotency keys, explicit status polling (S7 popup states), no duplicate charge from client retry. Networking SDK should let endpoints opt into retry policy.
Follow-up: Tie to payment-checkout.md briefly.

T7. "Man-in-the-middle on public Wi-Fi — what layers help?" `(90s)`

Trap: Only "use HTTPS."
Senior answer: ATS/HTTPS baseline, certificate transparency ecosystem, pinning for high-value, whitelist, no cleartext fallback, careful WebViews. User education ≠ engineering control.
Follow-up: S4 Ads threat model.

T8. "Where do you put Firebase Performance tracing — interceptor or call site?" `(90s)`

Trap: Only one right answer.
Senior answer: Interceptor for baseline latency by path; call sites/journeys for product traces (listing→checkout). Avoid double-counting; PII-safe names.
Follow-up: S5 p50/p90 culture.

7. Flashcards for today

Front	Back
Networking SDK core	Endpoint → build → intercept → session → decode · Trap: UI in client · Prod: Ads URLSession
Single-flight refresh	One refresh Task; waiters share · Trap: N refreshes · Prod: auth storms
Cancel search	Cancel Task on new query · Trap: show cancel as error · Prod: S3
Retry safe?	Idempotent GET + backoff · Trap: POST payments · Prod: S7
URLCache vs app cache	Headers vs domain/offline · Trap: cache private unkeyed · Prod: SDUI Day 10
SSL pinning	SPKI pin; MITM ↓ · Trap: no rotation = outage · Prod: S4

Domain whitelist	Only approved hosts · Trap: CMS open fetch · Prod: S4
Why leave Alamofire	Control + deps + pinning · Trap: hate libraries · Prod: S4 Ads
NetworkError kinds	Transport/HTTP/decode/cancel/auth · Trap: raw strings to UI · Prod: S3 states
Interceptor order	Auth → log/trace → send · Trap: log tokens · Prod: hygiene
Test strategy	Fake session + fixtures · Trap: live network CI · Prod: migration parity
Idempotency key	Safe payment retries · Trap: client-only duplicate POST · Prod: checkout
p90 networking	Trace spans; optimize tail · Trap: mean latency · Prod: S5
Break-glass pin	Monitored failover plan · Trap: silent disable forever · Prod: S4 lesson
Protocol NetworkSession	Inject URLSession mock · Trap: concrete everywhere · Prod: testability

8. Practice

- **Coding / SD:** On a whiteboard, draw token-refresh sequence for 3 parallel 401s. Then list Ads migration steps (protocol → URLSession → pin → whitelist → rotate runbook).
- **Complexity / agenda to say first:** "I'll scope the client, then auth/refresh, cancellation, cache boundaries, then BMS Alamofire→URLSession + pinning trade-offs."
- **Deep read:** Skim full networking-layer.md HLD + NFRs (30–40 min).

9. Timed drill

1. Pick 3 Normal + 2 Tricky (recommended: Q4, Q8, Q5 + T1, T3). Record.
2. Score against answer-timing-guide.md.
3. Log misses in gotchas (refresh concurrency + pin rotation).
4. Deliver S4 STAR in **2–3 min**; practice **5 min** networking HLD using the repo doc as checklist (prep for Mock #2 / Mock #3).

Day 10 — SDUI / CMS · Schema Versioning · Fallbacks

> *Week 2 · Phase: Architecture, networking, SDUI, UI · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- What **server-driven UI** is (and is not): schema, registry, actions, analytics — not "eval JSON as code"
- **Schema versioning**, unknown-component policy, and **fallback/cache** strategies that protect crash-free sessions
- How BMS **backend-driven header** + search CMS contracts worked in production
- How Aces **server-driven splash** improved cold-start flexibility / freshness

2. Concept deep dive

2.1 SDUI mental model (60s opener)

```
CMS / Backend  --JSON schema-->  Parser + Version gate
                                        ↓
                                ComponentRegistry (type → native view)
                                        ↓
                                LayoutResolver + ActionHandler
                                        ↓
                                SwiftUI/UIKit render + analytics
```

```
                                ↕
                FallbackEngine (disk cache / last-known-good)
```

Primary doc: [sdui-engine.md]

SDUI wins: ship content/layout experiments without App Store; personalize surfaces; share contracts across Android/iOS.

SDUI costs: schema discipline, client capability matrix, QA of combinations, offline/fallback, security of actions (no arbitrary code).

2.2 Schema & versioning

Strategy	Behavior
`schemaVersion` major/minor	Major mismatch → reject or safe fallback layout
Capability flags	Client advertises supported components
Unknown component	**Skip + log** (never crash)
Additive fields	Minor-compatible; clients ignore unknowns
Breaking change	New major + dual-publish period

Interview line: "Compatibility is a product feature. Unknown nodes fail soft; known nodes validate strictly."

2.3 Fallbacks (crash-free insurance)

1. **Network fail:** show last-known-good cached layout (TTL + freshness indicator if needed).
2. **Parse/version fail:** baked-in default header/splash.
3. **Partial fail:** render known children; skip bad nodes.
4. **Action fail:** no-op + analytics; never hard-crash on deep link miss.

At **30L+ DAU** and **99.95% crash-free**, SDUI without fallbacks is an incident generator.

2.4 Actions & security

Server can send `action: { type, payload }` (deeplink, open URL, refresh module). Client **allowlists** action types. Never execute scripts from CMS. Validate URLs against domain policy (ties to Day 09 whitelist mindset).

2.5 BMS header + Aces splash

BMS: Generalised protocol-driven main header from backend/CMS; paired with MVVM search (debounce/states). Faster content iteration without release for many header changes.

Aces: Server-driven splash to cut cold-start latency / improve freshness alongside audio streaming work — startup is a product surface (time-to-interactive).

2.6 Trade-offs

Choice	When	Cost
Full SDUI home	Super-app / high experiment velocity	QA matrix explosion
Hybrid (CMS slots + native chrome)	Most consumer apps	Need clear slot contracts
Native-only	Highly regulated / rare change	Slow iteration
Skip unknown components	Always for resilience	Possible visual gaps — measure
Disk cache splash/header	Cold start + offline	Stale content risk — version + TTL

3. Read these

Priority	Resource	Why
Must	sdui-engine.md	Interview spine for SDUI
Deepen	feature-flag-system.md · ab-testing-experimentation-sdk.md	Experiments vs SDUI overlap
Deepen	search-autocomplete.md	Header/search surface
Repo	S3 · S12 in story-bank.md	BMS header; Aces splash

4. Map to your work

Company / feature: BookMyShow — backend-driven header & search
What you did: Protocol-driven generalised main-screen header from CMS/backend; search debounce + MVVM states; API contracts for layout/content changes without release when possible.
Interview line (≤20s): "I made the BMS main header backend-driven with a protocolised client and fail-soft rendering, plus race-safer debounced search."

→ Full STAR: stories/story-bank.md#s3--backend-driven-header--search-bookmyshow

Secondary — Las Vegas Aces: Server-driven splash + audio streaming.
→ stories/story-bank.md#s12--audio-streaming--server-driven-splash-raw--las-vegas-aces

5. Normal questions

For each: answer out loud within the time. Skeleton only — expand from memory.

Q1. What is server-driven UI? `(30–45s)`

Skeleton: Backend sends structured layout/content schema; client maps types to native components via registry; actions allowlisted. Not WebView-only, not JS eval.
Follow-up: vs HTML WebView? → Native perf/a11y/brand control + schema constraints.
Story: S3 / S12.

Q2. Core client components? `(45–60s)`

Skeleton: Parser, version gate, ComponentRegistry, LayoutResolver, ActionHandler, FallbackEngine/cache, analytics hooks.
Follow-up: Where does VM sit? → Owns fetch/cache state; registry is pure mapping.
Story: Point to sdui-engine.md.

Q3. How do you version the schema? `(60s)`

Skeleton: Semantic version; client max-supported; dual-write on breaking changes; ignore unknown fields; reject incompatible major to fallback.
Follow-up: Who owns version bumps? → Contract between mobile + backend; changelog.
Story: S3 API contracts.

Q4. Unknown component arrives in prod — what happens? `(45s)`

Skeleton: Skip node, log/metric, continue siblings; never crash. Optional placeholder only if design allows.
Follow-up: How detect in QA? → Contract tests + canary payloads.
Story: Crash-free culture S8 + S3.

Q5. Offline / fetch failure strategy? `(45–60s)`

Skeleton: Disk last-known-good with TTL; if none, native default; show stale affordance if product needs; retry with backoff.
Follow-up: Splash specifically? → Cached splash critical for cold start (S12).
Story: S12.

Q6. How did BMS header SDUI help the business? `(45s)`

Skeleton: Faster content/layout iteration without app release for many cases; consistent protocolised implementation; paired with better search UX.
Follow-up: Limits? → Still need app release for new component types.
Story: S3.

Q7. SDUI actions security? `(45–60s)`

Skeleton: Allowlist action types; validate URLs/domains; no script execution; auth-sensitive actions re-check client-side.
Follow-up: Open external URL? → SFSafariViewController / controlled browser; ATS.
Story: Day 09 whitelist mindset + S4.

Q8. When is SDUI a bad idea? `(45s)`

Skeleton: Highly interactive unique UI, strong a11y/custom animation needs, rare change surfaces, team without schema QA. Prefer hybrid slots.
Follow-up: Ads HeroWidget? → Often native for revenue-critical media lifecycle (S1).
Story: Judgment vs S1 native ads.

Q9. How do analytics work in SDUI? `(45s)`

Skeleton: Server can attach event names/params; client validates and injects common context (user, screen). Don't trust server to log PII unchecked.
Follow-up: Mixpanel on Grizzlies? → Host analytics vs SDK — S13 adjacent.
Story: S3 + discipline.

Q10. Cold start + server splash — metrics? `(45–60s)`

Skeleton: Time-to-first-frame vs time-to-interactive; cache splash JSON/images; don't block forever on network; timeout → default.
Follow-up: Audio init parallel? → Don't serialize everything on main launch path (S12).
Story: S12.

Q11. Testing SDUI? `(45–60s)`

Skeleton: Fixture schemas per version; registry unit tests; snapshot critical layouts; contract tests with backend; chaos payload (unknown types).
Follow-up: UITests? → Few golden paths; not every CMS combo.
Story: S9 test mindset applied to SDUI.

Q12. SDUI vs feature flags vs A/B? `(45s)`

Skeleton: Flags toggle code paths; A/B assigns variants; SDUI changes layout/content via payload. Often combined: flag enables SDUI surface; CMS supplies variant layout.
Follow-up: Point to feature-flag / ab-testing docs.
Story: Experiment velocity narrative.

6. Tricky questions

T1. "JSON drives UI — isn't that just a WebView?" `(90–120s)`

Trap: Conflate SDUI with web.
Senior answer: SDUI maps to **native** components with typed schema, a11y, performance budgets, offline cache. WebView is a tool, not the architecture. Hybrid possible (HTML island) but BMS header was protocol-native.
Follow-up: When WebView wins? → Rare complex docs/legal; not primary nav chrome.

T2. "Backend sent a breaking schema to 30L users." `(120s)`

Trap: Only "we crashed."
Senior answer: Version gate + fallback layout + kill switch/feature guard + Crashlytics watch + IMOC coordination (S8). Canary %

rollout of schema. Client must survive bad payloads.
Follow-up: Who pages whom? → Mobile + backend owners; blast radius.

T3. “How do you keep Android and iOS parity?” `(90s)`

Trap: Copy-paste JSON hope.
Senior answer: Shared schema spec, capability negotiation, contract tests in CI per platform registry, dual-publish rules. Accept temporary capability gaps with server targeting.
Follow-up: New component: native apps ship first, then CMS uses it.

T4. “Skip unknown components — empty screen risk?” `(90s)`

Trap: Skipping forever without metrics.
Senior answer: Skip is crash-safe default; emit `unknown_component` metrics; alert on spike; require minimum viable tree (e.g. at least logo on splash). Empty root → hard fallback.
Follow-up: S12 splash minimum content policy.

T5. “Personalised SDUI + cache — privacy leak?” `(90–120s)`

Trap: One disk cache for all users.
Senior answer: Cache keys include user/session; clear on logout; don’t write sensitive payloads longer than needed; pin down what splash may contain.
Follow-up: Guest vs logged-in header variants.

T6. “Where should component registry live — app or SDK?” `(90s)`

Trap: Absolute.
Senior answer: App owns app-specific components; shared SDK can own common primitives (text, stack, image) — Stories SDK lesson (S10): clear public API and versioning. Header registry likely app module with protocol hooks.
Follow-up: S10 modularity crossover.

T7. “SDUI for revenue Ads?” `(90–120s)`

Trap: SDUI everything.
Senior answer: Ads often need strict native lifecycle (pause/play video — S1 HeroWidget). Use SDUI for placement/config; keep media renderers native and type-safe (POP+generics).
Follow-up: Perfect Mock #2 fork: Ads architecture vs SDUI engine.

T8. “Parser on main thread drops frames.” `(90s)`

Trap: Ignore NFRs.
Senior answer: Parse off main; budget <16ms interaction; prewarm splash cache; incremental render. Cite sdui-engine NFRs.
Follow-up: Instruments Time Profiler on launch (Week 3 preview).

7. Flashcards for today

Front	Back
SDUI definition	Schema → native registry render · Trap: JS eval · Prod: BMS header
Unknown component	Skip + metric · Trap: crash · Prod: 99.95% mindset
Schema major bump	Dual-publish + fallback · Trap: force upgrade only · Prod: contracts
FallbackEngine	Last-known-good disk · Trap: blank on offline · Prod: S12 splash
Action allowlist	No arbitrary code · Trap: open any URL · Prod: security
Hybrid SDUI	CMS slots + native chrome · Trap: 100% dynamic · Prod: most apps
Capability matrix	Client declares support · Trap: assume parity · Prod: iOS/Android

Cold start splash	Cache + timeout default · Trap: block on network · Prod: S12
BMS header win	Iterate without release · Trap: new types need app · Prod: S3
SDUI vs Ads native	Config dynamic; media native · Trap: SDUI video lifecycle · Prod: S1
Cache keying	Per user for personalised · Trap: shared cache leak · Prod: privacy
Analytics in SDUI	Validate server events · Trap: trust PII · Prod: hygiene
Canary schema	% rollout payloads · Trap: 100% push · Prod: S8 culture
Empty root policy	Hard fallback required · Trap: skip all → blank · Prod: splash rules
Contract tests	Fixtures per version · Trap: only manual QA · Prod: CI

8. Practice

- **Coding / SD:** Whiteboard SDUI HLD from sdui-engine.md in **5 minutes** (Mock #2 option). List BMS header component types you'd register and fallback rules.
- **Complexity / agenda to say first:** "I'll define schema + registry + versioning + fail-soft, then map to BMS header and Aces splash, then trade-offs vs native Ads."
- **Story polish:** S3 and S12 openers back-to-back (20s each).

9. Timed drill

1. Pick 3 Normal + 2 Tricky (recommended: Q3, Q4, Q10 + T2, T7). Record.
2. Score against answer-timing-guide.md.
3. Log misses in gotchas (versioning + unknown-component policy).
4. Optional: 5-min timed SDUI architecture talk — record and compare to Mock #2 rubric on Day 14.

Day 11 — UIKit Lifecycle, Cells, Prefetch · Hybrid UIKit ↔ SwiftUI

> *Week 2 · Phase: Architecture, networking, SDUI, UI · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- UIViewController **lifecycle** landmarks and where to start/stop work (incl. video/ads pause-play)
- `UITableView` / `UICollectionView` **cell reuse**, self-sizing pitfalls, and **prefetch**
- How to host **SwiftUI in UIKit** and **UIKit in SwiftUI** without identity/lifecycle bugs
- How Grizzlies hybrid UI + deeplinks and BMS **LE Bottom Sheet** (30%+ nav reduction) show product-minded UI engineering

2. Concept deep dive

2.1 UIViewController lifecycle (say in order)

`init` → `loadView` → `viewDidLoad` → `viewWillAppear` → `viewIsAppearing` (iOS 17+) → `viewDidAppear` → `viewWillLayoutSubviews` / `viewDidLayoutSubviews` → `viewWillDisappear` → `viewDidDisappear` → `deinit`

Hook	Put here	Avoid
`viewDidLoad`	One-time setup, bindings	Assuming final bounds
`viewWillAppear`	Refresh on-screen state	Heavy sync I/O
`viewDidAppear`	Analytics "seen", start players	Assuming still visible forever

`viewWillDisappear`	Pause video, cancel tasks	Forgetting Ads HeroWidget pause
`deinit`	Prove no retain cycle	Relying on it for critical cleanup

Ads link: HeroWidget pause/play tied to visibility / VC lifecycle (S1) — lifecycle is part of the revenue contract.

2.2 Cells & reuse

- Always reset cell state in `prepareForReuse` (images, gestures, selections).
- Prefer Diffable Data Source for identity correctness.
- Self-sizing: stable Autolayout constraints; avoid multi-pass thrash.
- Don't start unconstrained network in `cellForItem` without cancel on reuse.

2.3 Prefetching

`UICollectionViewDataSourcePrefetching` : warm images/data for near-viewport indexPaths; cancel on `cancelPrefetchingForItemsAt` . Couple with image pipeline (Week 3) and networking cancellation (Day 09).

Trap: Prefetch = unlimited download. Budget concurrent fetches.

2.4 Hybrid UIKit ↔ SwiftUI

Direction	API	Watch-outs
SwiftUI in UIKit	`UIHostingController`	Sizing, `additionalSafeAreaInsets`, parent VC lifecycle
UIKit in SwiftUI	`UIViewControllerRepresentable` / `UIViewRepresentable`	`make` vs `update`; coordinator; identity stability
Navigation	One "source of truth"	Dual stacks fight deeplinks

Grizzlies: Architected key UI with SwiftUI+UIKit interop; deeplinks + Mixpanel + Airship — interop costs (identity, lifecycle, hosting) designed, not bolted.

2.5 LE Bottom Sheet (product UI)

Reusable lightweight event-overview sheet reduced **full-screen navigations for 30%+ of user flows**. Cross-functional API/content contracts with PM/Design/Backend. Prefer sheet over push when overview depth is shallow.

2.6 Trade-offs

Choice	When	Cost
Pure UIKit	Mature codebase, fine control	Slower feature UI
Pure SwiftUI	New features, lists, forms	Interop/legacy gaps
Hybrid	Real production NBA apps	Lifecycle/identity complexity
Bottom sheet	Shallow overview	Not for deep multi-step flows
Full-screen push	Deep hierarchy	Nav fatigue (S6 problem)
Prefetch aggressive	Image-heavy feeds	Data/battery burn

3. Read these

Priority	Resource	Why
Must	UIViewController	Lifecycle source of truth

Must	UIViewControllerRepresentable	Hybrid hosting
Deepen	WWDC: demystify SwiftUI identity; UIKit interop sessions	Identity bugs
Repo	deep-linking-universal-links.md	Grizzlies deeplink context
Repo	S6 · S13 · S1 in story-bank.md	Sheet, hybrid, ads lifecycle

4. Map to your work

Company / feature: Raw / Memphis Grizzlies — SwiftUI↔UIKit + deeplinks + Mixpanel/Airship
What you did: Hybrid UI architecture; deep linking with navigation/lifecycle handling; analytics + push integrations.
Interview line (≤20s): "On Grizzlies I designed SwiftUI↔UIKit interop with deliberate lifecycle and deeplink ownership — not ad-hoc hosting."

→ Full STAR: stories/story-bank.md#s13--swiftuiuikit--deeplinks--analyticspush-raw--memphis-grizzlies

Secondary — BMS LE Bottom Sheet: End-to-end lightweight event overview → **30%+** fewer full-screen navs.
→ stories/story-bank.md#s6--le-bottom-sheet-bookmyshow

Tertiary — Ads HeroWidget lifecycle: pause/play with visibility (S1).

5. Normal questions

For each: answer out loud within the time. Skeleton only — expand from memory.

Q1. Walk UIViewController appearance lifecycle. `(30–45s)`

Skeleton: load → will/did appear → layout → will/did disappear; use appear for refresh, disappear for pause/cancel.
Follow-up: `viewDidLoad` vs appear? → One-time vs every show.
Story: S1 video pause.

Q2. Why does cell reuse show wrong images? `(45s)`

Skeleton: Async image completes after reuse; fix with tokens/IDs in `prepareForReuse`, cancel tasks, check identity before assign.
Follow-up: Prefetch interaction? → Cancel prefetch on leave.
Story: General; Ads cells caution.

Q3. Diffable data source benefit? `(30–45s)`

Skeleton: Identity-based updates, animated diffs, fewer reloadData footguns. Still need stable IDs.
Follow-up: Hashable pitfalls? → Unstable hashes → flicker.
Story: List-heavy BMS surfaces.

Q4. How does prefetching work? `(45–60s)`

Skeleton: System hints upcoming indexPaths; start warm fetches; cancel when not needed; bound concurrency.
Follow-up: Prefetch data or only images? → Often both via repository.
Story: Feed/list performance narrative.

Q5. Embed SwiftUI in a UIKit app? `(45–60s)`

Skeleton: `UIHostingController`; child VC containment correctly; pass state via Observable; size with constraints/intrinsic.
Follow-up: Safe area issues? → Hosting controller safeAreaInsets quirks.
Story: S13.

Q6. Embed UIKit in SwiftUI? `(45–60s)`

Skeleton: Representable; Coordinator for delegates; `updateUIViewController` for props; stable identity (`id`) to avoid recreate storms.

Follow-up: When recreate is required? → Rare; prefer updates.
Story: S13.

Q7. Deeplink into hybrid nav stack — approach? `(60–90s)`

Skeleton: Parse URL at app edge → typed Intent → single router builds/pushes correct UIKit or SwiftUI host; defer until root ready; avoid double-present.
Follow-up: Cold start deeplink? → Queue until main UI ready.
Story: S13 + deeplink doc.

Q8. Bottom sheet vs push — decision? `(45s)`

Skeleton: Sheet for glanceable overview; push for deep hierarchy/history. LE sheet cut 30%+ full-screen navs.
Follow-up: Accessibility/detents? → Design detents; VoiceOver focus.
Story: S6.

Q9. Where do you pause ad video? `(45s)`

Skeleton: On disappear / out of viewport / app background; resume on visible. Protocolised HeroWidget behavior.
Follow-up: Cell scrolled half off? → Visibility threshold policy.
Story: S1.

Q10. Retain cycle via closure in cell? `(45s)`

Skeleton: Cell → closure → VC → collection → cell. Use `[weak self]`, clear actions in `prepareForReuse`.
Follow-up: Instruments? → Leaks (Day 03 recall).
Story: Crash-free hygiene S8.

Q11. Mixpanel + Airship with UI lifecycle? `(45–60s)`

Skeleton: Screen events on appear/disappear; push notification taps route via same deeplink router; don't double-count.
Follow-up: Privacy consent? → Gate SDK init.
Story: S13.

Q12. Self-sizing collection jank causes? `(45–60s)`

Skeleton: Ambiguous constraints, estimated size far from real, image height changes post-bind, heavy work on main. Fix estimates, prefetch images, stable heights when possible.
Follow-up: Prefer SwiftUI list? → Trade-offs Day 12.
Story: Hybrid judgment.

6. Tricky questions

T1. "`viewDidLoad` isn't called again — bug or feature?" `(90s)`

Trap: Misunderstanding reuse of VC instances.
Senior answer: Feature — VC retained in stack. Put repeatable refresh in appear; one-time in didLoad. Tab VCs may load once and appear many times.
Follow-up: Memory warning reload strategies.

T2. "HostingController messes Auto Layout height." `(90–120s)`

Trap: Only `frame = bounds` hacks.
Senior answer: Intrinsic content size bridging, `sizingOptions`, constrain edges carefully, avoid nested scroll view fights; sometimes UIKit layout authority wins.
Follow-up: S13 production war story style answer.

T3. "Two sources of truth — NavigationPath and UINavigationController." `(120s)`

Trap: Sync both every time.
Senior answer: Pick one root owner; bridge at edges; deeplinks write to owner only. Hybrid apps fail when both stacks mutate

independently.
Follow-up: Grizzlies approach: single router.

T4. “Prefetch caused data usage complaints.” `(90s)`

Trap: Prefetch bad.
Senior answer: Bound concurrency, respect Low Data Mode, cancel aggressively, prefetch distance tuning, Wi-Fi vs cellular policy.
Follow-up: Image pipeline quality tiers (Week 3).

T5. “Why not make LE overview a new tab?” `(90s)`

Trap: Engineering-only answer.
Senior answer: Product: reduce steps in existing flows; sheet preserves context. Metric: 30%+ fewer full-screen navs. Tabs change IA; sheet fixes local friction.
Follow-up: Cross-functional contract story S6.

T6. “Representable `update` called too often — perf?” `(90–120s)`

Trap: Ignore SwiftUI invalidation.
Senior answer: Reduce state churn; Equatable inputs; avoid recreating VC; move heavy work out of update; identity stability.
Follow-up: Tie to Day 12 SwiftUI identity.

T7. “Cell starts URLSession that finishes after scroll away.” `(90s)`

Trap: Only image library blame.
Senior answer: Cancel on reuse/disappear; correlate request ID with model ID; repository-level coalescing optional. Same pattern as search cancel.
Follow-up: Day 09 cancellation.

7. Flashcards for today

Front	Back
Pause video where	willDisappear / offscreen · Trap: only deinit · Prod: S1 HeroWidget
prepareForReuse	Reset async + UI · Trap: stale images · Prod: lists
Prefetch cancel	cancelPrefetching… · Trap: unbounded downloads · Prod: data budget
SwiftUI in UIKit	UIHostingController · Trap: sizing/safeArea · Prod: S13
UIKit in SwiftUI	Representable + Coordinator · Trap: recreate storms · Prod: S13
Deeplink owner	App-edge router → intent · Trap: dual stacks · Prod: S13
Bottom sheet win	Context kept; fewer pushes · Trap: deep flows in sheet · Prod: S6 30%+
Diffable benefit	Identity diffs · Trap: unstable IDs · Prod: fewer reload bugs
Appear vs load	Refresh vs once · Trap: network only in didLoad · Prod: tabs
Cell retain cycle	weak self + clear handlers · Trap: strong VC in cell · Prod: leaks
Airship tap path	Same deeplink router · Trap: parallel nav · Prod: S13
Hybrid cost	Lifecycle + identity · Trap: bolt hosting · Prod: S13 lesson
Estimated size	Close to real · Trap: 1pt estimates · Prod: jank
LE metric	30%+ fewer full-screen navs · Trap: vanity UI · Prod: resume
Visibility ads	Threshold policy · Trap: play always · Prod: S1

8. Practice

- **Coding / SD:** Sketch Grizzlies deeplink → router → UIKit host vs SwiftUI host sequence. Separately, outline LE Bottom Sheet presentation API (inputs, dismiss, analytics).
- **Complexity / agenda to say first:** "Lifecycle ownership first, then cells/prefetch, then hybrid identity, then product sheet metric."
- **Hands-on (optional 30m):** In a scratch project, embed a representable and log `make` / `update` counts while typing — feel invalidation.

9. Timed drill

1. Pick 3 Normal + 2 Tricky (recommended: Q7, Q8, Q5 + T3, T5). Record.
2. Score against answer-timing-guide.md.
3. Log misses in gotchas (dual navigation + hosting sizing).
4. Deliver S6 and S13 STAR openers (20s each) + one full S6 in 2–3 min.

Day 12 — SwiftUI State, Identity, @Observable, Lists, Performance

> *Week 2 · Phase: Architecture, networking, SDUI, UI · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- SwiftUI **state ownership** rules: `@State`, `@Binding`, `@Observable` / `@Bindable`, environment — what belongs where
- Why **identity** (`id`, structural identity, `@State` lifetime) causes "random" bugs
- How to build **performant lists** (lazy stacks, stable IDs, avoid invalidation storms)
- How a **Stories SDK** public API keeps state/modularity clean across portfolio apps (Miami Heat / Raw)

2. Concept deep dive

2.1 State ownership (memorize)

Tool	Owner	Use
`@State`	View (private)	View-local ephemeral UI
`@Binding`	Child writes parent state	Two-way controls
`@Observable` model	Often VM / feature model	Async + shared screen state
`@Bindable`	Bridge to observable fields	Forms into models
`Environment` / `EnvironmentObject`	Tree-wide	Theme, DI-lite — not every service
`@StateObject` / `ObservableObject`	Legacy Combine path	Know for interviews; prefer Observation in new code

Rule: View-local for pure UI; hoist async/domain to observable VM (Day 08). Don't duplicate every toggle into VM.

2.2 Identity — the senior differentiator

SwiftUI diffs the view tree by **identity**:

- **Structural identity:** position in hierarchy / type
- **Explicit identity:** `.id(...)` / `ForEach(id:)`
- **`@State` lifetime** follows identity — change `id` → state resets

Bugs from bad identity: list flicker, lost scroll position, recreated `UIViewControllerRepresentable`, restarted animations, Stories page reset mid-swipe.

Stories SDK implication: Story page identity must be stable across updates; don't regenerate UUIDs each render.

2.3 Lists & performance

- Prefer `List` / `LazyVStack` in `ScrollView` for large data; avoid eager `VStack` of thousands.
- Stable `Identifiable` models; never `id: \.self` on mutable value types that change equality.
- Minimize work in `body` ; precompute in model; avoid heavy formatting per row.
- Images: async + size budgets (Week 3 image pipeline).
- Prefer Equatable view inputs / reduced dependency on huge observed graphs.

Invalidation storms: one giant `@Observable` god-object → entire screen redraws. Split models or observe granularly.

2.4 Animation & transitions

Intentional animations via explicit `withAnimation` / transaction; identity-preserving updates animate better than destroy/recreate. Stories progress bar: drive from model timeline, not ad-hoc view timers alone.

2.5 Stories SDK (modular SwiftUI)

Standalone reusable SDK: clear public API, isolation from host networking where possible, adopted across NBA/WNBA portfolio. State machine for story groups/pages, pause on disappear/background, prefetch adjacent media carefully.

2.6 Trade-offs

Choice	When	Cost
`@Observable` VM	Feature screens	Over-observation if monolithic
All `@State` in views	Tiny UI	Untestable async spaghetti
`AnyView` erasure	Rare type erasure need	Kills optimization / identity clarity
Eager VStack	Tiny lists	Scroll jank at scale
SDK owns networking	Convenience	Couples host; prefer injectable client (S10)
`.id(UUID())` per body	Never	Resets state every frame

3. Read these

Priority	Resource	Why
Must	State and data flow	Official model
Must	WWDC: **Demystify SwiftUI** · **SwiftUI performance**	Identity + perf vocabulary
Deepen	Observation framework docs	`@Observable`
Repo	app-modularization.md	SDK boundaries
Repo	stories/story-bank.md#s10--stories-sdk-raw--miami-heat	Production hook

4. Map to your work

Company / feature: Raw / Miami Heat — Stories SDK
What you did: Designed standalone reusable Stories SDK with clear public API and isolation from app-specific networking where possible; leveraged across portfolio apps.
Interview line (≤20s): "I built a reusable Stories SDK — stable public API and host isolation — so multiple NBA/WNBA apps shared one Instagram-style stories implementation."

→ Full STAR: stories/story-bank.md#s10--stories-sdk-raw--miami-heat

Also: Grizzlies hybrid hosting identity (S13); SDUI registry views as SwiftUI leaves (Day 10).

5. Normal questions

For each: answer out loud within the time. Skeleton only — expand from memory.

Q1. `@State` vs `@Observable` — when each? `(30–45s)`

Skeleton: `@State` for local UI; `@Observable` for feature model/async shared across child views. Binding into observable via `@Bindable`.
Follow-up: Legacy `ObservableObject`? → Still in codebases; know `@Published` + `objectWillChange`.
Story: Stories player model vs local chrome state.

Q2. What is view identity? `(45–60s)`

Skeleton: How SwiftUI recognizes same view across updates; structural vs explicit `.id`; `@State` tied to identity.
Follow-up: When force new identity? → Intentional reset (logout clears form).
Story: S10 page identity.

Q3. Why did my Representable reset? `(45s)`

Skeleton: Parent identity changed or `id` churn → remake. Stabilize IDs; update props in `update` instead of recreate.
Follow-up: Day 11 hybrid link.
Story: S13.

Q4. `ForEach` best practices? `(45s)`

Skeleton: Stable `Identifiable`; avoid indices as IDs when reordering; don't create new IDs in `body`.
Follow-up: Duplicate IDs? → Undefined weirdness / warnings.
Story: Stories pages array.

Q5. Make a long list smooth. `(60–90s)`

Skeleton: Lazy containers, async images, cheap `body`, stable IDs, pagination, avoid observing whole catalog in each row.
Follow-up: Instruments? → SwiftUI and Time Profiler.
Story: Portfolio apps scale; SDK consumers.

Q6. How do you design Stories SDK API? `(60–90s)`

Skeleton: `StoriesPlayer` entry, data source protocol, event callbacks (open/close/tap CTA), injectable image/video loader, versioned module. Host supplies stories DTOs.
Follow-up: Why not hardcode networking? → Portfolio variance; testability.
Story: S10.

Q7. Pause stories on background/disappear? `(45s)`

Skeleton: Scene phase / `onDisappear`; pause timers & AVPlayer; resume policy explicit. Same lifecycle discipline as Ads video.
Follow-up: S1 parallel.
Story: S10 + S1.

Q8. Environment for DI — good idea? `(45s)`

Skeleton: Good for theme, layout direction, shallow deps. Bad as hidden service locator for NetworkClient everywhere — prefer explicit init for SDKs.
Follow-up: Day 08 DI.
Story: S10 injectable deps.

Q9. Animation causes list jump — causes? `(45–60s)`

Skeleton: Identity changes, height changes without animation transaction, diffable mismatch, scroll position loss. Fix stable IDs + animate data changes carefully.

Follow-up: `.animation` on big trees? → Scope animations.
Story: General senior debugging.

Q10. Performance: `body` called often — problem? `(45s)`

Skeleton: `body` should be cheap and pure-ish; frequent calls expected. Problem is heavy work inside. Move to model.
Follow-up: Avoid side effects in `body` .
Story: Interview classic.

Q11. Cross-app reuse challenges for Stories? `(45–60s)`

Skeleton: Theming, analytics hooks, media formats, nav/deeplink exits, dependency versions. Solve with protocols + defaults.
Follow-up: Modularization Day 15 preview.
Story: S10 portfolio adoption.

Q12. SwiftUI + SDUI registry? `(45–60s)`

Skeleton: Registry returns SwiftUI views keyed by type; keep leaves dumb; VM owns payload; identity of each node from server id.
Follow-up: Day 10.
Story: S3 header components.

6. Tricky questions

T1. “I put UUID in `.id` inside body — why is state broken?” `(90s)`

Trap: Blame SwiftUI bugs.
Senior answer: New identity every render resets `@State` and remakes children. IDs must be stable model keys.
Follow-up: When UUID ok? → Model created once and stored.

T2. “One `@Observable` AppModel for everything.” `(90–120s)`

Trap: Convenient architecture.
Senior answer: Causes broad invalidation and tight coupling. Split by feature; pass slices; SDK shouldn’t depend on host AppModel.
Follow-up: S10 isolation.

T3. “LazyVStack still janky.” `(90–120s)`

Trap: Only “use List.”
Senior answer: Profile: image decode, main-thread JSON, overlapping observations, complex shadows, unbounded prefetch. Fix data path, not only container choice.
Follow-up: Week 3 image pipeline.

T4. “Should Stories SDK be SwiftUI-only?” `(90s)`

Trap: Binary.
Senior answer: SwiftUI-first OK if hosts can host; offer UIKit wrapper (`UIHostingController` façade) for legacy. Public API should not force one nav paradigm.
Follow-up: S13 hybrid hosts.

T5. “Equatable View conformance — worth it?” `(90s)`

Trap: Micro-optimize everything.
Senior answer: Useful for expensive subtrees with rare changes; don’t sprinkle early. Prefer smaller observed state. Measure first.
Follow-up: Observation already tracks properties — know interactions.

T6. “How do you test SwiftUI state logic?” `(90s)`

Trap: Only UITests.
Senior answer: Test observable models/UseCases in XCTest; light ViewInspector/snapshots optional; UITests for critical Stories

open/close.
Follow-up: S9 testing culture.

T7. "Progress bar desyncs when paging stories." `(90–120s)`

Trap: More timers in views.
Senior answer: Single source of truth timeline in model; view renders; identity of page must not reset timer state incorrectly; pause/resume explicit events.
Follow-up: State machine > scattered `onAppear` timers.

T8. "Host app injects different image pipelines." `(90s)`

Trap: SDK bundles Kingfisher only.
Senior answer: `ImageLoading` protocol in SDK; hosts adapt; default implementation provided. Same DI lesson as networking Day 09.
Follow-up: S10 API design.

7. Flashcards for today

Front	Back
`@State` use	Local ephemeral UI · Trap: async in every view · Prod: chrome
`@Observable` use	Feature/async model · Trap: god AppModel · Prod: Stories player
Identity rule	Stable IDs preserve state · Trap: UUID in body · Prod: S10 pages
ForEach ID	Identifiable stable keys · Trap: indices on reorder · Prod: lists
Lazy vs VStack	Lazy for large · Trap: eager thousands · Prod: perf
body rule	Cheap; no side effects · Trap: fetch in body · Prod: jank
Environment DI	Theme ok; services careful · Trap: hidden NetworkClient · Prod: S10
Stories API	DataSource + events + inject loaders · Trap: hardcode host net · Prod: S10
Pause stories	Scene phase / disappear · Trap: run in background · Prod: S10/S1
Invalidation storm	Split models · Trap: observe all · Prod: redraws
Representable reset	Parent id churn · Trap: SwiftUI bug · Prod: S13
Animation scope	Transaction local · Trap: animate whole tree · Prod: lists
Portfolio reuse	Protocols + theming hooks · Trap: fork per app · Prod: S10
SDUI leaf id	Server-stable id · Trap: array index · Prod: S3
Test SwiftUI	Model unit tests first · Trap: only UITest · Prod: S9

8. Practice

- **Coding / SD:** Sketch Stories SDK module boundary: public protocols, player state machine (idle → loading → playing → paused → finished), host callbacks.
- **Complexity / agenda to say first:** "State ownership, then identity, then list perf, then SDK API isolation with S10."
- **Hands-on:** Explain aloud why `.id(UUID())` on a form resets text — 30s drill.

9. Timed drill

1. Pick 3 Normal + 2 Tricky (recommended: Q2, Q6, Q5 + T1, T2). Record.
2. Score against answer-timing-guide.md.

3. Log misses in gotchas (identity + observation scope).
4. Deliver S10 STAR in **2–3 min**.

Day 13 — DSA: Stack / Queue / Linked List + Catch-up

> ***Full chapter:*** *`weeks/week-02/day-13/` · Revision twin — timed recall only after the full study.*

> *Week 2 · Phase: Architecture, networking, SDUI, UI · Time budget today: ~5–6 hrs (weekend-style volume)*

1. Outcome

By end of day you can explain aloud (no notes):

- When to reach for **Stack**, **Queue** / **Deque**, and **Linked List** (and when arrays beat linked lists in Swift)
- Complexities for push/pop/enqueue/dequeue and classic patterns (monotonic stack, BFS queue, fast/slow pointers)
- A crisp **agenda-first** coding approach (2–3 min) before typing Swift
- What Week 2 theory you still owe — and a catch-up plan without dropping Mock #2 prep

Track index: coding/dsa-track.md

2. Concept deep dive

2.1 Structure cheat-sheet

Structure	Ordered ops	Amortized / typical	Swift go-to
Stack LIFO	push/pop/peek	O(1)	`Array` append/removeLast
Queue FIFO	enqueue/dequeue	O(1) ideal	`Array` dequeue is O(n) — prefer `Deque` / ring / two-stack queue
Deque	ends both sides	O(1)	Swift Collections `Deque` or manual
Linked list	insert/delete given node	O(1) local	Rare in Swift interviews vs arrays; know algorithms

Interview honesty: In Swift production, `Array` dominates. Still implement linked-list **algorithms** (reverse, cycle, merge) on `ListNode`.

2.2 Pattern → structure

Pattern	Structure	Examples
Matching / nesting / undo	Stack	Valid parentheses, calc, decode string
Next greater / smaller	Monotonic stack	Daily temperatures, stock span
Level-order / BFS	Queue	Binary tree level order (preview Week 4)
Sliding window max (deque)	Deque	Max in window
Cycle / middle / reverse	Linked list pointers	Floyd, reverse list, merge two lists
LRU (preview)	Hash + DLL	Cache design

2.3 Complexity lines to say first

Before coding: “I’ll use a stack for LIFO matching — O(n) time, O(n) space worst case.”
Or: “Two pointers on linked list — O(n) time, O(1) space.”

2.4 Swift pitfalls

- `removeFirst()` on `Array` is **O(n)** — don't build queues that way in hot paths without noting it.
- Class `ListNode` reference semantics — careful aliasing in reverse.
- Prefer iterative reverse in interviews unless recursion depth discussed.

2.5 Catch-up (Week 2)

Use this day to close gaps from Days 08–12:

If weak on…	Catch-up block (45–60m)
MVVM/Clean/DI	Re-drill Day 08 Q5, Q8, T1 + S9 opener
Networking/refresh/pinning	Day 09 T1, T3 + skim networking-layer.md sequence
SDUI versioning/fallback	Day 10 Q3–Q4 + S3 opener
Hybrid lifecycle	Day 11 Q7 + S13 opener
SwiftUI identity	Day 12 T1 + S10 opener

Do **not** skip section 9 timed DSA drill — Mock #4 coding depends on reps.

2.6 Trade-offs

Choice	When	Cost
Array as stack	Default	Fine
Array as queue	Tiny n	O(n) dequeue
Two-stack queue	Interview FIFO	Extra code
Linked list	Pointer problems	Poor cache locality in real Swift
Recursion on list	Elegant reverse	Stack overflow risk on long lists

3. Read these

Priority	Resource	Why
Must	coding/dsa-track.md	Master list + machine-round mapping
Must	timing/answer-timing-guide.md (coding approach budget)	2–3 min plan before code
Deepen	LeetCode Explore: Stack/Queue · Linked List	Guided reps
Repo	Week 2 days 08–12 for catch-up links	Fill theory holes

4. Map to your work

DSA days are mostly pattern training — still **bridge to production** when interviewers ask "have you used this?"

Structure	Production bridge (honest, metric-safe)
Queue	Serialise work / token refresh waiters (Day 09 single-flight queue of waiters)
Stack	Nav back stacks; undo; nested CMS layout resolve (parent stack)
Linked list mentally	Not common in Swift UI — speak to pointer problems as interview skill; caches/LRU later

Story links (light touch today):

- Refresh waiters / in-flight management → S4#s4--ssl-pinning--alamofire--urlsession-bookmyshow mindset + Day 09
- Nested UI / nav → S6#s6--le-bottom-sheet-bookmyshow · S13#s13--swiftuiuikit--deeplinks--analyticspush-raw--memphis-grizzlies

Interview line (≤20s): "For coding I'll state structure + complexity first — e.g. monotonic stack O(n) — then code clean Swift; I use queues in networking refresh coalescing similarly."

5. Normal questions

For each: answer out loud within the time. Skeleton only — expand from memory.

Q1. Stack vs queue in one sentence each. `(30s)`

Skeleton: Stack LIFO; queue FIFO.
Follow-up: Deque? → Both ends.
Story: —

Q2. Implement stack with array — ops complexity? `(30–45s)`

Skeleton: append/removeLast O(1) amortized; random insert O(n).
Follow-up: Capacity growth? → Geometric resize amortized.
Story: —

Q3. Why is `Array.removeFirst` bad for queues? `(30–45s)`

Skeleton: Shifts all elements O(n). Use deque, linked list, or two-stack, or head index with periodic compact.
Follow-up: When OK? → Tiny n, rare calls.
Story: —

Q4. Valid parentheses approach? `(45–60s)`

Skeleton: Stack of opens; on close check match; empty at end. O(n)/O(n). Edge: odd length, only closers.
Follow-up: Unicode brackets? → Map table.
Story: —

Q5. Monotonic stack — when? `(45–60s)`

Skeleton: Next greater/smaller in O(n). Maintain increasing/decreasing stack of indices.
Follow-up: Example: daily temperatures.
Story: —

Q6. BFS uses which structure? `(30s)`

Skeleton: Queue. Level-order trees; shortest path in unweighted graphs.
Follow-up: DFS? → Stack/recursion.
Story: —

Q7. Reverse linked list — iterative steps? `(60s)`

Skeleton: prev=null, curr=head; while curr: next=curr.next; curr.next=prev; prev=curr; curr=next; return prev. O(n)/O(1).
Follow-up: Recursive? → O(n) stack space.
Story: —

Q8. Detect cycle — Floyd. `(45–60s)`

Skeleton: Slow/fast pointers; meet ⇒ cycle. Optional reset to find entrance. O(n)/O(1).
Follow-up: Hash set alternative? → O(n) space simpler.
Story: —

Q9. Merge two sorted lists. `(45–60s)`

Skeleton: Dummy head; append smaller; attach remainder. O(n+m)/O(1) extra.
Follow-up: Merge k lists? → Heap (Week 4 preview).
Story: —

Q10. Middle of linked list. `(30–45s)`

Skeleton: Slow/fast; fast ends → slow mid. Even-length policy: state which mid.
Follow-up: —
Story: —

Q11. Agenda you say before coding? `(45s)`

Skeleton: Restate · examples · brute · optimize · complexity · edge cases · then code. Budget 2–3 min.
Follow-up: Timing guide.
Story: —

Q12. Array vs linked list in Swift production? `(45s)`

Skeleton: Arrays for cache locality & Swift ergonomics; linked lists mainly for interview pointer skills / rare specialized structures (LRU nodes).
Follow-up: Don't force LL into iOS UI code.
Story: Honest senior answer.

6. Tricky questions

T1. "Implement a queue with two stacks." `(90–120s)`

Trap: Amortized vs worst-case confusion.
Senior answer: In-stack for enqueue; out-stack for dequeue; flush in→out when out empty. Amortized O(1). State complexities clearly.
Follow-up: Code on whiteboard.

T2. "Min stack in O(1)." `(90–120s)`

Trap: Scanning stack each time.
Senior answer: Aux stack or encode diffs; push/pop/getMin O(1). Discuss space.
Follow-up: Edge: duplicates of min.

T3. "Design hit counter / recent requests queue." `(120s)`

Trap: Unbounded memory.
Senior answer: Queue of timestamps; pop older than window; count = size. Discuss concurrency if asked (actor).
Follow-up: Production: analytics buffering caution.

T4. "Reverse nodes in k-group." `(120s)`

Trap: Off-by-one; broken links.
Senior answer: Count k; reverse segment; reconnect; iterate. Narrate pointers. O(n)/O(1).
Follow-up: If leftover < k leave as-is.

T5. "Why linked lists rarely appear in your iOS code?" `(90s)`

Trap: Sound anti-DSA.
Senior answer: Swift Array performance model + value semantics; LL shines in interview algorithms and specialized caches. I still practice pointer problems for interviews.
Follow-up: LRU as exception using DLL + dict.

T6. "Circular queue with fixed array." `(90–120s)`

Trap: Waste one slot vs count field.
Senior answer: Head/tail mod capacity; distinguish empty/full via count or reserved slot. Useful for ring buffers (audio — S12 adjacency).
Follow-up: Thread safety if producer/consumer.

T7. "Stack for calculator / expression." `(120s)`

Trap: Dive to code without grammar.
Senior answer: Clarify operators/precedence; shunting-yard or two stacks (vals/ops). State assumptions.
Follow-up: Integer overflow.

T8. "Catch-up: interviewer pivots from coding to SDUI mid-way." `(90s)`

Trap: Freeze.
Senior answer: Park code state verbally; answer SDUI with versioning/fallback (Day 10); offer to resume coding. Shows composure (S8 IMOC energy).
Follow-up: Mock #2 tomorrow energy.

7. Flashcards for today

Front	Back
Stack use	LIFO matching/undo · Trap: use for BFS · Prod: nav nested
Queue use	FIFO BFS / waiters · Trap: Array.removeFirst hot · Prod: refresh waiters
Monotonic stack	Next greater O(n) · Trap: O(n²) scan · Prod: —
Two-stack queue	Amortized O(1) · Trap: claim strict O(1) always · Prod: —
Floyd cycle	Slow/fast O(1) space · Trap: only hash set · Prod: —
Reverse list	Iterative 3 pointers · Trap: lose next · Prod: —
Merge lists	Dummy head · Trap: null edges · Prod: —
Array as queue cost	removeFirst O(n) · Trap: ignore · Prod: Swift
Coding agenda	Restate→edges→complexity→code · Trap: code immediately · Prod: interviews
Deque window max	Maintain mono deque · Trap: heap each slide only · Prod: —
Min stack	Aux mins · Trap: O(n) scan · Prod: —
LL in Swift apps	Rare vs Array · Trap: force LL UI · Prod: honesty
k-group reverse	Count-reverse-reconnect · Trap: leftover mishandle · Prod: —
Ring buffer	Head/tail mod · Trap: empty/full ambiguity · Prod: audio-ish
Week 2 catch-up	Drill weak day Qs · Trap: only DSA forever · Prod: Mock #2

8. Practice

- **Coding / SD:** From dsa-track.md, complete **at least**:
  - 2 Easy stack/queue
  - 2 Medium stack or monotonic
  - 2 Medium linked list (reverse, cycle, merge, or mid)
  - 1 "tricky" (min stack **or** two-stack queue **or** k-group)
- **Complexity / agenda to say first:** Every problem: structure + time/space **before** code.
- **Catch-up:** Pick **one** weak theory day (08–12); re-record 2 answers.

- **Swift:** Prefer clear `ListNode` class and array stacks; narrate aloud.

9. Timed drill

1. Pick 1 Easy + 1 Medium; **timed** (Easy ~15–20m, Medium ~25–35m). Record approach minutes separately.
2. Score approach against answer-timing-guide.md coding budget.
3. Log misses in gotchas (especially Array-as-queue and off-by-ones).
4. Tomorrow prep: skim Day 14 Mock #2 rubric; choose **Ads or SDUI** track tonight (don't do both cold).

Day 14 — Revision + Mock #2 (Ads or SDUI Architecture)

> ***Full chapter:*** *`weeks/week-02/day-14/` · Revision twin — timed recall only after the full study.*

> *Week 2 · Phase: Architecture, networking, SDUI, UI · Time budget today: ~5–6 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- A **5-minute** architecture talk on either **Ads (POP + HeroWidget + networking/pinning)** or **SDUI engine (schema → registry → fallback)** without notes
- Week 2 connective tissue: MVVM/DI → networking → SDUI → UIKit/SwiftUI hybrid → SwiftUI identity
- Timed Normal/Tricky answers from Days 08–12 with fewer stalls
- Honest gotchas list + plan for Week 3 (modularization, perf, security)

2. Concept deep dive

2.1 Week 2 synthesis map

```
Day 08  Layers / DI / search VM
   ↓
Day 09  URLSession client / refresh / pin / cancel
   ↓
Day 10  SDUI schema / registry / fallback
   ↓
Day 11  Lifecycle / cells / hybrid / bottom sheet
   ↓
Day 12  SwiftUI state / identity / Stories SDK
   ↓
Day 13  Stack·Queue·LL reps
```

One sentence senior narrative:
"I structure features with clear layers and DI, own networking with cancellation and security, use SDUI where content velocity matters with fail-soft schema, and treat UIKit/SwiftUI lifecycle and identity as production contracts — proven on BMS, District, and Raw apps."

2.2 Mock #2 format (self/peer, 60–90 min)

Block	Time	What
Warm-up Qs	15–20 min	4–5 Normal from Week 2 mixed
Deep dive	15–20 min	2 Tricky (refresh **or** SDUI outage **or** hybrid nav)
Architecture talk	**5 min timed**	**Ads path or SDUI path** (pick one)
STAR	10 min	S1 or S3/S12 + S9 or S4

Retro	10–15 min	Score timing; update gotchas

Parallel SD skim (optional after mock): payment-checkout.md · search-autocomplete.md

2.3 Track A — Ads architecture (5 min)

Agenda to say in first 20s:
"I'll cover problem scope, type-safe component pipeline (POP+generics), HeroWidget lifecycle, networking/pinning on URLSession, and trade-offs vs SDUI for media."

Outline (≈1 min each):

1. **Context:** Highest-revenue Ads module; need safe reusable rendering; video pause/play correctness.
2. **Component model:** Protocol-oriented ad contracts + generics pipeline (not inheritance trees).
3. **HeroWidget:** Visibility/VC lifecycle → pause/play; cell reuse cancel.
4. **Networking:** Alamofire→URLSession; HTTPS; SSL pinning; domain whitelist; pin rotation.
5. **Trade-offs / close:** Keep media native; CMS may configure placement; metrics/revenue safety; questions.

Docs: Day 08–09 notes · S1 · S4 · optional networking-layer.md

2.4 Track B — SDUI architecture (5 min)

Agenda (20s):
"I'll define SDUI scope, schema/versioning, registry+actions, fallback/cache for crash-free, and BMS header / Aces splash examples — then limits vs native Ads."

Outline:

1. **Problem:** Content/layout velocity without releases; personalisation.
2. **Pipeline:** Fetch → version gate → parse → registry → layout → allowlisted actions.
3. **Resilience:** Unknown skip; last-known-good; default splash/header; metrics.
4. **Prod:** BMS backend-driven header + search; Aces server splash / cold start.
5. **Trade-offs:** New component types need app release; Ads media often stays native (S1).

Docs: sdui-engine.md · S3 · S12 · Day 10

2.5 Trade-offs (meta for mock)

Choice	When	Cost
Ads 5-min talk	Strong POP/lifecycle/security stories	Less CMS vocabulary
SDUI 5-min talk	Strong schema/fallback stories	Must not ignore native limits
Doing both cold	Ego	Neither talk is crisp — pick one
Skip STAR	Fatigue	Lose behavioral signal

3. Read these

Priority	Resource	Why
Must	Days 08–12 flashcards (skim)	Active recall
Must	Chosen track doc: networking-layer.md **or** sdui-engine.md	5-min spine
Must	answer-timing-guide.md	Score mock
Deepen	payment-checkout.md · search-autocomplete.md	Week 2 parallel SD
Repo	Story bank S1, S3, S4, S6, S9, S10, S12, S13	Mock STAR pool

4. Map to your work

Pick **one** primary mock story set:

Track	Core stories	Supporting
Ads	S1#s1--ads-module-refactor--herowidget-bookmyshow · S4#s4--ssl-pinning--alamofire--urlsession-bookmyshow	S5 perf, S8 scale
SDUI	S3#s3--backend-driven-header--search-bookmyshow · S12#s12--audio-streaming--server-driven-splash-raw--las-vegas-aces	S8 fail-soft culture

Always available spice: District S9#s9--free-parking--cleanmvvm--ai-tooling-district (architecture migration + AI judgment) · S6#s6--le-bottom-sheet-bookmyshow (product UI) · S10#s10--stories-sdk-raw--miami-heat / S13#s13--swiftuiuikit--deeplinks--analyticspush-raw--memphis-grizzlies (modular/hybrid)

Interview line (≤20s) — Ads: "I'll walk a revenue Ads architecture — POP/generics, HeroWidget lifecycle, and URLSession pinning."
Interview line (≤20s) — SDUI: "I'll walk a fail-soft SDUI pipeline — schema versioning, registry, and BMS/Aces production usage."

5. Normal questions

Revision set — answer out loud within budget. Skeletons are pointers back to day depth.

Q1. MVVM vs Clean — when Clean? `(45s)`

Skeleton: Domain/shared rules/migration → UseCases; else MVVM. District incremental.
Follow-up: AI envelope? → S9.
Story: S9 · Day 08.

Q2. Search debounce + cancel ownership? `(45s)`

Skeleton: VM debounce; cancel Task; don't treat cancel as error.
Follow-up: Out-of-order responses.
Story: S3 · Day 08/09.

Q3. Single-flight token refresh? `(60s)`

Skeleton: One refresh Task; waiters; retry once; fail → logout.
Follow-up: Non-idempotent POST.
Story: Day 09.

Q4. SSL pinning failure mode? `(45–60s)`

Skeleton: MITM↓; bad rotation → outage; backup pins + runbook.
Follow-up: Ads migration why URLSession.
Story: S4.

Q5. Unknown SDUI component policy? `(30–45s)`

Skeleton: Skip + metric; never crash; empty root → hard fallback.
Follow-up: Version major mismatch.
Story: S3 · Day 10.

Q6. SDUI vs WebView? `(45s)`

Skeleton: Native registry, a11y, perf, allowlisted actions.
Follow-up: When WebView OK.
Story: Day 10 T1.

Q7. Where pause ad / story video? `(30–45s)`

Skeleton: Disappear / offscreen / background; protocolised.
Follow-up: Cell reuse.
Story: S1 · S10 · Day 11/12.

Q8. Hybrid deeplink ownership? `(60s)`

Skeleton: App-edge parse → intent → single router; no dual stacks.
Follow-up: Cold start queue.
Story: S13 · Day 11.

Q9. SwiftUI identity footgun? `(45s)`

Skeleton: Unstable `.id` resets state; UUID in body bad.
Follow-up: Stories page IDs.
Story: S10 · Day 12.

Q10. LE Bottom Sheet impact? `(30–45s)`

Skeleton: Lightweight overview; **30%+** fewer full-screen navs; cross-functional contracts.
Follow-up: Sheet vs push.
Story: S6.

Q11. Array as queue issue? `(30s)`

Skeleton: `removeFirst` O(n); use deque/two-stack.
Follow-up: Day 13.
Story: —

Q12. District AI tooling — senior framing? `(45–60s)`

Skeleton: Context Engineering inside protocols + review + tests; you own architecture.
Follow-up: What AI got wrong.
Story: S9.

6. Tricky questions

T1. "5 minutes — design our Ads rendering system." `(5 min talk)`

Trap: Dive into cell code; skip lifecycle/security.
Senior answer: Use Track A outline; end with trade-offs + metrics/revenue.
Follow-up: Pin rotation incident? · POP vs inheritance?

T2. "5 minutes — design SDUI for our home header." `(5 min talk)`

Trap: Ignore versioning/fallback.
Senior answer: Track B outline; cite BMS; unknown skip; cache; action allowlist.
Follow-up: Breaking schema at 30L DAU?

T3. "Refresh stampede + pinning outage same week — how do you lead?" `(120s)`

Trap: Only technical rabbit hole.
Senior answer: IMOC-style: blast radius, feature guards, rollback/break-glass, owners, comms; then technical fixes (single-flight, pin backup). Metrics: crash-free / TLS failure rate.
Follow-up: S8 + S4.

T4. "Why not SDUI the Ads video player?" `(90–120s)`

Trap: SDUI religion.
Senior answer: Revenue media needs typed native lifecycle (S1); SDUI for configuration/placement; native renderer.
Follow-up: Bridge Mock tracks A/B.

T5. “SwiftUI Stories SDK in a UIKit-heavy host.” `(90–120s)`

Skeleton: HostingController façade; injectable deps; deeplink exits via host router; identity stable.
Follow-up: S10 + S13.

T6. “Debounce in repository vs VM — argue both sides, then pick.” `(90s)`

Trap: No decision.
Senior answer: Presentation policy → VM; repository may cancel; pick VM for BMS search.
Follow-up: Day 08 T2.

T7. “Mock feedback: you ran 8 minutes on architecture.” `(60s)`

Trap: Defensiveness.
Senior answer: Cut examples; agenda first; timeboxes per section; practice with timer. Timing guide.
Follow-up: Re-record 5:00 hard stop.

7. Flashcards for today

Front	Back
Week 2 one-liner	Layers→network→SDUI→UI identity · Trap: isolated facts · Prod: narrative
Ads 5-min agenda	POP·lifecycle·URLSession pin · Trap: no agenda · Prod: S1/S4
SDUI 5-min agenda	Schema·registry·fallback·prod · Trap: skip fail-soft · Prod: S3/S12
Refresh stampede	Single-flight · Trap: N refreshes · Prod: Day 09
Pin outage	Rotation/runbook · Trap: pinning only · Prod: S4
Unknown SDUI node	Skip+metric · Trap: crash · Prod: crash-free
Dual nav stacks	One owner · Trap: sync forever · Prod: S13
UUID in body	Resets state · Trap: SwiftUI bug · Prod: Day 12
Sheet metric	30%+ fewer pushes · Trap: vague UX · Prod: S6
AI seniority	Envelope+review · Trap: AI wrote app · Prod: S9
Search cancel	Task cancel silent · Trap: error toast · Prod: S3
HeroWidget	Pause on invisible · Trap: play forever · Prod: S1
Stories SDK	Public API+inject · Trap: host coupled · Prod: S10
Queue pitfall	removeFirst O(n) · Trap: ignore · Prod: Day 13
Mock discipline	Timer + retro gotchas · Trap: skip retro · Prod: growth

8. Practice

- **Coding / SD:**
  - **Required:** One full **5:00** architecture recording (Ads **or** SDUI).
  - **Optional:** 20 min skim of the *other* track’s doc bullets (familiarity only).
  - Light DSA: 1 stack + 1 linked-list Easy/Medium from dsa-track.md if energy remains.
- **Complexity / agenda to say first:** Always open architecture with agenda + scope; invite questions at end.
- **Peer option:** Swap 5-min talks; score each other with timing guide.

9. Timed drill

1. **Mock #2 full run** (60–90 min) per section 2.2 — record architecture block separately.
2. Score all answers against answer-timing-guide.md.
3. Log misses in gotchas; tag `week-02`.
4. Checklist before Week 3:
   - [ ] 5-min Ads **or** SDUI talk hits <5:30
   - [ ] S1/S3/S4/S9 openers each ≤20s
   - [ ] Token refresh + unknown component answers clean
   - [ ] DSA: stack/queue/LL basics not blocking
5. Rest: Week 3 starts modularization / images / perf / crashes / security — don't cram past sleep.

Revision Week 03

Day 15 — App Modularization, SPM & DI Graphs

> ***Full chapter:*** *`weeks/week-03/day-15/`* · *Revision twin — timed recall only after the full study.*

> *Week 3 · Phase: Full architecture, performance, security, platform · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- Why feature modules must depend on **interfaces**, never other feature **implementations**
- How **SPM** targets map to Interface / Impl / Core layers and what that buys for build parallelism
- How a **composition root** (App target) wires a DI graph without feature-level singletons
- How you designed the **Stories SDK** as a reusable module adopted across NBA/WNBA apps
- Trade-offs: CocoaPods vs SPM, dynamic vs static linking, service locator vs constructor/tree DI

2. Concept deep dive

2.1 Module hierarchy that scales

Senior answer structure: **App → Feature Impl → Feature Interface → Core**.

Layer	Owns	Must not own
App target	Lifecycle, DI root, module registration, deep-link → feature routing	Business logic of features
Feature Impl	Views, ViewModels, feature use-cases	Direct imports of other Feature Impls
Feature Interface	Public protocols, DTOs needed by peers	Heavy UI / networking impl
Core	Network, storage, design system, analytics abstractions	Feature-specific rules

Cross-feature navigation: Feature A imports `FeatureBInterface`, asks DI for `FeatureBBuildable`, App registered the real builder. **Compile-time** missing deps beat runtime Swinject surprises.

2.2 SPM as the packaging model

```
Package.swift
  CheckoutInterface  (leaf — builds fast, rarely changes)
  Checkout           (depends on CheckoutInterface + CartInterface + CoreNetwork)
```

```
CartInterface
Cart
CoreNetwork / CoreUI / CoreAnalytics
```

Why Interface/Impl split: Impls can compile in parallel; changing Checkout UI doesn't rebuild Cart. Circular deps fail at resolve time instead of mysterious runtime loops.

SPM vs CocoaPods (interview-ready):

- SPM: native Xcode, no Ruby, better parallel resolution, first-party Apple path
- Pods: still common in legacy monorepos; binary pods / complex resource bundles; migration cost is real — don't pretend otherwise

2.3 DI graphs without hidden globals

Prefer **constructor injection** or a **tree of components** (Needle-style) over service locators:

```
protocol CheckoutDependency: Dependency {
    var network: NetworkProviding { get }
    var cart: CartProviding { get }  // protocol from CartInterface
}

final class CheckoutComponent {
    let dependency: CheckoutDependency
    func makeCheckoutVC() -> UIViewController { /* wire VM */ }
}

// App = composition root
final class AppComponent: CheckoutDependency { /* fulfill + register */ }
```

Anti-patterns to call out:

- Feature modules calling `NetworkManager.shared`
- "God" AppDelegate holding 40 singletons
- Interface modules that secretly import Impl (defeats the graph)

2.4 Stories SDK as a reusable module (your production proof)

Treat the Stories SDK like a **product surface with a public API**, not a folder of views copied per app:

1. **Standalone package / framework** — host app supplies config (theme, analytics, media loader protocols)
2. **Stable public API** — entry points, callbacks, error types; hide internal VCs/SwiftUI
3. **Host-agnostic networking** — inject `StoriesContentProviding` so Miami Heat and portfolio apps don't fork fetch code inside the SDK
4. **Versioning mindset** — additive API changes; breaking changes = major bump + migration notes
5. **Adoption** — one implementation leveraged across the client portfolio → faster feature parity

2.5 Trade-offs

Choice	When	Cost
Interface/Impl split	Multi-team / large features	More targets, more Package.swift discipline
Static linking	Default for app modules	Larger link step; prefer over many dynamic frameworks at launch
Dynamic frameworks	App + Extension shared code	dyld overhead; Apple guidance ~keep dynamic count low
Tree DI / constructor	New modules, compile-time safety	More boilerplate at App root

Service locator (Swinject)	Legacy glue	Runtime missing-deps; harder to reason
SPM	Greenfield / Apple-first	Some binary/vendor pods still awkward
Monolith “just folders”	Tiny app / spike	Build time + merge conflict pain at scale

3. Read these

Priority	Resource	Why
Must	app-modularization.md	HLD, Interface/Impl, Needle-style DI, linking trade-offs
Deepen	WWDC: Modularizing your app / Swift packages in depth	Official SPM + module boundary language
Deepen	Uber Needle / Airbnb modularization posts (skim)	Industry vocabulary interviewers recognize
Repo	cheatsheet.md — concurrency/DI snippets	Tie modules to injectable actors/services
Story	story-bank.md#S10	Stories SDK STAR

4. Map to your work

Company / feature: Raw Engineering — Miami Heat; **standalone reusable Stories SDK** adopted across portfolio.
What you did: Designed SDK boundaries (public API, isolation from host shortcuts), shipped Instagram-style fan Stories once, reused across NBA/WNBA apps.
Interview line (≤20s):

> *“I shipped Stories as a standalone SDK with a clear public API and injected host dependencies — one module, multiple apps, no copy-paste forks.”*

→ Full STAR: story-bank.md#S10

Secondary hooks: BookMyShow Ads as a **pod/module** with POP+Generics pipeline (S1#S1); District Clean/MVVM boundaries (S9#S9).

5. Normal questions

For each: answer out loud within the time. Skeleton only — expand from memory.

Q1. Why modularize an iOS app? `(30–45s)`

Skeleton: Build time + team ownership + testability + binary/feature isolation → Interface/Impl → Stories SDK reuse as proof.
Follow-up: What’s the smallest useful first module?
Story: S10

Q2. CocoaPods vs SPM — how do you choose? `(30–45s)`

Skeleton: SPM default for new; Pods if binary/legacy; migration is incremental (leaf packages first).
Follow-up: How do you share resources (assets) across SPM targets?
Story: Ads pod migration context (S4) if asked about existing pods

Q3. What belongs in a Feature Interface module? `(30–45s)`

Skeleton: Protocols + lightweight models + builder/factory types — no UIKit heavy VCs, no URLSession impl.
Follow-up: Can Interface depend on Core? When is that OK?
Story: —

Q4. Explain a DI composition root. `(30–45s)`

Skeleton: App target wires graph; features receive protocols; no feature-level `.shared`.
Follow-up: Where do you put deep-link → feature factories?
Story: S10 (host injects providers)

Q5. How do features navigate without importing each other? `(30–45s)`

Skeleton: Depend on `FeatureBBuildable` protocol; App registers impl; coordinator uses protocol.
Follow-up: What about shared "Home" that routes to everything?
Story: —

Q6. Dynamic vs static frameworks — launch impact? `(30–45s)`

Skeleton: Many dynamic frameworks → dyld cost; prefer static for internal modules; dynamic when sharing with extensions.
Follow-up: How does this interact with app thinning / binary size?
Story: —

Q7. How do you keep modules independently testable? `(30–45s)`

Skeleton: Inject protocols; Interface packages enable mock doubles; CI can test Checkout without full app.
Follow-up: Snapshot tests — which layer owns them?
Story: S9 (tests + architecture envelope)

Q8. What is a circular dependency and how do you prevent it? `(30–45s)`

Skeleton: A Impl ↔ B Impl; prevent via Interface-only cross deps; CI/graph lint if needed.
Follow-up: Ever had SPM resolve fail — how did you debug?
Story: —

Q9. How would you extract Stories into an SDK today? `(45–60s)`

Skeleton: Public API surface → inject content + analytics + image loader → versioning → demo app → adopt in second host.
Follow-up: How do you handle theming across brands?
Story: S10

Q10. Singletons — ever OK? `(30–45s)`

Skeleton: Rare, at App/Core boundary if truly process-wide (e.g. analytics sink); never inside feature modules as hidden deps.
Follow-up: How do actors change the singleton debate?
Story: S2 (shared state patterns)

Q11. How does modularization help CI? `(30–45s)`

Skeleton: Parallel package builds; selective test of changed modules; cache SPM artifacts on runners.
Follow-up: Tie to GitHub Actions (preview Day 20).
Story: BMS CI (resume) — high level only

Q12. Binary size — modularization risk? `(30–45s)`

Skeleton: Dead code stripping helps; watch duplicate symbols / unused dynamic frameworks; budget gates in CI.
Follow-up: What metric do you track per release?
Story: —

6. Tricky questions

T1. "Just use folders as modules — why packages?" `(90–120s)`

Trap: Equating namespace folders with dependency boundaries.
Senior answer: Folders don't enforce import graphs; SPM/Xcode targets do. Enforcement > convention at 30L+ DAU team scale.

Stories reuse required a real boundary, not a folder name.
Follow-up: When are folders enough? (tiny app / spike)

T2. Service locator vs constructor injection — pick one and defend. `(90–120s)`

Trap: "Whatever the team uses."
Senior answer: Prefer constructor/tree DI for compile-time completeness; locator pushes failures to runtime and hides the graph. Admit pragmatism for legacy bridges with a migration path.
Follow-up: How do you migrate a singleton-heavy feature?

T3. Your Feature Interface started importing UIKit — what's wrong? `(90–120s)`

Trap: "UIKit is fine everywhere."
Senior answer: Interfaces should stay lean for fast rebuilds and non-UI consumers (extensions, tests). Prefer opaque builders returning `UIViewController` via type erasure / factory protocols defined carefully — or keep UI types in Impl.
Follow-up: SwiftUI `AnyView` as API — when is that a smell?

T4. Two features need the same User model — where does it live? `(90–120s)`

Trap: Duplicate models or dump everything in Core.
Senior answer: Shared kernel / `DomainModels` for true shared entities; feature-specific DTOs stay local; avoid Core becoming a junk drawer.
Follow-up: Versioning shared models across SDK hosts

T5. How do you modularize without slowing juniors down? `(90–120s)`

Trap: Process theater.
Senior answer: Templates for new Feature Interface+Impl, documented DI registration checklist, one "golden path" package, lint for forbidden imports. Velocity comes from clear rails.
Follow-up: How AI tooling fits (Context Engineering inside module boundaries — S9)

T6. Ads was a pod — how does that relate to SPM modularization? `(90–120s)`

Trap: Mixing package manager with architecture.
Senior answer: Pod vs SPM is packaging; POP+Generics + clear module API is architecture. You can have a well-modularized CocoaPod or a messy SPM. At BMS, Ads ownership + URLSession/pinning lived behind a module boundary.
Follow-up: Migration Alamofire → URLSession inside the module (S4)

T7. Build times got worse after splitting 80 modules — why? `(90–120s)`

Trap: "More modules always faster."
Senior answer: Too-fine splits, chatty Interface changes, excessive dynamic frameworks, poor caching. Use Build Timing Summary; merge leaf packages; stabilize Interface APIs.
Follow-up: Tuist/Bazel — when do you mention them?

T8. SDK versioning across Heat / Aces / Sky — strategy? `(90–120s)`

Trap: "Always latest everywhere."
Senior answer: Semantic versioning, staggered adoption, feature flags in host if needed, changelog + API diffs. Portfolio reuse only works with release discipline.
Follow-up: How do you deprecate a Stories callback?

7. Flashcards for today

Front	Back
Interface vs Impl module	Interface = protocols/DTOs; Impl = UI+logic. Cross-feature deps only on Interface · Trap: Impl→Impl · Prod: Stories public API

Composition root	App wires DI; features don't resolve globals · Trap: Swinject everywhere · Prod: host injects Stories providers
Why SPM	Native, parallel, no Ruby · Trap: ignore legacy Pod constraints · Prod: new modules SPM-first
Circular dependency fix	Depend on Interface; App registers builders · Trap: import Impl "just once" · Prod: portfolio SDK boundary
Static vs dynamic	Prefer static internal; dynamic for app↔extension share · Trap: 40 dynamic frameworks · Prod: launch dyld cost
Needle-style DI	Typed dependency protocols + component tree · Trap: runtime locator · Prod: CheckoutDependency pattern
Stories SDK lesson	Reuse = API + isolation + versioning · Trap: copy views per app · Prod: S10 portfolio adoption
Core junk drawer	Shared kernel only for true shared types · Trap: everything in Core · Prod: feature DTOs stay local
Forbidden import	FeatureAImpl must not import FeatureBImpl · Trap: convenience import · Prod: graph as architecture
Module testability	Mock Interface protocols in unit tests · Trap: only UITests on full app · Prod: S9 test discipline
Binary size gate	CI budget + thinning · Trap: modules magically shrink binary · Prod: watch duplicates
Singleton rule	OK rarely at process boundary; never hidden in features · Trap: NetworkManager.shared in VM · Prod: S2 shared-state API
First module to extract	Leaf with clear API (design system / networking / Stories) · Trap: split biggest mess first · Prod: S10
Build Timing Summary	Measure before/after splits · Trap: split blindly · Prod: xcodebuild flag
SDK host config	Theme, analytics, loader via protocols · Trap: hardcode Heat branding · Prod: multi-app Stories

8. Practice

- **Coding / SD:** Sketch (paper/whiteboard) a 4-layer module graph for a ticketing app: `App`, `Checkout`, `Search`, `Ads`, `Stories`-like `MediaStories`, `CoreNetwork`. Mark Interface vs Impl boxes and one cross-feature navigation arrow that uses a protocol only.
- **Complexity / agenda to say first:**

  > *"I'll clarify module boundaries, show Interface/Impl + DI composition root, then deep-dive Stories-as-SDK and build/link trade-offs."*

- **Optional code:** Write a tiny `Package.swift` with `StoriesInterface` + `Stories` + app-side `StoriesDependency` stub (no need to compile in CI — interview fluency).

9. Timed drill

1. Pick 3 Normal + 2 Tricky (recommend **Q4, Q9, Q6** + **T1, T6**). Record.
2. Score against answer-timing-guide.md.
3. Deliver **S10** in ≤3 min once.
4. Log misses in gotchas (especially "folders = modules" and locator vs constructor).

Day 16 — Image/Video Pipelines, Caching & Memory Pressure

> ***Full chapter:*** `weeks/week-03/day-16/` *· Revision twin — timed recall only after the full study.*

> *Week 3 · Phase: Full architecture, performance, security, platform · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- End-to-end **image pipeline**: L1 memory → L2 disk → network, with **downsample-before-cache**
- Why **decoded bitmap size** (`width × height × 4`) dominates RAM — not JPEG file size
- Request **deduplication**, **cancellation**, and cell **reuse** for scroll performance
- How **HeroWidget** pause/play ties video ads to visibility/lifecycle on a revenue-critical path
- How **Aces live audio** differs from image caching (buffering, interruptions, cold-start splash)
- Memory-pressure strategy: `NSCache` , cost limits, purge on warnings, never decode full-res on main

2. Concept deep dive

2.1 Universal image pipeline (interview diagram)

```
imageView.load(url, targetSize: bounds × scale)
    → L1 NSCache (key: url + targetSize)     HIT → UIImage
    → L2 Disk (SHA256(url))                  HIT → decode bg → L1 → UI
    → In-flight dedupe?                      YES → attach observer
    → URLSession fetch
    → ImageIO downsample (CGImageSourceCreateThumbnailAtIndex)
    → Store L1 + L2 → notify observers
```

Critical API: `CGImageSourceCreateThumbnailAtIndex` + `kCGImageSourceThumbnailMaxPixelSize` — decode *at display size*, avoid allocating a 12MP bitmap for a 120pt thumbnail.

Math to quote: 1000×1000 ARGB8888 ≈ **4MB decoded**. A feed of 30 full-res images = catastrophic. Always key cache by **url + targetSize**.

2.2 Caching policy (tie to cheatsheet)

Tier	Tech	Eviction	Notes
L1	`NSCache` / custom LRU actor	Memory warning + max cost	Auto-evicts under pressure better than raw `Dictionary`
L2	FileManager `/Caches` + metadata	LRU + TTL (~7 days)	OS may purge; OK for images
L3	CDN / URLSession	HTTP cache / ETag	Don't rely on URLCache alone for decoded bitmaps

Write policy: images usually **write-around** to L1 after decode (don't block UI writing huge disks synchronously).

2.3 Scroll-safe loading

1. **Cancellation** — cell leaves screen → cancel token; ignore late callbacks (generation id / URL match)
2. **Dedup** — 10 cells, same poster URL → one download, fan-out completions
3. **Priority** — visible > prefetch; downgrade when scrolled past
4. **Prefetch** — `UICollectionViewDataSourcePrefetching` with budget; cancel aggressive prefetch on fast fling
5. **Main thread** — never decode JPEG on main; aim <16ms frame budget

2.4 Video ads & HeroWidget (your production proof)

Revenue-critical Ads module: video inside a reusable **HeroWidget** needs explicit **pause/play** tied to:

- View visibility (partially off-screen / pager page change)
- ViewController lifecycle (`viewWillDisappear` / appear)
- App lifecycle (background → pause; foreground → resume policy)
- Cell reuse (prepareForReuse must stop player / clear item)

POP + Generics rendering pipeline: ad component protocols so new creative types don't fork the loader. Lifecycle is part of the **product contract**, not an afterthought — wasted playback and glitches hurt fill/UX on the highest-revenue module.

2.5 Aces audio + splash (media beyond images)

Las Vegas Aces: live **audio streaming** for broadcasts + **server-driven splash** to cut cold-start latency / improve freshness.

Interview distinctions:

- Audio: buffering, route changes (Bluetooth), interruptions (`AVAudioSession`), background modes
- Splash SDUI: measure **time-to-interactive**, not vanity first-frame alone
- Don't conflate image disk cache with AV player item caching — different failure modes

2.6 Trade-offs

Choice	When	Cost
NSCache	Decoded images L1	Non-deterministic eviction; no ordered LRU guarantees
Custom LRU actor	Need strict cost + metrics	More code; must handle memory warnings yourself
Downsample at decode	Always for UI display	Extra CPU; still cheaper than OOM
Keep full-res on disk only	Zoom / edit flows	Second decode pass when needed
Aggressive prefetch	Slow networks / predictable lists	Bandwidth + decode storms on fling
Third-party (Kingfisher/SDWebImage)	Speed to market	Bundle size; know internals for interviews
Pause video when offscreen	Ads / autoplay feeds	Complexity in visibility tracking

3. Read these

Priority	Resource	Why
Must	image-loading-library.md	Full HLD: cache tiers, dedupe, downsample, cancellation
Must	cheatsheet.md — Image Pipeline + Caching Policy	Numbers + universal flow
Deepen	WWDC: Image and Graphics Best Practices / Downsampling	ImageIO APIs
Story	story-bank.md#S1 · #S12	HeroWidget; Aces audio + splash

4. Map to your work

Company / feature: BookMyShow — **Ads / HeroWidget** pause-play lifecycle; Raw — **Aces** audio streaming + server-driven splash.
What you did: Type-safe ad rendering (POP+Generics); visibility-correct video lifecycle; live audio integration; splash content driven by server to improve cold start.
Interview line (≤20s):

> *"On our highest-revenue Ads surface I built HeroWidget with explicit pause/play tied to visibility — and on Aces I shipped live audio plus a server-driven splash to tighten cold start."*

→ Full STAR: S1#S1 · S12#S12

5. Normal questions

Q1. Design an image loader for a feed. `(45–60s)`

Skeleton: L1/L2/network → downsample → dedupe → cancel on reuse → memory cost math.
Follow-up: Where do you put this — Core module?
Story: —

Q2. Why not cache full-resolution UIImages? `(30–45s)`

Skeleton: Decoded size ≫ file size; 4 bytes/pixel; OOM under scroll.
Follow-up: When *do* you keep full-res?
Story: —

Q3. NSCache vs Dictionary for images? `(30–45s)`

Skeleton: NSCache evicts under pressure, thread-safer for this use; Dictionary needs manual purge + locks/actor.
Follow-up: Cost limits (`totalCostLimit`)?
Story: —

Q4. How do you handle cell reuse bugs (wrong image)? `(30–45s)`

Skeleton: Capture expected URL/token; cancel prior; clear image on reuse; ignore stale completions.
Follow-up: SwiftUI `.id` / task cancellation?
Story: —

Q5. Explain request deduplication. `(30–45s)`

Skeleton: Map URL → in-flight task + observer list; one network, N callbacks.
Follow-up: What if different targetSizes?
Story: —

Q6. Video ad is playing off-screen — what's wrong? `(30–45s)`

Skeleton: Missing visibility/lifecycle pause; wasted QoS and UX; HeroWidget contract.
Follow-up: How do you detect visibility in a pager?
Story: S1

Q7. Memory warning mid-scroll — what should happen? `(30–45s)`

Skeleton: Trim L1; pause non-visible video/decodes; keep L2; avoid main-thread work storm.
Follow-up: MetricKit memory metrics? (bridge Day 17)
Story: —

Q8. Prefetching — benefits and footguns? `(30–45s)`

Skeleton: Warm L1/L2 for next cells; cancel on direction change; cap concurrency.
Follow-up: Interaction with HTTP/2 multiplexing?
Story: —

Q9. Downsampling API you'd name? `(30–45s)`

Skeleton: ImageIO `CGImageSourceCreateThumbnailAtIndex` + max pixel size; background queue.
Follow-up: UIImageJPEGRepresentation pitfalls?
Story: —

Q10. How is live audio different from image caching? `(30–45s)`

Skeleton: Continuous buffer, session categories, interruptions; not LRU bitmap cache.
Follow-up: Background audio modes / Now Playing?
Story: S12

Q11. Server-driven splash — what do you measure? `(30–45s)`

Skeleton: Time-to-interactive / first meaningful content; fallback cached splash if network slow.
Follow-up: Tie to SDUI versioning.
Story: S12

Q12. Where do GIFs/animated images fit? `(30–45s)`

Skeleton: Separate decoder path; higher memory; often out of scope in SD interviews — call it out.
Follow-up: Video vs animated WebP trade-off?
Story: —

6. Tricky questions

T1. "URLCache is enough for images." `(90–120s)`

Trap: Confusing HTTP cache with decoded bitmap cache.
Senior answer: URLCache stores bytes; UI still needs decode/downsample and memory policy. You need an app-level image pipeline.
Follow-up: When *do* you lean on URLCache?

T2. Same URL, avatar 40pt vs hero 400pt — one cache entry? `(90–120s)`

Trap: Single key by URL only.
Senior answer: Key includes target size (and maybe format). Storing one huge bitmap and scaling down on main is a hitch source.
Follow-up: Disk store original bytes once, derive sizes?

T3. Player retain cycle in HeroWidget. `(90–120s)`

Trap: Only talk UI.
Senior answer: AVPlayer / observers / closures capturing self; pause alone doesn't release. `prepareForReuse` + break observation + nil player on disappear. Instruments Allocations for leaked players.
Follow-up: How POP helps test fake players (S1)

T4. Memory graph shows UIImage spike but disk cache is small. `(90–120s)`

Trap: Blame network.
Senior answer: Decoded bitmaps in L1 or image views retaining full-res; check downsample path and image view contents.
Follow-up: Cost calculation for NSCache

T5. Autoplay video policy for ads vs editorial. `(90–120s)`

Trap: One policy.
Senior answer: Ads may require viewability for billing; editorial may mute-autoplay. Encode policy in protocol; HeroWidget enforces lifecycle; product/legal constraints differ.
Follow-up: How you coordinated without breaking fill rates (S1)

T6. Fast fling causes decode backlog — design fix. `(90–120s)`

Trap: "Buy a bigger device."
Senior answer: Bound decode queue QoS/concurrency; cancel stale; prioritize visible indexPaths; optionally show placeholder longer.
Follow-up: OperationQueue vs task groups

T7. Aces audio drops when user takes a call. `(90–120s)`

Trap: Ignore AVAudioSession.
Senior answer: Handle interruption notifications; resume policy; route changes; don't assume pipeline equals image retry.
Follow-up: UX for "live" vs VOD resume

T8. Should Stories SDK own the image pipeline? `(90–120s)`

Trap: Hardcode Kingfisher inside SDK.
Senior answer: Inject `ImageLoading` protocol from host (Day 15) so portfolio apps share one loader/policy; SDK stays reusable.
Follow-up: S10 module boundary

7. Flashcards for today

Front	Back
Decoded image math	w×h×4 bytes · Trap: think JPEG size · Prod: downsample always
L1/L2/L3	NSCache / disk / network · Trap: URLCache = UIImage cache · Prod: cheatsheet pipeline
Thumbnail API	CGImageSourceCreateThumbnailAtIndex · Trap: draw full UIImage then scale · Prod: ImageIO
Cache key	url + targetSize · Trap: URL only · Prod: avatar vs hero
Deduplication	One in-flight per key, N observers · Trap: N downloads · Prod: feed posters
Cancellation	Token + ignore stale · Trap: late setImage · Prod: cell reuse
NSCache benefit	Evicts under pressure · Trap: deterministic LRU · Prod: memory warnings
HeroWidget rule	Pause offscreen / on disappear · Trap: fire-and-forget AVPlayer · Prod: S1 ads
prepareForReuse	Stop player, clear image, cancel loads · Trap: only clear text · Prod: revenue ads
Prefetch footgun	Decode storm on fling · Trap: prefetch everything · Prod: cancel + cap
Aces audio	Session + interruptions · Trap: treat like image TTL · Prod: S12
Splash metric	Time-to-interactive · Trap: only TTFF vanity · Prod: S12 SDUI splash
Write-around images	Decode then memory; disk async · Trap: sync disk on main · Prod: hitch budget
Memory warning	Trim L1, pause media · Trap: wipe user data · Prod: /Caches OK to lose
GIF scope	Separate path / call out of scope · Trap: same as JPEG pipeline · Prod: SD interviews
Inject loader in SDK	Protocol from host · Trap: hardcode dependency · Prod: S10 Stories

8. Practice

- **Coding / SD:** Whiteboard the image loader HLD from image-loading-library.md in 10 minutes. Then add a **video ad** box: visibility → pause/play → analytics beacon.
- **Complexity / agenda to say first:**

  > *"I'll define cache tiers and downsample math, then deep-dive cancellation/dedupe, then map HeroWidget video lifecycle and Aces audio differences."*

- **Optional code:** Sketch an `actor ImageCache` with cost-based eviction + a `load(url:targetSize:)` pseudocode that cancels via `Task`.

9. Timed drill

1. Pick 3 Normal + 2 Tricky (recommend **Q1, Q6, Q10** + **T2, T3**). Record.
2. Score against answer-timing-guide.md.
3. Deliver **S1** (HeroWidget) in ≤3 min; **S12** opener in ≤45s.
4. Log misses (esp. URLCache confusion, cache key size).

Day 17 — Performance: Instruments, MetricKit, Startup & Scrolling (p50/p90)

> *Week 3 · Revision twin · Full study: weeks/week-03/day-17/*
> *Time budget today: ~4–5 hrs (full) · revision pass ~45–60 min*

1. Outcome

Explain aloud (no notes):

- Perf loop: symptom → hypothesis → lab/field tool → fix → **p50/p90**
- Lab (Instruments) vs field (MetricKit, Firebase Performance)
- Cold start: pre-main → init → first frame → TTI
- Scroll hitch ~16ms @ 60fps
- **CORRECT:** retain cycles ≠ **Leaks** → use **Memory Graph / Allocations**
- **Verified · S5:** Firebase Performance listing/checkout/search p50/p90

2. Concept deep dive (revision)

Perf loop

Define SLI → field or lab → attribute → smallest fix → re-verify percentiles.

Instruments (purpose)

Time Profiler · Signposts · **Allocations** · **Leaks (unreachable only)** · **Memory Graph (cycles)** · Hitches · Network · App Launch · Core Animation

MetricKit

Daily-ish OS aggregates: hangs/hitches, CPU/memory, exit reasons. Pair with custom journey traces.

Startup / scroll

Pre-main cost, defer SDKs, hitch ≠ hang.

S5

Journey traces; p50/p90; no invented ms SLAs.

Trade-offs

Choice	When	Cost
Instruments	Repro lab	Not fleet-representative
MetricKit/Firebase	Field	Delayed / sampled
Optimize average	Never for UX SLOs	Hides p90

3. Map to work

S5 ≤20s: “I instrumented Firebase Performance for listing, checkout, and search — p50/p90 so decisions followed tails, not averages.”
Soft: S12 splash · S6 30%+ flows · S3 search lag checklist.

4. Drill questions (skeleton only)

N: Q1 slow-app loop · Q2 why p90 · Q6 cold-start phases · Q5 MetricKit vs Firebase
T: T2 network p90 + clean profiler · T5 p50↑ p90↓ after cache · T1 average FPS ship?

Must-correct line: "Cycles → Graph/Allocations, not Leaks."

5. Flashcards

Front	Back
Perf loop	Symptom→fix→p50/p90 · Trap: vibe · Prod: S5
Cycles tool	Graph/Allocations · Trap: Leaks · Prod: correctness
Why p90	Tail pain · Trap: averages · Prod: BMS journeys
Hitch	Missed frame · Trap: low CPU=smooth · Prod: decode on main
Lab vs field	Instruments vs MetricKit/Firebase · Trap: one device · Prod: 30L DAU

6. Timed drill

Record Q1, Q2, Q6 + T2, T5. S5 STAR ≤3 min. Score timing guide. Log: averages, Leaks-for-cycles, profiler-for-waits.

Day 18 — Crash Reporting, Observability & IMOC

> *Week 3 · Revision twin · Full study: weeks/week-03/day-18/*

1. Outcome

- Crash pipeline: handlers → **async-signal-safe** persist → next-launch upload → **dSYM**
- Crash vs OOM vs hang (CFS can look fine while users freeze)
- **Verified · S8:** 30L+ DAU, **99.95%+ CFS**, Crashlytics, **IMOC**
- **HARD: Do NOT claim S2 alone caused CFS** — S2 is path-scoped race fix; S8 is the reliability system
- IMOC: owner, blast radius, mitigate first, comms, postmortem

2. Concept deep dive (revision)

```
Init → handlers + breadcrumb ring
Crash → signal-safe mmap write → terminate
Next launch → upload → symbolicate (dSYM UUID)
```

Forbidden in handler: malloc, ObjC, locks, OSLog, network.

Triage: detect → classify → reproduce → mitigate → fix → blameless write-up.

S2 OK line: "Removed race crashes on that shared-state path; contributed to reliability."
S2 forbidden: "Dictionaries are why we have 99.95% CFS."

3. Map to work

S8 ≤20s: "At 30L+ DAU we held 99.95%+ crash-free — Crashlytics triage plus IMOC on P0/P1s, not solo hero debugging."
S2 add-on ≤90s: path-scoped + explicit non-sole-cause.

4. Drill

N: Q5 sale CFS drop · Q6 talk 99.95% · Q9 sync dicts (honesty) · Q2 signal-safe
T: T3 CFS fine but freezes · T4 iOS vs backend blame · T1 try/catch myth

5. Flashcards

Front	Back
CFS	99.95%+ · Trap: vanity · Prod: S8
S2 vs CFS	Path contributor · Trap: sole cause · Prod: S2+S8
Signal safety	No alloc/lock · Trap: OSLog in handler · Prod: mmap
Hang gap	CFS≠no freezes · Trap: trust CFS only · Prod: MetricKit
IMOC	Owner+blast+mitigate · Trap: hero debug · Prod: S8

6. Timed drill

Q5, Q6, Q9 + T3, T4. Log S2-sole-cause slips.

Day 19 — Security: ATS, SSL Pinning, Keychain & Persistence

> *Week 3 · Revision twin · Full study: weeks/week-03/day-19/*

1. Outcome

- ATS ≠ pinning
- **SPKI = SHA-256(Subject Public Key Info DER)** — **not** `SecKeyCopyExternalRepresentation` bytes
- **Verified · S4 only** for shipped: Ads URLSession + HTTPS + pinning + domain whitelist
- **S4-A1** rotation / backup / break-glass = **design**, not shipped runbook claim
- Recite **full persistence decision tree** (below)

2. Persistence decision tree (FULL — memorize)

Use case	Store	Notes
Structured queries	**SQLite (WAL)**	Off-main writes; concurrent readers
Object graph	**Core Data**	Background context writes
Secrets / tokens	**Keychain**	AfterFirstUnlock or stricter; no sync for auth
Simple prefs	**UserDefaults**	Non-sensitive **only** — never tokens
Large media	**FileManager** Caches / App Support	Caches purgeable
In-session auto-evict	**NSCache / LRU**	Memory-pressure aware
Concurrent in-session	**Actor** / serial queue	Race-free boundary

3. Pinning revision

```
ATS → system trust → SPKI pin (delegate) → domain allowlist
```

Rotation design (S4-A1): backup pins → ship client before server key rotate → staged → monitored break-glass → fail closed for Ads.

4. Map to work

S4 ≤20s: “Moved Ads off Alamofire onto URLSession with HTTPS, SSL pinning, and domain allowlisting — and I’d pair pinning with backup pins and break-glass as design.”

5. Drill

N: Q2 cert vs SPKI · Q4 rotation (label S4-A1) · Q6 persistence split · Q5 Keychain vs UD
T: T1 renew bricked app · T8 Core Data for everything · T2 UD obfuscated token

6. Flashcards

Front	Back
SPKI	DER hash · Trap: SecKey raw · Prod: S4
S4 vs S4-A1	Shipped pins · Design rotation · Trap: runbook claim
ATS	Baseline HTTPS · Trap: equals pinning · Prod: no cleartext
Tokens	Keychain · Trap: UD/base64 · Prod: auth
Tree opener	Sensitivity + access pattern · Trap: one store · Prod: table

7. Timed drill

Q2, Q4, Q6 + T1, T8. Recite 7-row tree once.

Day 20 — Deep Links, Push & CI/CD

> *Week 3 · Revision twin · Full study: weeks/week-03/day-20/*

1. Outcome

- UL vs custom schemes; AASA; cold-start **queue**; one **DeepLinkRouter** for UL + push
- Push: permission → APNs token → **Airship** → payload → router
- **GitHub Actions → build/lint → TestFlight** + pause on CFS/perf
- **Verified · S13:** Grizzlies SwiftUI↔UIKit, deeplinks, Mixpanel, Airship
- BMS CI automation; AI PR review = assist (**S9**), not owner

2. Concept deep dive (revision)

```
UL/scheme/push tap → Parser → AppRoute → AuthGate → Coordinator | PendingQueue
PR → Actions (lint/build/test) → sign/archive → TestFlight → phased → monitor → PAUSE
```

Debug UL→Safari: AASA HTTPS, teamID.bundle, paths, entitlements, CDN cache, user preference — not “iOS bug” first.

Hybrid footgun (S13): hosting identity / stack ownership — design coordinator.

3. Map to work

S13: “On Grizzlies I owned deeplinks and Airship push atop hybrid SwiftUI/UIKit, with Mixpanel analytics.”
CI: “At BMS I automated GitHub Actions into TestFlight so releases weren’t manual ceremony.”

4. Drill

N: Q3 cold-start race · Q7 CI pipeline · Q10 hybrid+deeplink · Q4 push→router
T: T1 Safari instead of app · T5 CFS drop at 10% phased · T2 malformed push JSON

5. Flashcards

Front	Back
One router	UL+push share table · Trap: dual switches · Prod: S13
Cold-start	Queue until ready · Trap: route instantly · Prod: race bugs
Airship	Engagement on APNs · Trap: skip APNs knowledge · Prod: S13
Actions→TF	Automate build/lint/upload · Trap: no pause gates · Prod: BMS
AI PRs	Assist · Trap: "AI approved" · Prod: S9

6. Timed drill

Q3, Q7, Q10 + T1, T5.

Day 21 — System-Design Mock #3 (45 min)

> *Week 3 · Revision twin · Full study: weeks/week-03/day-21/*
> ***Do the live 45-min mock from the full chapter scripts.***

1. Outcome

- Spine: clarify→HLD→API→deep dive→ops
- Full scripts exist for **SDUI OR networking+pinning** — pick one live
- Two-layer scoring (structure + content honesty)
- Resume-only metrics; no fabricated QPS

2. Spine (memorize)

```
0–5 clarify · 5–15 HLD · 15–25 API · 25–40 dive · 40–45 ops
```

Opener: "~5 on scope/scale, architecture pass, deep-dive X and Y — match?"

Prompt A deep dives

Versioning/unknown fallback · action routing · (cache / perf / allowlist)

Prompt B deep dives

Single-flight refresh · SPKI+rotation **design (S4-A1)** · (cache / p50/p90)

3. Honesty guards

Guard
S4 shipped pins; S4-A1 rotation = design
SPKI = DER hash ≠ SecKey raw bytes
S2 ≠ sole CFS cause
30L DAU / 99.95% CFS only where natural

4. Rubric (100) — pass ≥70, ops ≥6, no fake metrics

Dimension	Pts
Agenda & clarify	15
HLD clarity	20
API / data	15
Deep dive quality	25
Production grounding	10
Ops / rollout	10
Communication	5

Two-layer: Structure (timebox/diagram/ops) + Content (provenance/SPKI/S2-S8 labels).

5. Drill / practice

1. Live 45-min mock (record).
2. Score harshly with Part II scoring Qs in full `04-questions.md`.
3. Lightning 20-min on the other prompt.

6. Flashcards

Front	Back
45 spine	5/10/10/15/5 · Trap: dive first · Prod: cheatsheet
SDUI unknown	Placeholder+metric · Trap: crash · Prod: S3-A1
Pin dive	SPKI DER + S4-A1 design · Trap: SecKey/runbook · Prod: S4
Ops closer	Failures+SLIs+pause · Trap: end on boxes · Prod: S8/S5
Pass bar	≥70 + ops≥6 + no fake metrics · Trap: pretty diagram only

---

Revision Week 04

Day 22 — DSA Trees: BFS / DFS Patterns

> *Week 4 · Phase: DSA polish + interview communication · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- When to reach for **BFS** vs **DFS** (level order / shortest path in unweighted trees vs path/subtree recursion) and state **time/space** in one breath.
- The four DFS shapes: **preorder / inorder / postorder / "any order recurse children"**, plus iterative stack variants.
- How to open every tree problem with a **2–3 min "say this first"** script before typing code.
- Solve 8–12 medium tree problems from the track list, narrating complexity *before* implementation.
- Map tree traversal patterns to production mental models (UIKit view hierarchy walk, SDUI node tree, deeplink route tree) without forcing a fake LeetCode story.

2. Concept deep dive

2.1 Pattern catalog (memorize the trigger → algorithm)

Trigger phrase in prompt	Default pattern	Why
"Level by level", "leftmost/rightmost at each depth", "zigzag level order"	**BFS** + queue	Natural FIFO; `O(w)` space for width
"Shortest path" in **unweighted** tree/graph	**BFS**	First time you hit target = shortest
"Path sum", "root-to-leaf", "all paths"	**DFS** (usually preorder with accumulator)	Path state rides the recursion stack
"Subtree of another", "same tree", "symmetric"	**DFS** compare two pointers / mirrored	Structural recursion
"LCA of two nodes"	**DFS** postorder return markers	Bottom-up combine
"Serialize / deserialize", "clone N-ary"	**DFS** or BFS with null markers	Encoding must be reversible
"Max depth / diameter / balanced?"	**DFS** return height + side effects	One pass compute multiple facts
"Validate BST"	**DFS** with low/high bounds (or inorder)	Inorder must be sorted
"k-th smallest in BST"	**Inorder** (stack or Morris)	Sorted order
"Connect next pointers / cousins"	**BFS** with level size loop	Sibling awareness

Complexity script (say verbatim until automatic):

> *"n = number of nodes. Visit each once → **O(n)** time. Recursion depth or queue holds up to **O(h)** or **O(w)** space; worst case skewed tree **O(n)**, balanced **O(log n)** height."*

Never say "O(1) space" for recursive DFS unless you explicitly discuss the call stack as free (interviewers will push). Prefer: "auxiliary O(1) besides the call stack O(h)."

2.2 BFS skeleton (level-order discipline)

Always use the **level-size** idiom so you never confuse "nodes in queue" with "this level":

```
queue = [root]
while queue not empty:
  levelCount = queue.count
  for _ in 0..<levelCount:
    node = dequeue
    // process node (or collect into level array)
    enqueue children if non-nil
```

Interview habits:

1. Clarify: binary vs N-ary? values unique? need path or just bool? mutable tree OK?
2. Draw 5–7 node example; walk one BFS level out loud.
3. State edge cases: empty root, single node, skewed, duplicate values, negative vals.

2.3 DFS skeletons

Recursive (default for trees):

```
func dfs(node):
  if node == nil: return base
  // pre: act before children
  L = dfs(node.left)
  // in: act between (BST / inorder)
  R = dfs(node.right)
  // post: combine L,R with node
  return combined
```

Iterative preorder (stack): push root; while stack: pop, visit, push right then left (so left processed first).

Iterative inorder: walk left pushing; pop/visit; go right — classic BST k-th / validate helper.

When iterative wins in interview: "I need explicit control / avoid deep recursion on skewed input" or follow-up asks for O(1) Morris (mention only if comfortable — don't derail).

2.4 Trade-offs

Choice	When	Cost
BFS	Levels, width-aware, unweighted shortest	Queue `O(w)`; poor for deep path reconstruction without parent map
DFS recursive	Structure, path, subtree, LCA	Call stack `O(h)`; skewed → stack overflow risk in prod (rare in LC)
DFS iterative	Same as DFS, explicit stack	More boilerplate; easier to pause/resume
Inorder on BST	Sorted queries, validate, k-th	Wrong on non-BST; don't force inorder on general binary trees
Parent pointer / map	Path to root, cousin distance	Extra `O(n)` space; clarify if tree nodes have parent
Two-pass vs one-pass	Diameter (height + max path)	Prefer one DFS returning height and updating global max

2.5 "Say this first" master script (2–3 min coding approach)

Use every tree problem today. Aligns with answer-timing-guide.md coding section.

1. **Clarify (20–30s):** "Binary tree or BST? Need the path or a boolean? Can I mutate? n up to?"
2. **Brute (20–30s):** "I could enumerate all paths / compare every subtree — typically $O(n^2)$."
3. **Optimize (45–60s):** Name pattern (BFS level loop / DFS postorder / bounds for BST) + why correct.
4. **Complexity (15s):** O(n) time, O(h) or O(w) space; worst case.
5. **Edges (20s):** null root, one child, skewed, duplicates.
6. **Commit (5s):** "I'll code the optimized version now."

Example opener — Binary Tree Level Order:

> *"I'll BFS with a queue and process by level size so each array in the result is one depth. Empty root returns []. Time O(n), space O(w) for the widest level. Edges: single node, skewed left chain. Coding now."*

Example opener — Lowest Common Ancestor (binary tree, not BST):

> *"Postorder DFS: recurse left/right; if both sides find targets, current is LCA; if one side finds, bubble that node up. Assumes both nodes exist in tree — I'll confirm. O(n)/O(h)."*

Example opener — Validate BST:

> *"Not enough to compare only to parent. I'll DFS with inclusive/exclusive low-high bounds, or inorder and ensure strictly increasing. O(n)/O(h)."*

3. Read these

Priority	Resource	Why
Must	coding/dsa-track.md — Trees / BFS / DFS section	Canonical problem list + your solve log
Must	timing/answer-timing-guide.md § Coding approach	Enforce 2–3 min agenda before code
Deepen	LeetCode Explore: Binary Tree (BFS/DFS cards)	Visual reinforcement of level vs recurse
Deepen	Swift `Collection` / recursive enums mental model	Trees as value types vs class nodes (interview aside)
Repo	Production analogy only — no new SD doc today	Save bandwidth for Day 24 AI + Day 27 mock

4. Map to your work

Company / feature: BookMyShow SDUI / header tree; Raw deeplink + navigation graph; view hierarchy debugging.
What you did: Walked nested UI / CMS component trees, routed deeplinks through hierarchical destinations, reasoned about parent-child layout and lifecycle — same *shape* as tree DFS/BFS, different domain.
Interview line (≤20s):

> *"Tree BFS/DFS show up in product as view and SDUI trees — I pick BFS for level-aware layout passes and DFS when I need path or subtree decisions."*

→ Full STAR (architecture, not DSA): story-bank.md#S3 · #S10 · #S13

Do not invent a LeetCode metric. Keep resume numbers for behavioral days: **30L+ DAU**, **99.95% crash-free**, **30%+ nav reduction** (LE sheet).

5. Normal questions

For each: answer out loud within the time. Skeleton only — expand from memory.

Q1. When do you choose BFS over DFS on a tree? `(30–45s)`

Skeleton: Levels / width / unweighted shortest → BFS; path/subtree/structure → DFS. Space: queue width vs recursion height.
Follow-up: "What if the tree is extremely wide?" → BFS memory; maybe DFS + explicit depth param.
Story: SDUI/layout "by section depth" analogy only if natural — don't force.

Q2. What's the time and space of tree traversal? `(30–45s)`

Skeleton: Visit each node once → O(n). Space O(h) recursion or O(w) queue; worst O(n) skewed/full wide.
Follow-up: Balanced BST height? → O(log n).
Story: —

Q3. How do you validate a BST correctly? `(30–45s)`

Skeleton: Bounds (low, high) on each node **or** inorder strictly increasing. Parent-only check is wrong.
Follow-up: Duplicates allowed? → clarify strict vs non-strict.
Story: —

Q4. Explain level-order traversal implementation. `(30–45s)`

Skeleton: Queue + `levelCount = queue.count` inner loop; collect or process per level.
Follow-up: Zigzag? → alternate reverse or deque.
Story: —

Q5. How does recursive LCA work on a binary tree? `(45s)`

Skeleton: If node is null or matches p/q return node; recurse L/R; if both non-null return node else return non-null side.
Follow-up: BST LCA? → walk comparing values — O(h).

Story: —

6. Tricky questions

T1. “Can you do this in O(1) extra space?” `(90–120s)`

Trap: Forgetting call stack; proposing Morris without knowing it; claiming BFS is O(1).
Senior answer: Distinguish auxiliary heap space vs call stack. Morris threading for inorder exists but mutates links temporarily — say trade-offs. For BFS, O(w) is inherent unless output-only tricks. Prefer honest O(h) recursive.
Follow-up: Interviewer wants Morris — outline link/restore; if shaky, stay with stack iterative.

T2. “Diameter of binary tree — why isn’t it just max depth left + right at root?” `(90–120s)`

Trap: Computing only at root; missing diameter entirely in a subtree.
Senior answer: Diameter = max over all nodes of leftHeight + rightHeight; DFS returns height, updates global max edges/nodes-1 as you go. One O(n) pass.
Follow-up: Define diameter as edges vs nodes — confirm with interviewer.

T3. “Serialize binary tree — how do you handle nulls and ambiguity?” `(90–120s)`

Trap: Preorder without null markers → ambiguous structure.
Senior answer: Preorder with explicit null tokens, or BFS with nulls (LeetCode codec style); deserialize with queue or index pointer. Complexity O(n). Mention JSON/SDUI schemas need versioning too (light production nod).
Follow-up: N-ary? → child count header or delimiter scheme.

T4. “DFS recursion blows the stack on skewed trees — what do you do in production?” `(90–120s)`

Trap: “Never happens” or only “increase stack size.”
Senior answer: Prefer iterative; validate input depth; chunk processing; for UI trees depths are usually small but CMS/SDUI can nest — guard with max depth + fail-soft UI. Same judgment as on-device AI fail-soft (preview Day 24).
Follow-up: Tie to crash-free culture at BMS (99.95%) — defensive depth limits, not hero recursion.

7. Flashcards for today

Front	Back
BFS vs DFS trigger	Levels/shortest unweighted → BFS; path/subtree/LCA/structure → DFS · Trap: using DFS for “level averages” · Prod: layout pass vs path resolve
Level-size BFS idiom	`for i in 0..<queue.count` each wave · Trap: mixing levels · Prod: batch UI updates by depth
BST validate	Bounds or inorder sorted · Trap: only compare parent · Prod: schema validation analogy
LCA binary tree	Postorder both-sides found → node · Trap: assuming BST walk · Prod: nearest shared nav ancestor
Tree complexity script	O(n) time; O(h)/O(w) space; worst O(n) · Trap: saying O(1) recursive · Prod: skewed CMS tree
Diameter one-pass	DFS height + global left+right max · Trap: root-only sum · Prod: longest dependency chain metaphor
Serialize need	Explicit nulls or count headers · Trap: values-only preorder · Prod: SDUI versioning
Inorder use	BST sorted ops, k-th · Trap: inorder on non-BST for “sorted” · Prod: —
Say this first	Clarify → brute → optimize → complex → edges → code · Trap: silent coding · Prod: senior signal
Symmetric tree	Mirror DFS or BFS pair queue · Trap: only check root children · Prod: mirrored layout QA

8. Practice

- **Coding / SD:** Work the **Trees · BFS/DFS** list in dsa-track.md. Target **8–12** problems today (prefer Medium). Suggested set (adjust to track IDs if numbered differently):

  1. Binary Tree Level Order Traversal
  2. Binary Tree Zigzag Level Order Traversal
  3. Maximum Depth of Binary Tree
  4. Diameter of Binary Tree
  5. Path Sum II (or Path Sum)
  6. Lowest Common Ancestor of a Binary Tree
  7. Validate Binary Search Tree
  8. Kth Smallest Element in a BST
  9. Symmetric Tree
  10. Binary Tree Right Side View
  11. Serialize and Deserialize Binary Tree *(stretch)*
  12. Construct Binary Tree from Preorder and Inorder *(stretch)*

  Log each: pattern tag · complexity said aloud? (Y/N) · time to first correct · one miss note.

- **Complexity / agenda to say first:** Before *every* solve, record 60–90s voice memo of the master script. No memo → doesn't count as done.

9. Timed drill

1. Pick **3 Normal + 2 Tricky** from §§5–6. Record.
2. Score against answer-timing-guide.md (target ≥4 Normal, ≥3 Tricky).
3. Log misses in gotchas (especially: BST validate trap, diameter root-only, O(1) space confusion).
4. Bonus 10 min: cold-open **Right Side View** — only BFS agenda + code outline, no full polish.

Day 23 — DSA HashMaps / Heaps + Mixed Unknown-Pattern Simulation

> *Week 4 · Phase: DSA polish · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- HashMap patterns: **frequency map, index map, prefix/state key, sliding window counts, two-sum family**, and when hashing beats sorting.
- Heap patterns: **Top-K, merge K, running median sketch, "K closest", schedule/next event** — min-heap vs max-heap intuition in Swift (`Heap` / custom, or sorted array for tiny K with caveat).
- A **mixed unknown-pattern** drill: 60–90s to classify *before* coding, even when the topic isn't labeled "heap."
- Complexity scripts: average vs worst hash; heap `O(log k)` ops; space trade-offs.
- Stop pattern-worship: if stuck, fall back to brute → constrain → named structure.

2. Concept deep dive

2.1 HashMap pattern catalog

Trigger	Pattern	Complexity script
"Two sum / complement"	value → index map	O(n) time / O(n) space; sorted two-pointer if mutable sort OK

“Group anagrams / isomorphic”	canonical key → list	Key = sorted string O(k log k) or count tuple O(k)
“Subarray sum equals K”	prefixSum → frequency	O(n); careful with 0 / negatives
“Longest substring without repeat”	window + last-seen index/count	O(n) sliding window
“First unique / LRU building block”	ordered dict mental model	Dict + doubly list in design interviews; LC may allow `Dictionary` + heuristics
“Clone graph / random pointer”	old → new map	O(n); DFS/BFS + map
“Time-based / snapshot”	key → sorted timestamps	Binary search values

Say this first (hash):

> *“I’ll clarify uniqueness and whether order matters. Brute is nested loops O(n²). Optimized: hash complements / frequencies for expected O(n). Worst-case hash degradation is theoretical on interviews — I’ll state expected O(n). Edges: empties, duplicates, negatives, unicode keys if strings.”*

2.2 Heap pattern catalog

Trigger	Pattern	Notes
Top K frequent / largest	size-K heap	Freq map then heap; or bucket sort O(n) when constrained
Kth largest in stream	min-heap size K	Root = kth
Merge K sorted lists	min-heap of heads	O(N log K)
K closest points	max-heap size K by distance	Or quickselect
Meeting rooms / CPU intervals	sort + min-heap of ends	Classic scheduling
Reorganize string / task scheduler	max-heap by freq + cooldown	Greedy with heap

Heap complexity script:

> *“Each offer/poll is O(log k). Building from n with heapify is O(n). For top-K I keep heap size K → O(n log k), better than full sort O(n log n) when k ≪ n.”*

Swift interview note: Prefer clarity — `var heap` with clear comparator comments, or sort when n is small and you say so. Don’t lose the round fighting PriorityQueue boilerplate; narrate operations.

2.3 Mixed unknown-pattern simulation (today’s differentiator)

Interviewers often paste an unlabeled Medium. Your job in the first **90 seconds**:

1. **Restate + constraints** (sorted? online? duplicates? memory?).
2. **Brute** one sentence.
3. **Classify:** array scan / two pointer / window / hash / heap / tree / graph / binary search / DP-lite.
4. **Pick one** and justify; mention alternative you rejected.

Classification cheat sheet (ask yourself):

Question	If yes →
Need counts / complements / seen set?	Hash

Always care about “best K so far”?	Heap
Contiguous constraint?	Window / prefix
Sorted property exploitable?	Two pointer / binary search
Hierarchy / nested?	Tree DFS/BFS (Day 22)
Overlapping subproblems stated?	DP (don’t force today)

Drill protocol (90 min block):

- Blindfold topic: pick **6** mixed problems from dsa-track.md Mixed / review set (or random Mediums tagged Array/Hash/Heap/Tree).
- For each: **timer 90s classification only** (no code) → write pattern name on paper → then solve or skip to next if over time budget.
- Goal: **5/6 correct pattern tags** before coding.

2.4 Trade-offs

Choice	When	Cost
HashMap	Lookups, counts, grouping	O(n) space; worse locality; key design bugs
Sort + two pointer	Memory tight, sorted OK	Destroys indices unless index pairs kept; O(n log n)
Heap size K	Top-K online or streaming	O(n log k); comparator bugs
Full sort	Need full order anyway	Overkill for top-K
Bucket / counting sort	Freq in bounded range	Extra O(n) buckets; interview clarity win for “top K frequent”
TreeMap / sorted dict	Range queries on keys	Heavier; rarely needed on iOS LC

2.5 Failure modes seniors mention

- Hashing **mutable** objects as keys (don’t).
- Forgetting to **decrement/remove** keys at zero in window maps (memory + correctness).
- Using max-heap when you needed min-heap for “Kth largest stream.”
- Returning heap contents **unsorted** when problem wants sorted top-K — clarify.
- Mixed drill: jumping to DP because the problem “feels hard.”

3. Read these

Priority	Resource	Why
Must	coding/dsa-track.md — HashMap, Heap, Mixed sections	Problem IDs + logging format
Must	timing/answer-timing-guide.md	Coding agenda under pressure
Deepen	Day 22 tree flashcards (quick skim)	Mixed set will include 1–2 trees
Repo	—	Pure DSA day; AI is Day 24

4. Map to your work

Company / feature: BMS search debounce + request coalescing; synchronised dictionaries; analytics event aggregation; GymFlow cosine top-K neighbors.
What you did: Frequency/coalesce maps for network & state; top similar exercises via embeddings ≈ “K nearest” (heap/select mental model); crash/IMOC triage often starts with **keyed aggregation**.
Interview line (≤20s):

> *"In product code I use maps for coalescing and counts, and top-K style selection for recommenders — same complexity instincts as heap/hash interview problems."*

→ STAR: story-bank.md#S2 · #S3 · #S16

5. Normal questions

Q1. Two Sum — hash vs sort? `(30–45s)`

Skeleton: Hash complement O(n)/O(n); sort + two pointer O(n log n)/O(1) aux but loses indices unless store pairs. Prefer hash when need original indices.
Follow-up: Multiset duplicates?
Story: —

Q2. Top K frequent elements approach? `(30–45s)`

Skeleton: Count map → min-heap size K **or** bucket by frequency O(n). State O(n log k) vs O(n).
Follow-up: Stream version?
Story: GymFlow "top similar" at tiny K — heap/select OK.

Q3. What is average vs worst-case for Dictionary? `(30–45s)`

Skeleton: Expected O(1) lookup; pathological collisions O(n) (theoretical in interview). Design: good hash + load factor.
Follow-up: Swift `Hashable` pitfalls?
Story: —

Q4. When is a heap the wrong tool? `(30–45s)`

Skeleton: Need full sort; need random access by key; K ≈ n (just sort); need arbitrary delete-by-id without handle.
Follow-up: Indexed priority queue?
Story: —

Q5. Sliding window + hash — longest substring without repeating? `(45s)`

Skeleton: Right expands; while duplicate, advance left using last-seen/count; track max length. O(n).
Follow-up: Exactly K distinct?
Story: —

6. Tricky questions

T1. "Subarray sum equals K with negatives — why can't I use two pointers?" `(90–120s)`

Trap: Sliding window monotonicity assumption.
Senior answer: Negatives break window shrink logic. Use prefix sums + hash of prefix frequencies. O(n)/O(n). Two pointers work for non-negative only.
Follow-up: Count of such subarrays vs boolean.

T2. "Merge K sorted lists — why not concatenate and sort?" `(90–120s)`

Trap: Accepting O(N log N) without discussing K.
Senior answer: Concat+sort works but heap-of-heads is O(N log K); better when K ≪ N. Mention divide-and-conquer merge.
Follow-up: Memory if lists are lazy streams.

T3. "Design a structure for O(1) insert, delete, getRandom" `(90–120s)`

Trap: Only HashMap (no random) or only array (delete O(n)).
Senior answer: Dict value→index + array swap-with-last delete. Classic.
Follow-up: Duplicates allowed? → multiset variant.

T4. Unknown-pattern meta: "I don't recognize this — what do you do?" `(90–120s)`

Trap: Silence or random DP.
Senior answer: Clarify → brute on example → name constraints (online? contiguous? K?) → pick hash/heap/window/tree → state complexity → code brute if needed then optimize. Seniors communicate uncertainty with a plan.
Follow-up: Tie to machine round Day 25 — same discipline under 3 hrs.

7. Flashcards for today

Front	Back
Two Sum default	Hash complement O(n) · Trap: sort loses indices · Prod: id→model maps
Top-K heap size	Min-heap size K → O(n log k) · Trap: max-heap confusion for Kth largest stream · Prod: top similar items
Prefix sum + hash	Subarray sum K with negatives · Trap: two pointer · Prod: analytics rollups
Window + counts	Shrink when invariant breaks · Trap: forget remove at 0 · Prod: debounce coalescing keys
Merge K lists	Heap heads O(N log K) · Trap: full sort only · Prod: merge paginated cursors metaphor
Hash complexity say	Expected O(1); space O(n) · Trap: claiming worst O(1) always · Prod: synchronised dict access ≠ hash math
Unknown pattern 90s	Clarify brute classify pick · Trap: code silently · Prod: senior signal
Bucket top-K freq	O(n) when freq ≤ n · Trap: always heap · Prod: —
Heap wrong when	Need full order / keyed delete · Trap: heap for everything · Prod: —
Clone graph	Hash old→new + DFS/BFS · Trap: no map infinite loop · Prod: object graph copy

8. Practice

- **Coding:** From dsa-track.md:
  - **Hash block (4–5):** Two Sum, Group Anagrams, Subarray Sum Equals K, Longest Substring Without Repeating, Top K Frequent (hash+heap bridge).
  - **Heap block (3–4):** Kth Largest Element, Merge K Sorted Lists, K Closest Points, Task Scheduler *(optional)*.
  - **Mixed simulation (6 tags @ 90s each, then solve ≥3):** include at least one tree from Day 22 leftovers and one "feels like DP but is hash/window."
- **Complexity / agenda to say first:** Every problem: spoken script before code. For mixed set, **pattern tag must be written first** or restart the timer.
- **Anti-perfectionism:** If 25+ min stuck, write brute, state optimize idea, move on — log and revisit after mixed drill.

9. Timed drill

1. Pick **3 Normal + 2 Tricky** (§§5–6). Record.
2. Score vs answer-timing-guide.md.
3. Extra: **one cold mixed Medium** — only 3-minute approach aloud (no code). Grade: Did you classify correctly?
4. Log gotchas: negatives + window, heap direction, silent coding urge.

Day 24 — On-Device AI Architecture (FinTrack / GymFlow)

> *Week 4 · Phase: AI system design + privacy judgment · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- End-to-end **on-device AI assistant** architecture from on-device-llm-ai-engine.md: RAG → prompt assemble → local inference → stream tokens → **hybrid fail-soft** fallback.
- **FinTrack:** BM25 offline RAG + Apple Intelligence / Foundation Models bridge + deterministic rule fallback; **privacy-first** (no cloud sync of financial data).
- **GymFlow:** INT8 **MiniLM TFLite** embeddings + cosine top-K + **TF-IDF fail-soft**.
- Why seniors design **degradation paths** (unsupported device, thermal, model missing, low battery) before demoing the happy path.
- Answer both **normal** and **tricky** AI interview questions with agenda + trade-off + production proof (S15/S16).

2. Concept deep dive

2.1 Staff-level mental model (draw this every time)

```
User query
   → Device eligibility (OS, Neural Engine, memory, thermal, Low Power)
   → Retrieve local context (BM25 / vector / rules)
   → Assemble prompt (budget tokens; strip/minimize PII for any cloud path)
   → Local generate OR cloud fallback OR deterministic template
   → Stream tokens to UI (AsyncSequence / partial updates)
   → Log quality + failure reason (not raw private prompts in cleartext analytics)
```

Agenda opener for SD (10s):

> *"I'll cover privacy constraints, retrieval, on-device inference, hybrid fallback, and ops metrics — starting with FinTrack/GymFlow as concrete instances."*

2.2 Primer you must say cleanly (45–60s each)

Concept	One-liner + mobile sting
Token	Model I/O unit ≈ ¾ word; context window is a hard budget
Quantization	FP16 3B ≈ huge RAM; INT4/INT8 makes on-device feasible; quality trade-off
Embedding	Fixed-dim vector of meaning; cosine ≈ semantic closeness
RAG	Retrieve private/local chunks → stuff into prompt; model never "trained" on your Hive DB
KV-cache	Speeds decode; eats RAM; drop under memory pressure
Hybrid router	Local first; cloud/rules when thermal, complexity, or capability fails
BM25	Classical lexical retrieval — strong offline, no embedder required
Fail-soft	Feature degrades to useful subset, never hard-crashes the app

2.3 FinTrack — privacy-first Savings Coach

Problem: Expense coach that must not ship ledger data to a cloud LLM by default.

Architecture you shipped / can defend:

1. **Local store:** Flutter + Hive; biometric lock; no cloud sync of financial records.
2. **Retrieval: BM25** over local spending index (lexical, offline, explainable).
3. **Generation path:** `flutter_native_ai` → **Apple Foundation Models** / platform AI where available; else Gemini Nano-class path when present.

4. **Fail-soft: Deterministic rule engine** (budget thresholds, category heuristics) when model unavailable — still a coach, not a blank screen.
5. **Privacy invariant:** Prefer on-device; if any cloud path exists in a design interview variant, **PII sanitize + user consent + minimize fields**.

Interview line (≤20s):

> *"FinTrack is local-first — BM25 RAG over Hive, Apple Intelligence when present, rules when not; financial data doesn't leave the device."*

→ STAR: story-bank.md#S15

2.4 GymFlow — edge embeddings recommender

Problem: Workout recommendations without cloud LLM cost/latency/privacy drag.

Architecture:

1. **INT8 MiniLM TFLite** + WordPiece tokenizer in Dart.
2. Embed query / user context; **cosine similarity** over exercise catalog; trainer routine as RAG context.
3. **Fail-soft: TF-IDF** lexical fallback if TFLite model missing/slow/thermal.
4. Same product principle as FinTrack: **retrieval + ranking + degradation**, not "call an LLM."

→ STAR: story-bank.md#S16

2.5 Map portfolio → generic on-device LLM doc

Use the repo doc as the **interview HLD**; use FinTrack/GymFlow as **proof you shipped a slice**.

Doc subsystem	Your proof
Local RAG / vector or lexical index	FinTrack BM25; GymFlow MiniLM + cosine
Quantized runtime	GymFlow INT8 TFLite; Apple FM path on FinTrack
Hybrid router / thermal	Speak design intent; implement eligibility checks
Token streaming UI	Partial answer updates; cancel on navigate away
Memory / Jetsam	Unload model, clear caches on warning
Privacy guard	FinTrack no-sync invariant; cloud only with sanitize

2.6 Trade-offs

Choice	When	Cost
BM25 lexical RAG	Offline, explainable, no embedder binary	Weaker paraphrase matching ("UBER" vs "rideshare")
MiniLM / vector RAG	Semantic paraphrase	Model size, tokenizer bugs, ANR/jank if on main
On-device LLM generate	Privacy, latency, offline	Quality, thermal, device coverage
Cloud LLM fallback	Hard reasoning, huge context	Privacy, cost, latency, consent
Deterministic rules / TF-IDF	Fail-soft always-on	Less "magical"; must set user expectation
Apple Intelligence / FM	Native path, privacy story	OS/device gating; abstraction for Android
Always-cloud "AI feature"	Fast demo	Fails senior privacy + offline bars — avoid as default

2.7 Fail-soft matrix (memorize)

Failure	Detection	Fallback
OS / device lacks FM / TFLite	Capability flag at launch	Rules / TF-IDF; hide generative UI chrome
Model file missing / checksum fail	Load error	Same; remote config to disable
Thermal `.serious` / Low Power	`ProcessInfo`	Pause inference; lexical or cached tips
Memory warning	Notification	Unload weights, clear KV; degrade
Retrieval empty	Top-K = 0	Honest empty state + generic tips — don't hallucinate spend data
Cloud path needed (design)	Router decision	Sanitize PII; TLS; timeout → local message

2.8 Privacy-first script (90s)

> *"Financial and health-adjacent data stays on device. Retrieval runs locally. Generative models are optional accelerators behind capability checks. Analytics get coarse events — coach_shown, fallback_rule — not raw transactions. If a hybrid cloud path is required, it's consent-gated, field-minimized, and never the only path."*

3. Read these

Priority	Resource	Why
Must	on-device-llm-ai-engine.md	HLD, RAG, memory, hybrid router, mock Qs
Must	story-bank.md S15, S16	Timed STAR proof
Deepen	how-ai-summarization-agents-work.md	Agent loop vocabulary if asked
Deepen	Apple Foundation Models / WWDC on-device AI sessions	Platform vocabulary
Repo	cheatsheet.md	45-min SD timing for Day 27

4. Map to your work

Company / feature: FinTrack Savings Coach; GymFlow recommender; contrast with BMS scale judgment (you know when *not* to run heavy on-device on 30L DAU paths without caps).
What you did: Privacy-first RAG + native AI bridge + deterministic/TF-IDF fail-soft; Play Store ops awareness (Crashlytics, Remote Config).
Interview line (≤20s):

> *"I treat on-device AI as retrieval plus guarded inference with an explicit degrade path — FinTrack BM25 and GymFlow MiniLM both ship that way."*

→ Full STAR: story-bank.md#S15 · #S16

Scale anchors (use when contrasting consumer apps): BookMyShow **30L+ DAU**, **99.95% crash-free** — you would not roll unconstrained on-device LLM on main checkout without kill switch + sampling (tie to feature-flag thinking).

5. Normal questions

Q1. What is on-device RAG? `(30–45s)`

Skeleton: Embed or index local docs → retrieve top-K → inject into prompt → generate. Keeps private data local; model stays general.

Follow-up: BM25 vs embeddings?
Story: FinTrack BM25.

Q2. Why quantize models on mobile? `(30–45s)`

Skeleton: RAM/disk; INT8/INT4 trade accuracy for fit; mmap + Neural Engine. Unquantized 3B blows Jetsam budgets.
Follow-up: Quality monitoring?
Story: GymFlow INT8 MiniLM.

Q3. How do you stream tokens to UI without jank? `(30–45s)`

Skeleton: Background inference; `AsyncSequence` /callbacks; main-actor append; cancel on dismiss; batch UI updates.
Follow-up: Partial markdown rendering?
Story: —

Q4. Privacy benefits of on-device AI? `(30–45s)`

Skeleton: Data minimization; offline; lower exfil risk; aligns Apple privacy narrative; still need local disk security (biometric, encryption).
Follow-up: Analytics?
Story: FinTrack no cloud sync.

Q5. What is fail-soft in an AI feature? `(30–45s)`

Skeleton: Capability detect → degrade to rules/TF-IDF/cached → user-visible honesty; never crash; remote config kill.
Follow-up: How do you measure fallback rate?
Story: S15/S16.

Q6. BM25 vs vector search — when each? `(45s)`

Skeleton: BM25: keyword, offline light, explainable. Vector: paraphrase/semantic. Hybrid retrieval often best. FinTrack chose BM25 for spend text + simplicity.
Follow-up: Hybrid reciprocal rank fusion?
Story: FinTrack vs GymFlow split.

6. Tricky questions

T1. “Your on-device model hallucinates a transaction amount — who’s at fault and how do you prevent it?” `(90–120s)`

Trap: “Models just do that” with no product control.
Senior answer: Never let generative text be source of truth for money. RAG must ground numbers from **deterministic DB reads**; prompt instructs “only use provided context”; UI shows retrieved rows; rules path for balances; eval set of prompts; fallback if retrieval score low.
Follow-up: Show citation chips from retrieved chunks.

T2. “Why not always call GPT-4o from the phone?” `(90–120s)`

Trap: Cost-only answer.
Senior answer: Privacy, offline, latency, cost, App Store review optics for finance; cloud as optional escalate with consent. On-device for personal context; cloud for heavy reasoning if ever needed.
Follow-up: How would BMS ads AI differ at 30L DAU? → sampling, server-side, strict budgets.

T3. “Thermal throttling mid-generation — design the behavior.” `(90–120s)`

Trap: Ignore ProcessInfo; block UI.
Senior answer: Observe thermal/Low Power; pause or switch to cloud/rules; keep last partial answer; don’t spin until Jetsam. GymFlow: skip embed refresh, use TF-IDF.
Follow-up: Metrics: throttle_events, completion_rate.

T4. “How is this different from a server RAG demo you copied from a blog?” `(90–120s)`

Trap: Buzzword soup.
Senior answer: Mobile constraints (RAM ≤ hundreds of MB, battery, offline), **fail-soft**, privacy invariants, capability matrix across devices, cancelation/lifecycle, and shipped FinTrack/GymFlow proof — not notebook cosine demos.
Follow-up: Walk FinTrack vs doc HLD mapping (§2.5).

T5. “PII in prompts when hybrid cloud is on — concrete steps.” `(90–120s)`

Trap: “We encrypt HTTPS” only.
Senior answer: HTTPS necessary not sufficient. Redact account numbers, names; send category aggregates not raw SMS; user toggle; retention limits; audit logs; prefer local.
Follow-up: Keychain/biometric for local vault (FinTrack).

T6. “District used AI for tests — isn’t that the same as on-device AI?” `(90–120s)`

Trap: Tool-worship conflation.
Senior answer: Separate concerns. District: AI as **engineering accelerator** for tests (judgment on review). FinTrack/GymFlow: **product AI** with privacy/fallback architecture. Seniors don’t confuse Cursor/codegen with on-device RAG.
Follow-up: Day 26 leadership — context engineering without tool-worship.

7. Flashcards for today

Front	Back
On-device RAG	Local retrieve → prompt → generate · Trap: “fine-tune on user data” · Prod: FinTrack BM25
Fail-soft AI	Rules/TF-IDF/capability flags · Trap: blank error only · Prod: S15/S16
Quantization why	Fit RAM/ANE; INT8/INT4 · Trap: ignore quality · Prod: GymFlow MiniLM
Hybrid router inputs	Thermal, battery, tokens, capability · Trap: cloud always · Prod: doc HybridAIRouter
Privacy script	Local-first; minimize; consent cloud · Trap: HTTPS = privacy · Prod: FinTrack no sync
BM25 vs MiniLM	Lexical vs semantic · Trap: one-size · Prod: FinTrack vs GymFlow
Hallucinated money	DB is source of truth · Trap: trust tokens · Prod: FinTrack
KV-cache	Faster decode; RAM cost; drop on warning · Trap: ignore Jetsam · Prod: doc model manager
Token stream UI	Background + main append + cancel · Trap: sync main inference · Prod: —
AI tooling vs product AI	Accelerator ≠ on-device architecture · Trap: tool worship · Prod: District vs FinTrack
30L DAU caution	Kill switch, sample, budgets · Trap: run LLM on all sessions · Prod: BMS judgment
Empty retrieval	No invent; generic + CTA · Trap: hallucinate · Prod: coach empty state

8. Practice

- **Coding / SD:**
  1. Whiteboard (25–30 min) full HLD from on-device-llm-ai-engine.md using cheatsheet timing micro: Clarify 5 + HLD 10.
  2. Deep-dive verbally (15 min): FinTrack BM25 path **or** GymFlow TFLite path with fail-soft matrix.
  3. Optional: sketch Swift `actor` RAG + router pseudocode from the doc (don’t rewrite the whole file).
- **Complexity / agenda to say first:** For SD: “privacy → retrieve → infer → fallback → metrics.” For STAR S15/S16: 2–3 min timed each.

9. Timed drill

1. Pick **3 Normal + 2 Tricky** (include T1 or T4). Record.
2. Score vs answer-timing-guide.md — architecture answers use **3–5 min** budget once; others 45s/120s.
3. Deliver **S15 and S16** once each at 2:30; re-cut to 2:00.
4. Log gotchas: hallucination on money, tool-vs-product AI confusion, missing fail-soft.

Day 25 — Machine-Round Practice (3 Hours)

> *Week 4 · Phase: Implementation under timebox · Time budget today:* ***~3 hrs machine + 1 hr debrief*** *(cap ~4–5 hrs total)*

1. Outcome

By end of day you can explain aloud (no notes):

- How you **timebox** a 3-hour machine round: clarify → skeleton → vertical slice → tests → polish — **no perfectionism**.
- Deliver a **working vertical slice** for either brief **(A) paginated list + cache + tests** or **(B) SDUI component renderer**, with explicit cut lines for what you skip.
- Self-grade against the rubrics below (correctness, architecture, tests, communication, honesty about gaps).
- Narrate trade-offs the way you would with a proctor watching (same "say this first" discipline as DSA).

Rule of the day: Ship a demoable core. A polished unfinished cathedral scores worse than a tested happy path + listed TODOs.

2. Concept deep dive

2.1 Machine-round operating system (memorize)

Phase	Clock	Do	Don't
0. Clarify	0:00–0:15	Restate brief, ask API shape, offline?, UIKit/SwiftUI, test framework	Code immediately
1. Agenda	0:15–0:20	Say layers + milestone plan out loud	Silent architecture
2. Skeleton	0:20–0:45	Types, protocols, empty UI, fake repository	Pixel-perfect UI
3. Vertical slice	0:45–2:00	One happy path end-to-end	All edge cases first
4. Cache / SDUI depth	2:00–2:30	Second requirement (cache **or** 2nd component)	Refactor aesthetics
5. Tests	2:30–2:50	3–6 meaningful unit tests	100% coverage chase
6. Buffer	2:50–3:00	README of trade-offs + known gaps	New features

Proctor script at 0:15:

> *"I'll clarify for 10 more minutes max, then build a vertical slice: UI → ViewModel → repository with protocol — fake data first, then networking/cache. I'll leave time for tests. If truncated, I'll document cut lines."*

2.2 Brief A — Paginated list + cache + tests

Prompt (use as-is):

> *Build a mobile screen that loads a **paginated** remote list (cursor or page number). Show loading / empty / error. Support **pull-to-refresh** and next-page on scroll. Add a **cache** so revisiting shows last-good data quickly, then refresh. Include **unit tests** for pagination and cache policy.*

Suggested architecture:

```
ListView (SwiftUI or UIKit)
  → ListViewModel (state: items, page/cursor, isLoading, error)
    → ListRepository protocol
        → RemoteDataSource (URLSession)
        → CacheDataSource (memory + optional disk)
```

Must-have acceptance:

1. First page renders from network (or stub).
2. Next page appends without wiping.
3. Failure on page 2 keeps page 1 visible.
4. Cache: cold open can show stale then refresh (define policy aloud).
5. Tests: pagination reducer / repository with mock — **not** only UI snapshots.

Cache policy options (pick one, say why):

Policy	When	Cost
Memory LRU only	Fast interview slice	Lost on kill
Memory + disk (Codable)	Stronger "senior"	Serialization time
Stale-while-revalidate	Best UX story	Need version/TTL field

Cut lines (explicitly OK): fancy skeletons, DiffableDataSource animations, image pipeline, auth refresh, Perfect offline merge.

BMS hook (≤20s if asked why you know this): Search/listings at scale; debounce + state; Firebase p50/p90 mindset for list latency — don't overclaim machine-round code is production BMS.

2.3 Brief B — SDUI component renderer

Prompt (use as-is):

> *Build a **server-driven UI** renderer: given a JSON document of components (`type` , `props` , optional `children`), map to native views. Support at least **3 component types** (e.g. `text` , `image` , `button` or `vstack`). Unknown `type` → safe fallback view. Include a **version/schema** comment or simple `schemaVersion` check. Unit-test decoding + unknown-type fallback.*

Suggested architecture:

```
JSON Data
  → SDUIDocument (Codable, schemaVersion)
  → ComponentNode enum / protocol + factory
  → SDUIRenderer → AnyView / UIView
  → Feature flag / default fallback leaf
```

Must-have acceptance:

1. Decode sample JSON into model.
2. Render ≥3 types.
3. Unknown type does **not** crash — placeholder + analytics hook stub.
4. Nested children for one container type.
5. Tests: decoder + factory fallback.

Cut lines: full CMS tooling, live reload, expression language, actions beyond `print` / `callback` stub, pixel parity with Figma.

Production hook: BMS backend-driven header / Aces splash / Stories — sdui-engine.md; STAR story-bank.md#S3 · #S12.

2.4 Rubrics (score yourself 1–5 each)

Dimension	5	3	1
Correctness	Happy path + key failure solid	Happy path only	Doesn't run
Architecture	Clear layers + protocols	Mixed but navigable	Massive VC / God ViewModel
Caching or SDUI depth	Policy explained + coded	Partial	Missing second requirement
Tests	3+ meaningful, fast	1 weak test	None
Communication	Agenda + trade-offs spoken	Occasional notes	Silent grind
Time honesty	Cut lines documented	Ran over chasing polish	Perfectionism, unfinished core

Pass bar for Day 25: average **≥3.5**, correctness ≥4, tests ≥3.

2.5 Trade-offs

Choice	When	Cost
SwiftUI list	Faster slice	Hybrid apps may want UIKit — say assumption
UIKit + Diffable	Matches many codebases	More boilerplate in 3 hrs
Protocol + fake repo first	Tests & progress	Must swap to real URLSession later
Real network in round	Impressive if stable	Flaky Wi-Fi kills demo — prefer stub + one live call
Snapshot tests	UI confidence	Slow setup — skip unless already scaffolded
Perfect Clean Architecture	Rarely fits 3 hrs	Prefer pragmatic MVVM + protocols

2.6 Anti-perfectionism checklist (print mentally)

- [ ] Am I styling fonts while pagination is broken? → stop
- [ ] Can I demo in 60s right now? → if no, vertical slice
- [ ] Do I have ≥1 test? → if no after 2:30, write two now
- [ ] Did I narrate cache/SDUI policy? → if no, say it in buffer
- [ ] Am I rewriting names for beauty? → stop

3. Read these

Priority	Resource	Why
Must	coding/dsa-track.md — Machine round section	Any stubs / prior notes
Must	This day's rubrics + timebox	Operating system
Deepen	networking-layer.md	Pagination / interceptors ideas for A
Deepen	sdui-engine.md	Schema, fallbacks for B
Repo	search-autocomplete.md	Optional list UX inspiration — don't expand scope

4. Map to your work

Company / feature: BMS search & listings; SDUI header; District Clean/MVVM; Raw server-driven splash.
What you did: Debounced search MVVM; generalised header; production SDUI awareness; AI-assisted tests at District (**judgment** on generated tests — not blind accept).
Interview line (≤20s):

> *"In machine rounds I optimize for a tested vertical slice — same bias I use shipping SDUI and list UX: contracts, fallbacks, and observable state."*

→ STAR: story-bank.md#S3 · #S9 · #S12

5. Normal questions

(Proctor / debrief style — answer aloud.)

Q1. How do you approach a 3-hour take-home or onsite machine round? `(30–45s)`

Skeleton: Clarify → agenda → vertical slice → second requirement → tests → known gaps. Optimize demoability.
Follow-up: What do you cut first?
Story: —

Q2. How do you paginate without duplicates or lost pages? `(45s)`

Skeleton: Cursor vs page; append-only; in-flight guard; ignore stale responses (token/id).
Follow-up:
Story: Search lists.

Q3. What cache policy did you pick and why? `(30–45s)`

Skeleton: Name policy (SWR / TTL / memory-only); stale shown; refresh error keeps stale.
Follow-up: Invalidation on logout?
Story: —

Q4. SDUI unknown component — what should happen? `(30–45s)`

Skeleton: Fallback view; don't crash; log type; optional skip children; schema version gate.
Follow-up:
Story: S3 header.

Q5. How many tests are "enough" in 3 hours? `(30–45s)`

Skeleton: Happy path pagination or decode; one failure; one unknown type / cache hit — quality over count.
Follow-up: District AI tests judgment.
Story: S9.

6. Tricky questions

T1. "Your cache served stale event prices — defend the design." `(90–120s)`

Trap: "Cache is always good."
Senior answer: TTL + SWR; price-critical rows bypass or short TTL; pull-to-refresh; show last-updated; never cache across users. For machine round, state what you'd harden in prod at 30L DAU.
Follow-up: —

T2. "Why not generate the whole app with AI in the machine round?" `(90–120s)`

Trap: Tool-worship or total ban.
Senior answer: AI can scaffold boilerplate; you own architecture, edge cases, tests, and can explain every line. Same District lesson — accelerator, not author of record.
Follow-up: Day 26.

T3. “You didn’t finish image loading — is that a fail?” `(90–120s)`

Trap: Apologize spiraling.
Senior answer: Pre-declared cut line; core pagination/SDUI works; placeholders documented; production would use shared image pipeline (HeroWidget experience). Graders prefer honesty + working core.
Follow-up: —

T4. “Show me concurrency bugs in your pager.” `(90–120s)`

Trap: No cancellation story.
Senior answer: Task/id for request generation; cancel in-flight on refresh; main-actor state updates; don’t append out-of-order pages. Tie to synchronised-state lessons (S2) at a high level.
Follow-up: —

7. Flashcards for today

Front	Back
3-hr phases	Clarify 15 → slice → depth → tests → buffer · Trap: polish first · Prod: shipping bias
Vertical slice	One E2E happy path · Trap: all layers half-done · Prod: —
Pagination guard	In-flight + stale token · Trap: double append · Prod: search lists
Cache SWR	Show stale then refresh · Trap: empty until network · Prod: perceived perf
SDUI unknown type	Fallback not crash · Trap: force unwrap · Prod: S3/S12
Tests enough	3 meaningful · Trap: 0 or 40 flaky · Prod: District AI judgment
Cut line	Say what you skip · Trap: hide gaps · Prod: senior honesty
Protocol repo	Fake first then live · Trap: URLSession in VM · Prod: Clean/MVVM
Proctor agenda	Speak plan at 0:15 · Trap: silent · Prod: timing guide
Perfectionism flag	Styling while core broken · Trap: rename churn · Prod: —

8. Practice

- **Coding / SD:** Pick **one** brief — **A or B** (if time surplus, outline the other in 20 min notes only).

  Recommended:

  - If weaker on networking/state → **A**.
  - If weaker on SDUI → **B** (high ROI before Day 27).

- **Setup (before timer):** Xcode project or playground; sample JSON/API stub ready; timer visible.
- **Complexity / agenda to say first:** Record first 3 minutes of your clarifying + plan — treat as graded.
- **Debrief (60 min after):** Score rubric; write 5 bullets “keep / fix”; add gotchas; do **not** rebuild from scratch tonight.

9. Timed drill

1. Full **3:00:00** machine session — single brief.
2. Afterward: **3 Normal + 1 Tricky** from §§5–6 about *your* implementation (not generic).
3. Score machine rubrics + Qs vs answer-timing-guide.md.
4. Log misses; park improvements for after Mock #4 — **no all-night rewrite**.

Day 26 — Behavioral + Leadership (STAR Polish)

> *Week 4 · Phase: Behavioral / leadership judgment · Time budget today: ~4–5 hrs*

1. Outcome

By end of day you can explain aloud (no notes):

- Deliver any story from story-bank.md in **2–3 minutes** with clock discipline (20s S/T → 90s Action → 30s Result → 20s Lesson).
- Handle **conflict, incident, mentorship, and AI tooling judgment** without rambling or inventing metrics.
- Tell the **District Context Engineering** story as *architecture/process judgment* — **not tool-worship** (Cursor/Claude are accelerators).
- Map interviewer prompts → story IDs in ≤10s using the quick picker.
- Use only resume-backed numbers: **30L+ DAU**, **99.95% crash-free**, **30%+ nav reduction** (LE Bottom Sheet), plus qualitative SDK/migration outcomes.

2. Concept deep dive

2.1 STAR operating rhythm

Segment	Time	Do	Don't
Situation + Task	~20s	Scale + your ownership	Company history lecture
Action	~90s	3–5 concrete steps, one trade-off	"We" without your role
Result	~30s	Metric or crisp outcome	Fake numbers
Lesson	~20s	Principle you'd reuse	Humble-brag essay

Opener template:

> *"I'll take ~2 minutes on [X] — context, what I owned, outcome."*

2.2 Prompt → story routing (practice until instant)

Interviewer asks	Reach for	Backup
Hard technical project	S1 Ads/HeroWidget	S10 Stories SDK
Conflict / disagreement	S6 LE sheet contracts **or** S8 IMOC	S14 migration priorities
Incident / on-call / ownership	S8 IMOC + crash-free culture	S2 races
Mentorship / leadership sans title	S6, S10, S14	S1 stakeholder coord
Failure / mistake	Pinning/ops gap, race shipped, SDUI fallback miss — real	Don't invent hero failure
Why senior / impact	S6 **30%+ nav**; S8 **99.95%**; scale **30L+ DAU**	S10 portfolio reuse
AI / productivity tools	**S9 District** — context engineering	S15/S16 product AI (different!)
Privacy / on-device AI	S15 FinTrack	S16 GymFlow
Cross-platform / hybrid	S13 Grizzlies	S12 Aces
Security	S4 pinning	—

Performance culture	S5 Firebase p50/p90	S17-era Instruments language

2.3 Conflict script (structure, not drama)

1. Disagreement named specifically (API shape, scope, timeline, approach).
2. What you did: data, prototype, user impact, written options.
3. How you disagreed *with* people, not *at* them.
4. Resolution + relationship intact.
5. Lesson: prefer contracts and metrics over opinions.

LE Bottom Sheet angle (S6): Align PM/Design/Backend on API + UX; shipped lighter overview; **30%+ flows** with fewer full-screen navigations.

2.4 Incident script (S8)

1. Severity + user impact (crash/reliability).
2. Your role in triage (IMOC-style): stabilize → communicate → root cause → prevent.
3. Concrete technical action (not "worked hard").
4. Result tied to **99.95%+ crash-free** culture / reduction of class of failures.
5. Lesson: observability + ownership > blame.

2.5 Mentorship / leadership without title

Use **end-to-end ownership** stories: S1, S6, S10, S14.

Say:

> *"Leadership was setting the technical direction, unblocking others, and owning the rollout checklist — not waiting for a manager title."*

Concrete mentorship beats: pairing on architecture, writing the zero-regression checklist others used (S14), SDK boundaries that helped multiple apps (S10).

2.6 AI tooling judgment — District Context Engineering (no tool-worship)

Wrong answer: "I use Cursor/Claude for everything; AI writes my code; 10× engineer."
Senior answer:

1. **Problem:** migration / test generation needed leverage without surrendering design.
2. **Action:** You shaped **context** — modules, conventions, acceptance criteria, review bar — so AI output was usable.
3. **Judgment:** You rejected bad generations; tests still need human oracle; architecture stays human-owned.
4. **Separate from product AI:** FinTrack/GymFlow are *product* on-device systems (Day 24); District is *engineering process*.
5. **Lesson:** Tools amplify clear thinking; they don't replace ownership at **30L DAU** stakes.

Timed opener:

> *"At District I used AI as an accelerator for migration/tests — the skill was context engineering and review judgment, not prompting theater."*

→ story-bank.md#S9

2.7 Trade-offs

Choice	When	Cost
Data-backed disagreement	Cross-functional conflict	Takes prep time
Escalate early	Safety/security/revenue	Political capital
Absorb and ship	Low-risk preference fights	Resentment if chronic
AI-generated tests	Boilerplate coverage	False green; review cost

Hand-written critical path tests	Money, identity, crashes	Slower
Long STAR (>4 min)	Never in interview	Looks junior / unfocused

3. Read these

Priority	Resource	Why
Must	stories/story-bank.md	Full bank + quick picker
Must	timing/answer-timing-guide.md § Behavioral	2–3 min budgets
Deepen	Day 24 notes — product AI vs tooling AI	Avoid conflation in interviews
Repo	Resume bullets only for metrics	No invented stats

4. Map to your work

Company / feature: BMS (Ads, LE sheet, IMOC/crash-free, pinning); District (Clean/MVVM + AI-assisted engineering); Raw (SDK, hybrid, migrations); FinTrack/GymFlow (on-device AI).
What you did: Owned revenue-critical and reliability paths; drove cross-functional UX wins; migrated with checklists; used AI with judgment; shipped privacy-first AI products.
Interview line (≤20s):

> *"I lead through ownership — crash-free discipline at 30L+ DAU scale, measurable UX wins like the LE sheet, and clear judgment on where AI helps versus where architecture must stay human."*

→ Rotate: story-bank.md S1–S16 as needed.

5. Normal questions

Q1. Tell me about yourself / walk me through your experience. `(90–120s — still structured)`

Skeleton: Current focus → BMS scale highlights → District architecture → Raw SDK/apps → FinTrack/GymFlow AI → what you want next (senior iOS ownership).
Follow-up: Deep dive one stop.
Story: Collage, not one STAR.

Q2. Tell me about a conflict with PM/Design. `(2–3 min STAR)`

Skeleton: S6 — contracts, options, **30%+ nav** outcome.
Follow-up: What would you do differently?
Story: S6.

Q3. Describe an incident you owned. `(2–3 min)`

Skeleton: S8 IMOC path + crash-free mindset; or S2 race class.
Follow-up: Prevention.
Story: S8 / S2.

Q4. How have you mentored or led without authority? `(2–3 min)`

Skeleton: S10 SDK reuse / S14 checklist / S1 ads standards.
Follow-up: Difficult mentee?
Story: S10/S14.

Q5. How do you use AI in your workflow? `(45–90s)`

Skeleton: District S9 — context, review, accelerator; **not** autopilot; contrast product AI S15.
Follow-up: When do you forbid AI?

Story: S9.

Q6. What's your biggest impact metric? `(30–45s)`

Skeleton: Pick one: **30%+ nav reduction**, **99.95% crash-free**, **30L+ DAU** ownership context — explain briefly.
Follow-up:
Story: S6 / S8.

6. Tricky questions

T1. "Tell me about a failure." `(90–120s)`

Trap: Fake failure that's actually a win; or blame-only.
Senior answer: Real miss (e.g. under-specified pin rotation, SDUI unknown-type gap, race that escaped) → impact → what you changed in process → lasting prevention. Keep ego out; keep ownership in.
Follow-up: How do you know it won't recur?

T2. "Engineers on your team lean on Copilot and ship junk — what do you do?" `(90–120s)`

Trap: Ban all AI or shrug.
Senior answer: Set review bar; critical paths require human-designed tests; pair on architecture; measure escaped defects; teach context engineering (S9). Tools stay; accountability stays.
Follow-up: Personal use policy.

T3. "Why should we hire you as senior vs mid?" `(90–120s)`

Trap: Buzzwords only.
Senior answer: Scope (30L+ DAU), revenue module ownership, reliability culture, SDK-level thinking, cross-functional delivery with metrics, incident ownership, architecture migration judgment, privacy-aware AI.
Follow-up: Growth area? — honest (e.g. deeper ML math) + how you compensate with systems thinking.

T4. "District migration — did AI do the work?" `(90–120s)`

Trap: Yes/no extremism.
Senior answer: AI accelerated boilerplate/tests; you owned module boundaries, Clean/MVVM decisions, and acceptance. Context engineering = specifying constraints models don't know.
Follow-up: Example of rejected AI output.

T5. "Push back on a bad deadline." `(90–120s)`

Trap: Hero overtime narrative only.
Senior answer: Make risk visible (quality, crash-free, scope cut options); propose MVP cut; protect **99.95%** class outcomes; escalate with data.
Follow-up: S14 / S6 style negotiation.

7. Flashcards for today

Front	Back
STAR timing	20 / 90 / 30 / 20 · Trap: 5-min Action · Prod: all stories
Conflict router	S6 or S8 · Trap: vague drama · Prod: 30% nav / IMOC
Incident lesson	Stabilize → RCA → prevent · Trap: blame · Prod: 99.95% CFS
Mentorship signal	Checklist, SDK, standards · Trap: "I'm nice" · Prod: S10/S14
AI tooling	Context + review · Trap: tool-worship · Prod: S9
Product vs process AI	S15/S16 vs S9 · Trap: conflate · Prod: Day 24

Metrics allowed	30L DAU, 99.95%, 30% nav · Trap: invent · Prod: resume
Failure STAR	Real miss + process change · Trap: humblebrag fail · Prod: —
Senior why	Scope + ownership + judgment · Trap: years only · Prod: —
Quick picker	Tags → IDs in story bank · Trap: search mid-answer · Prod: —

8. Practice

- **Coding / SD:** None required. Optional: 15-min SDUI or networking **agenda only** if restless — do not deep-work code today.
- **Story bank reps:**
  1. Full pass: **S6, S8, S9, S1** (priority).
  2. Second pass: **S10, S14, S15, S2**.
  3. Cold prompts: friend/partner throws random questions from §5–6; you only answer with bank stories.
- **Complexity / agenda to say first:** For every STAR, speak the 10s opener before the story.

9. Timed drill

1. Record **5 STARs** (must include conflict, incident, AI tooling).
2. Score each vs answer-timing-guide.md behavioral rubric (target ≥4).
3. Re-record any story >3:15 — cut Action to 3 bullets.
4. Log gotchas: invented metrics, tool-worship, missing lesson line.
5. Tomorrow is Mock #4 — sleep; do not cram new tech.

Day 27 — Expert Mock #4 (Full Loop)

> *Week 4 · Phase: Expert mock · Time budget today: **~4.5–5.5 hrs** (3×45 min + breaks + debrief)*

1. Outcome

By end of day you can explain aloud (no notes):

- Execute a **full senior loop**: **Coding 45 + iOS deep dive 45 + System design 45** with calendar discipline.
- Open each segment with the correct **agenda** from answer-timing-guide.md.
- Self-/expert-score against the **scorecards** below; identify ≤3 fix-forward items for Day 28 (flashcards only — no new topics).
- Keep production proof ready: BMS **30L+ DAU**, **99.95% crash-free**, **30%+ nav** LE sheet; District S9; FinTrack/GymFlow if AI asked; Raw SDK/hybrid as needed.

Mock #4 is the dress rehearsal. Treat it like the real loop: no notes in-frame, timer visible, water, breaks.

2. Concept deep dive

2.1 Day schedule (stick to it)

Block	Duration	Mode
Warm-up	15 min	Flashcards weak only; one STAR opener (S6 or S8)
Coding	**45 min**	Expert or self-proctor
Break	10–15 min	Walk; no phone doomscroll
iOS deep dive	**45 min**	Conceptual + production
Break	10–15 min	

System design	**45 min**	Full cheatsheet timing
Debrief	45–60 min	Scorecards + gotchas; **stop**

2.2 Segment A — Coding (45 min)

Timing inside the 45:

Min	Action
0–3	Clarify + "say this first" (brute → optimize → complexity → edges)
3–35	Code + narrate; run mental/examples
35–42	Tests / edge cases aloud; fix bugs
42–45	Complexity recap + alternative

Problem sources: dsa-track.md Medium — prefer **mixed unknown pattern** (Day 23 skill). Trees / hash / heap / window all fair.

Scorecard — Coding

Criterion	5	3	1
Clarify + agenda	Explicit, timed	Partial	Jumped to code
Correctness	Passes cases	Minor bug	Wrong approach
Complexity	Stated accurately	Vague	Missing/wrong
Communication	Continuous narrate	Occasional	Silent
Edges / tests	Covered	One mention	None
Time use	Finished with buffer	Barely done	Incomplete core

Target: average ≥3.5; agenda ≥4.

2.3 Segment B — iOS deep dive (45 min)

Format: Interviewer drives 4–6 topics. You answer with **30–45s** definitions or **90–120s** deep dives. Sprinkle architecture **3–5 min** once if asked "design X."

Likely topic pools (Week 1–3 spine):

- Memory / ARC / retain cycles (S2 adjacent)
- Concurrency: GCD vs actors / races
- MVVM–Clean–DI (District)
- URLSession, pinning, refresh (S4)
- SDUI + fallbacks (S3)
- SwiftUI identity / hybrid UIKit (S13)
- Perf: Instruments, p50/p90 (S5)
- Crash / IMOC / 99.95% (S8)
- Modularization / SPM / Stories SDK (S10)
- On-device AI privacy/fail-soft if they pivot (S15/S16) — keep tight

Scorecard — iOS deep dive

Criterion	5	3	1
Timing discipline	Hits budgets	±30%	Ramble / blank

Mechanism accuracy	Correct	Fuzzy	Wrong
Trade-off	Present	Weak	None
Production proof	BMS/Raw/District	Generic “in apps”	None
Agenda on long answers	Yes	Sometimes	Never
Honesty	Clear assumptions	Hedge filler	Bluff

Target: ≥4 on Normals; ≥3 on Trickies; at least **two** production hooks.

2.4 Segment C — System design (45 min)

Use cheatsheet clock from cheatsheet.md / timing guide:

```
0–5   Clarify: scope, DAU, offline?, iOS-only?, latency SLO
5–15  HLD: 4-layer client + data flow
15–25 Data/API: entities, pagination, payloads, idempotency
25–40 Deep dive: 2–3 hardest subsystems
40–45 Ops: failure modes, metrics, rollout, kill switch
```

Pick one primary prompt (expert chooses; otherwise rotate):

Option	Doc	Why
SDUI engine	sdui-engine.md	Your strongest BMS/Raw hook
Networking + security	networking-layer.md + pinning	S4 proof
On-device AI assistant	on-device-llm-ai-engine.md	Differentiator FinTrack/GymFlow
Image pipeline / feed	image-loading-library.md / social-feed.md	Perf depth

Staff opener:

> *“I’ll spend 5 minutes on scope, then architecture, then deep-dive retrieval/fallback and ops. Does that work?”*

Scorecard — System design

Criterion	5	3	1
Clarify quality	Constraints + SLOs	Few Qs	Assumed all
HLD clarity	4 layers drawn	Blob diagram	No structure
Deep dive	2 subsystems crisp	One shallow	Skipped
Failure modes	Kill switch, degrade	Mention only	Happy path only
Metrics / rollout	Concrete	Vague	None
Timing	On cheatsheet	Drifted 10+ min	Chaos
Personal proof	Tied to shipped work	Light	Generic blog

Target: ≥3.5 overall; clarify + failure modes ≥4.

2.5 Trade-offs (meta — how you run the mock)

Choice	When	Cost

Expert human proctor	Best signal	Scheduling
Self + recorded	Always available	Blind spots
Peer iOS eng	Good pushback	May go soft
New problem never seen	Realism	Higher stress — intended
Reusing Day 25 machine code	Comfort	Weaker coding signal — prefer fresh DSA

3. Read these

Priority	Resource	Why
Must	timing/answer-timing-guide.md	All segment budgets
Must	cheatsheet.md	SD clock
Must	coding/dsa-track.md	Coding problem pick
Deepen	Primary SD doc for your chosen prompt	Refresh diagrams only — no new reading rabbit holes
Repo	stories/story-bank.md	One warmup STAR

4. Map to your work

Company / feature: Full career spine — BMS scale/reliability/SDUI/security; District architecture + AI judgment; Raw modularity/hybrid; FinTrack/GymFlow on-device AI.
What you did: Own the narrative: senior = trade-offs + production proof + calm timing.
Interview line (≤20s) before mock:

> *"I'll run this like a real loop — agenda first, trade-offs, and metrics I actually shipped."*

→ Stories on demand from story-bank.md.

5. Normal questions

(Warmup / between segments — not instead of the mock.)

Q1. How will you open the coding round? `(30–45s)`

Skeleton: Restate → clarify → brute → optimize → complexity → edges → code.
Follow-up:
Story: —

Q2. How will you open system design? `(30–45s)`

Skeleton: Staff agenda + 5 min clarify; ask DAU/offline/SLO.
Follow-up:
Story: —

Q3. Name three production proofs you'll keep loaded. `(30–45s)`

Skeleton: 30L+ DAU context; 99.95% crash-free; 30%+ nav LE sheet — plus one of SDUI/pinning/SDK/AI.
Follow-up:
Story: S6/S8/S3/S4/S10/S15.

Q4. What do you do if you blank? `(30–45s)`

Skeleton: Say assumption; start from requirements; draw; offer brute; recover. No apology spiral.
Follow-up:

Story: —

Q5. What's out of scope for today's debrief? `(30s)`

Skeleton: Learning brand-new frameworks tonight; rewriting Day 25 app; new LeetCode topics.
Follow-up:
Story: —

6. Tricky questions

T1. "Your SD deep dive is running long — what do you cut?" `(90s)`

Trap: Panic-skip ops.
Senior answer: Park third subsystem; always reserve **ops/failure modes** 5 min — seniors protect that. Summarize remaining.
Follow-up: —

T2. "Coding solution is O(n²) and time is almost up." `(90s)`

Trap: Silent rewrite from scratch.
Senior answer: State optimal approach + complexity; sketch key function; note what you'd change; partial credit via communication.
Follow-up: —

T3. "Interviewer challenges your BMS metric." `(90s)`

Trap: Inflate or collapse.
Senior answer: Clarify what the number measures (crash-free sessions; nav reduction on targeted flows); don't generalize beyond resume. Offer related proof.
Follow-up: —

T4. "They ask on-device AI but you prepared SDUI." `(90s)`

Trap: Force SDUI anyway.
Senior answer: Pivot gracefully — privacy, RAG, fail-soft, FinTrack/GymFlow; reuse 4-layer client; shorter HLD OK if clarify strong.
Follow-up: Day 24 matrix.

7. Flashcards for today

Front	Back
Coding first 3 min	Clarify brute optimize complex edges · Trap: silent code · Prod: —
SD 45 clock	5–10–10–15–5 · Trap: deep dive forever · Prod: cheatsheet
Deep dive budget	45s / 120s / 5 min arch · Trap: lecture · Prod: timing guide
Proof trio	30L DAU / 99.95% / 30% nav · Trap: invent · Prod: resume
Protect ops	Always last 5 SD · Trap: cut metrics · Prod: kill switch
Blank recovery	Assumption + draw + brute · Trap: apology loop · Prod: —
Mock breaks	10–15 min real · Trap: cram mid-loop · Prod: —
Debrief cap	≤3 fix-forwards · Trap: rewrite life · Prod: Day 28
AI pivot	Fail-soft + privacy · Trap: tool worship · Prod: S15/S9
Score target	≥3.5 segments · Trap: ignore timing · Prod: —

8. Practice

- **Coding / SD:** The mock *is* the practice. No extra LeetCode volume after.
- **Complexity / agenda to say first:** Before each segment, 30s quiet rehearsal of the opener only.
- **Materials to have open for proctor (not you mid-answer):** scorecards printed; problem statement; SD prompt; timer.

Expert briefing (paste to interviewer):

> *Muhammed is targeting senior iOS. Please run Coding 45 (Medium, unlabeled pattern), iOS deep dive 45 (concurrency, architecture, networking/SDUI, perf/crashes — push for trade-offs), System design 45 (SDUI **or** networking+pinning **or** on-device AI). Score communication and timing, not only correctness. Metrics he may use: 30L+ DAU, 99.95% crash-free, 30%+ nav reduction.*

9. Timed drill

1. Run the **full three segments** with scorecards.
2. Debrief: write scores; list **exactly three** weak flashcard fronts for Day 28.
3. One STAR only if an iOS answer needed a story and failed timing — else rest.
4. **Hard stop.** Sleep hygiene starts tonight for Day 28 → interview readiness.

Day 28 — Game Day (No New Topics)

> *Week 4 · Phase: Taper / interview readiness · Time budget today: **~2–3 hrs max** then rest*

1. Outcome

By end of day you can explain aloud (no notes):

- Your **weak-area flashcards only** (from Mock #4's ≤3 fix-forwards + any chronic gotchas) — not the entire deck.
- Each priority story in story-bank.md **once** at 2–3 min — then stop.
- A calm **pre-interview checklist** (logistics, environment, agenda openers, metric trio).
- **Zero new topics:** no new LeetCode patterns, no new WWDC videos, no new system-design docs, no machine-round rewrites.

North star today: Protect sleep and confidence. Fresh recall beats last-minute cram.

2. Concept deep dive

2.1 Taper rules (non-negotiable)

Allowed	Forbidden
Weak flashcards (≤30–40 cards)	Entire Anki deck binge
Story bank **once each** (priority set)	Rewriting STARs from scratch
Skim timing guide budgets	New SD HLD from scratch
Checklist + bag/laptop setup	Day 25 app rewrite
Light walk / meal prep	"Just one more Medium"
Early sleep	Caffeine late / all-nighter

If anxiety says "study more," do **one** 45s definition aloud, then close the laptop.

2.2 Weak-area flashcard protocol

1. Open gotchas log + Mock #4 three fix-forwards.
2. Pull **only** matching cards (examples: BST validate, fail-soft AI, SD ops last 5, heap direction, STAR timing).
3. Round 1: read front → say back aloud → check.
4. Round 2: misses only.

5. Cap **45–60 minutes**. Box closed.

2.3 Story bank — once each (priority set)

Run in order; timer 2:30; stop at end — **no third takes** unless a story completely derailed (<1 re-take max).

Order	ID	Why on game day
1	S6 LE Bottom Sheet	Impact metric **30%+ nav**
2	S8 IMOC / crash	**99.95%** reliability ownership
3	S9 District AI tooling	Judgment without tool-worship
4	S1 Ads / HeroWidget	Hard technical + POP
5	S3 or S4	SDUI **or** pinning (pick based on target company)
6	S10 or S15	SDK modularity **or** on-device AI

Optional micro (30–45s only): “Tell me about yourself” spine — BMS scale → District → Raw → FinTrack/GymFlow.

Metric trio (say cleanly once):

> *“I’ve owned features on BookMyShow at **30L+ DAU**, helped drive **99.95%+ crash-free** discipline on my paths, and shipped UX like the LE bottom sheet that cut full-screen navigations by **30%+** on targeted flows.”*

2.4 Agenda openers (read once, don’t drill forever)

Coding: Clarify → brute → optimize → complexity → edges → code.
Deep dive: Definition → mechanism → trade-off → production.
System design: “5 min scope, then HLD, deep dives, ops — sound good?”
Behavioral: “I’ll take ~2 minutes — context, what I owned, outcome.”

2.5 Trade-offs

Choice	When	Cost
Taper	Day before / morning of	FOMO
Cram new topic	Never on Day 28	Confusion + poor sleep
Full mock today	Only if Mock #4 missed — prefer rest	Fatigue
Light flashcards	Anxiety channel	Over-long session if uncapped

3. Read these

Priority	Resource	Why
Must	Gotchas + Mock #4 notes	Weak cards only
Must	stories/story-bank.md	One pass priority IDs
Must	timing/answer-timing-guide.md	Skim budgets table only (5 min)
Deepen	—	**None**
Repo	This checklist §8–9	Logistics

4. Map to your work

Company / feature: You already mapped four weeks of work to interview language. Today you only **retrieve**, not invent.
What you did: BMS / District / Raw / FinTrack / GymFlow — enough proof.
Interview line (≤20s):

> *"I'm ready to talk ownership, trade-offs, and systems I've shipped — I'll keep answers timed and concrete."*

→ story-bank.md priority set above.

5. Normal questions

(Light only — max 5 total aloud today.)

Q1. What is your metric trio? `(30–45s)`

Skeleton: 30L+ DAU · 99.95% crash-free · 30%+ nav reduction — with honest scope wording.
Follow-up: —
Story: S6/S8.

Q2. Coding opener? `(30s)`

Skeleton: Say-this-first script in one breath.
Follow-up: —
Story: —

Q3. SD opener? `(30s)`

Skeleton: Staff agenda + clarify first.
Follow-up: —
Story: —

Q4. How do you talk about AI tools? `(45s)`

Skeleton: S9 — accelerator + review; not author of record; product AI separate (S15).
Follow-up: —
Story: S9.

Q5. What will you not do tonight? `(20s)`

Skeleton: No new topics; sleep.
Follow-up: —
Story: —

6. Tricky questions

(At most one — skip if tired.)

T1. "Anxiety: I didn't cover X." `(60–90s to yourself)`

Trap: Open a new doc.
Senior answer: Name X; if it was on Mock #4 fix list, flashcard it; else park. In interview: clarify, state assumption, relate to nearest shipped system.
Follow-up: Close laptop.

7. Flashcards for today

Do not learn new cards. Review **only** weak set. Template if you must jot one gotcha:

Front	Back
(your Mock #4 miss 1)	Short fix · Trap: … · Prod: …

(your Mock #4 miss 2)	…
(your Mock #4 miss 3)	…
Metric trio	30L+ DAU / 99.95% CFS / 30%+ nav · Trap: invent · Prod: resume
No new topics	Flash + stories + sleep · Trap: one more LC · Prod: —
Fail-soft one-liner	Degrade usefully; don't crash · Trap: blank error · Prod: S15/S16
SD protect ops	Last 5 min failure/metrics · Trap: skip · Prod: —
STAR cut	3 Action bullets if long · Trap: 5 min story · Prod: —

8. Practice

- **Coding / SD: None.** No problems, no whiteboarding new systems.
- **Complexity / agenda to say first:** Speak the four openers **once** (§2.4).
- **Stories:** Priority set **once each** (§2.3).
- **Flashcards:** Weak-only, ≤60 min.
- **Logistics checklist** (complete today):

Pre-interview checklist

Environment

- [ ] Quiet room booked / headphones tested
- [ ] Camera angle, lighting, neutral background
- [ ] Charger plugged; notifications DND
- [ ] Water; notepad + pen (if allowed)
- [ ] Backup network (phone hotspot) known

Materials

- [ ] Calendar link / laptop login works
- [ ] IDE ready only if machine round — else closed
- [ ] Resume PDF synced with story metrics
- [ ] Timing budgets mental (or tiny sticky: 45s / 2m / 5m / 45m SD)

Content (retrieval only)

- [ ] Metric trio once
- [ ] S6, S8, S9 once
- [ ] Coding + SD openers once
- [ ] Weak flashcards capped

Body

- [ ] Real meal; light movement
- [ ] **Sleep target:** 7–8 hours; alarm set
- [ ] No new coffee experiment

9. Timed drill

1. **45–60 min** weak flashcards only.
2. **40–50 min** story bank priority once each (timer on).
3. **15 min** checklist + environment dry run (join dummy Meet/Zoom).
4. **5 min** skim answer-timing-guide.md quick budgets table.
5. **Stop.** Log "Day 28 complete — tapered." No gotchas expansion unless one flashcard miss — single line max.

You've finished the 4-week system. Trust the reps: **agenda · trade-off · proof · time.**

Week 1 Flashcards — Swift, Memory, Concurrency, DSA Warm-up

Candidate: **Muhammed Shahid** · BMS / District / Raw
Metrics only from resume: **30L+ DAU**, **99.95%+ CFS**, **30%+ fewer full-screen navs** (LE sheet).

Source days: `weeks/week-01/day-01/README.md` … `day-07`.
Practice with answer-timing-guide.md.

≈71 cards · Tags: `swift` · `memory` · `concurrency` · `dsa` · `mock`

Swift — value / POP / generics (`swift`) · Days 01–02

Front	Back
struct vs class	Value vs reference · Trap: class for all models · Prod: BMS ad DTOs as structs
COW	Share buffer until write · Trap: assume A sees B's mutation · Prod: listing arrays
actor (1-liner)	Isolated reference type · Trap: use for all UI · Prod: modernize sync dicts
enum state machine	Exhaustive states · Trap: boolean flags · Prod: payment popup
===	Referential identity · Trap: use for structs · Prod: VC identity
indirect enum	Heap box for recursion · Trap: forget indirect · Prod: nested domain trees
Prefer values when	No identity needed · Trap: premature class · Prod: Clean models
Nested class in struct copy	Reference shared · Trap: think deep copy · Prod: mixed graphs
`@MainActor` vs actor	UI affinity vs general isolation · Trap: conflate · Prod: VM on main
LoadState<T>	idle/loading/loaded/failed · Trap: isLoading+optional · Prod: search MVVM
POP	Compose protocols · Trap: deep inheritance · Prod: Ads pipeline
associatedtype	Conformer picks type · Trap: use as easy existential · Prod: AdRenderable
some vs any	Opaque vs existential · Trap: synonym · Prod: View returns
Type erasure	Box PAT · Trap: free · Prod: heterogeneous ads list
Protocol extension dispatch	Defaults may be static · Trap: expect override · Prod: shared track()
Generics benefit	Specialize + safety · Trap: Any everywhere · Prod: HeroWidget
AnyObject protocol	Class-bound · Trap: struct conform · Prod: weak delegate
Conditional conformance	where on extension · Trap: always free · Prod: model arrays
Composition A & B	Multi-capability · Trap: multiple inheritance myth · Prod: Render+Track
Open vs closed SDUI	Registry vs enum · Trap: enum forever · Prod: BMS header
Witness table	Dynamic protocol dispatch · Trap: always slow · Prod: cell bind
SDK public API	Protocols at boundary · Trap: expose concretes · Prod: Stories SDK

Memory / ARC (`memory`) · Day 03

Front	Back
ARC	Retain count via compiler · Trap: GC pauses · Prod: BMS reliability
weak	Optional zeroing · Trap: overuse unowned · Prod: network callbacks
unowned	Non-optional · Trap: outlive owner · Prod: nested owned child
Closure cycle	self↔closure · Trap: forget capture list · Prod: ad completion
Leaks vs Graph	Unreachable vs cycles · Trap: same tool · Prod: triage
Timer cycle	Target retains · Trap: never invalidate · Prod: live scoreboard ticks
Task retain	Task holds locals · Trap: fire-and-forget · Prod: search debounce
Abandoned memory	Still referenced · Trap: call it leak · Prod: nav stack caches
deinit missing	Cycle checklist · Trap: blame ARC bug · Prod: Crashlytics + Graph
Autorelease	Drain temporaries · Trap: always needed · Prod: image loops
99.95% CFS	Reliability bar · Trap: invent metrics · Prod: S8
Delegate weak	Break VC↔delegate · Trap: strong delegate · Prod: UIKit

GCD (`concurrency`) · Day 04

Front	Back
Serial queue	Mutex via queue · Trap: sync re-entry · Prod: S2 dicts
Concurrent + barrier	RW pattern · Trap: write without barrier · Prod: read-heavy caches
sync deadlock	Wait on current queue · Trap: only main · Prod: UI hops use async
DispatchGroup	Join parallel work · Trap: forget leave · Prod: prefetch
QoS	Priority/energy · Trap: always userInteractive · Prod: analytics batch
Race	Unsynchronized share · Trap: intermittent ignore · Prod: BMS crashes
Safe dict API	Hide storage · Trap: return mutable ref · Prod: S2
Semaphore	Limit concurrency · Trap: deadlock wait · Prefer TaskGroup later
Main UI rule	UI on main · Trap: parse JSON on main · Prod: search bind
async write / sync read	Common serial pattern · Trap: return unsafely · Prod: S2
Reader-writer	Parallel reads · Trap: writer starve · Prod: optional upgrade
Actor vs GCD	Language isolation · Trap: rewrite all now · Prod: new code actors

Swift Concurrency (`concurrency`) · Day 05

Front	Back
Structured concurrency	Parent owns children · Trap: detached everywhere · Prod: search tasks
Actor	Isolated mutable state · Trap: no reentrancy · Prod: S2 future
Reentrancy	State may change after await · Trap: assume continuity · Prod: token refresh

Sendable	Cross-domain safe · Trap: @unchecked casually · Prod: Swift 6
@MainActor	UI isolation · Trap: heavy work on main · Prod: VM
Task cancel	Cooperative · Trap: kill thread · Prod: debounce
async let	Fixed parallel · Trap: unbounded fanout · Prod: dual fetch
TaskGroup	Dynamic parallel · Trap: forget await all · Prod: prefetch
async vs GCD	Await + structure · Trap: throw GCD away blindly · Prod: mixed codebase
Detached	Independent lifetime · Trap: default choice · Prod: rare
Hop to main	await MainActor · Trap: sync main · Prod: bind UI
Migration pitch	Queue dict → actor · Trap: big-bang rewrite · Prod: S2

DSA warm-up (`dsa`) · Day 06

Front	Back
Two pointers opposite	Sorted pair / palindrome · O(n)
Sliding window variable	Longest with constraint · expand/shrink
Kadane	Max subarray · running reset
Prefix sum	Range sum O(1) after O(n)
Frequency map	Anagram / counts
Say first	Clarify→brute→opt→edges
Swift String index	Not random O(1) · use Array
Write pointer	In-place filter/dedup
Two Sum map	value→index · O(n)
Container water	Ends inward · O(n)

Mock meta (`mock`) · Day 07

Pin weak cards from Days 01–06; full mock agenda lives in `weeks/week-01/day-07/README.md.md`. Keep only:

Front	Back
Mock agenda opener	“Defs → concurrency deep dive → story → feed HLD”
Score 5 means	On time + trade-off + prod proof
SD clarify first	DAU, offline, pagination

Drill tips

1. Cover Back; speak Front answer in **30–45s** with one BMS/District/Raw hook when relevant.
2. Pin weak cards into Week 2 daily warm-up.
3. Full list for Anki: anki-import.csv.

Week 2 Flashcards — Architecture, Networking, SDUI, UIKit, SwiftUI

Candidate: **Muhammed Shahid** · BMS / District / Raw
Resume metrics only: **30L+ DAU**, **99.95%+ CFS**, **30%+ fewer full-screen navs**.

Source days: `weeks/week-02/day-08/README.md` … `day-14`.

≈90 cards · Tags: `architecture` · `networking` · `sdui` · `uikit` · `swiftui` · `dsa` · `mock`

Architecture / MVVM / Clean (`architecture`) · Day 08

Front	Back
MVVM one-liner	View renders state; VM presentation + async; Model domain · Trap: networking in View · Prod: BMS search VM
Clean one-liner	UseCases/entities independent of UI · Trap: Clean everywhere · Prod: District Free Parking rules
MVI one-liner	Intent → reduce → State → View · Trap: boilerplate for CRUD · Prod: payment/live state cousins
Where debounce lives	VM/UseCase presentation policy · Trap: only in URLSession · Prod: BMS search
DI default	Protocol + constructor injection · Trap: service locator for domain · Prod: testable VMs
Repository duty	Cache/remote + DTO map · Trap: business rules in repo · Prod: search repository
Fat VM smell	Networking + routing + rules · Fix: UseCase + Router · Prod: S9 extract
AI in migration	Accelerate inside envelope + review · Trap: “AI wrote it” · Prod: S9 Context Engineering
Navigation ownership	Events from VM; Coordinator for cross-feature · Trap: UIKit in UseCase · Prod: S13 deeplinks
Fail-soft UI states	loading/empty/error/success · Trap: spinner forever · Prod: BMS search
When not Clean	Trivial UI, no shared domain · Trap: resume padding · Prod: judgment call
Test pyramid tip	UseCase unit tests cheapest · Trap: only UITests · Prod: District XCTest
Strangler migration	Incremental extract, flags · Trap: rewrite · Prod: District
Protocol boundary	Network/Search/Analytics · Trap: protocol every struct · Prod: S1/S10
Scale link	Clear layers reduce blast radius · Trap: architecture = crash-free alone · Prod: 30L DAU + S8

Networking (`networking`) · Day 09

Front	Back
Networking SDK core	Endpoint → build → intercept → session → decode · Trap: UI in client · Prod: Ads URLSession
Single-flight refresh	One refresh Task; waiters share · Trap: N refreshes · Prod: auth storms
Cancel search	Cancel Task on new query · Trap: show cancel as error · Prod: S3
Retry safe?	Idempotent GET + backoff · Trap: POST payments · Prod: S7
URLCache vs app cache	Headers vs domain/offline · Trap: cache private unkeyed · Prod: SDUI Day 10
SSL pinning	SPKI pin; MITM ↓ · Trap: no rotation = outage · Prod: S4

Domain whitelist	Only approved hosts · Trap: CMS open fetch · Prod: S4
Why leave Alamofire	Control + deps + pinning · Trap: hate libraries · Prod: S4 Ads
NetworkError kinds	Transport/HTTP/decode/cancel/auth · Trap: raw strings to UI · Prod: S3 states
Interceptor order	Auth → log/trace → send · Trap: log tokens · Prod: hygiene
Test strategy	Fake session + fixtures · Trap: live network CI · Prod: migration parity
Idempotency key	Safe payment retries · Trap: client-only duplicate POST · Prod: checkout
p90 networking	Trace spans; optimize tail · Trap: mean latency · Prod: S5
Break-glass pin	Monitored failover plan · Trap: silent disable forever · Prod: S4 lesson
Protocol NetworkSession	Inject URLSession mock · Trap: concrete everywhere · Prod: testability

SDUI (`sdui`) · Day 10

Front	Back
SDUI definition	Schema → native registry render · Trap: JS eval · Prod: BMS header
Unknown component	Skip + metric · Trap: crash · Prod: 99.95% mindset
Schema major bump	Dual-publish + fallback · Trap: force upgrade only · Prod: contracts
FallbackEngine	Last-known-good disk · Trap: blank on offline · Prod: S12 splash
Action allowlist	No arbitrary code · Trap: open any URL · Prod: security
Hybrid SDUI	CMS slots + native chrome · Trap: 100% dynamic · Prod: most apps
Capability matrix	Client declares support · Trap: assume parity · Prod: iOS/Android
Cold start splash	Cache + timeout default · Trap: block on network · Prod: S12
BMS header win	Iterate without release · Trap: new types need app · Prod: S3
SDUI vs Ads native	Config dynamic; media native · Trap: SDUI video lifecycle · Prod: S1
Cache keying	Per user for personalised · Trap: shared cache leak · Prod: privacy
Analytics in SDUI	Validate server events · Trap: trust PII · Prod: hygiene
Canary schema	% rollout payloads · Trap: 100% push · Prod: S8 culture
Empty root policy	Hard fallback required · Trap: skip all → blank · Prod: splash rules
Contract tests	Fixtures per version · Trap: only manual QA · Prod: CI

UIKit / hybrid (`uikit`) · Day 11

Front	Back
Pause video where	willDisappear / offscreen · Trap: only deinit · Prod: S1 HeroWidget
prepareForReuse	Reset async + UI · Trap: stale images · Prod: lists
Prefetch cancel	cancelPrefetching… · Trap: unbounded downloads · Prod: data budget
SwiftUI in UIKit	UIHostingController · Trap: sizing/safeArea · Prod: S13

UIKit in SwiftUI	Representable + Coordinator · Trap: recreate storms · Prod: S13
Deeplink owner	App-edge router → intent · Trap: dual stacks · Prod: S13
Bottom sheet win	Context kept; fewer pushes · Trap: deep flows in sheet · Prod: S6 30%+
Diffable benefit	Identity diffs · Trap: unstable IDs · Prod: fewer reload bugs
Appear vs load	Refresh vs once · Trap: network only in didLoad · Prod: tabs
Cell retain cycle	weak self + clear handlers · Trap: strong VC in cell · Prod: leaks
Airship tap path	Same deeplink router · Trap: parallel nav · Prod: S13
Hybrid cost	Lifecycle + identity · Trap: bolt hosting · Prod: S13 lesson
Estimated size	Close to real · Trap: 1pt estimates · Prod: jank
LE metric	30%+ fewer full-screen navs · Trap: vanity UI · Prod: resume
Visibility ads	Threshold policy · Trap: play always · Prod: S1

SwiftUI (`swiftui`) · Day 12

Front	Back
`@State` use	Local ephemeral UI · Trap: async in every view · Prod: chrome
`@Observable` use	Feature/async model · Trap: god AppModel · Prod: Stories player
Identity rule	Stable IDs preserve state · Trap: UUID in body · Prod: S10 pages
ForEach ID	Identifiable stable keys · Trap: indices on reorder · Prod: lists
Lazy vs VStack	Lazy for large · Trap: eager thousands · Prod: perf
body rule	Cheap; no side effects · Trap: fetch in body · Prod: jank
Environment DI	Theme ok; services careful · Trap: hidden NetworkClient · Prod: S10
Stories API	DataSource + events + inject loaders · Trap: hardcode host net · Prod: S10
Pause stories	Scene phase / disappear · Trap: run in background · Prod: S10/S1
Invalidation storm	Split models · Trap: observe all · Prod: redraws
Representable reset	Parent id churn · Trap: SwiftUI bug · Prod: S13
Animation scope	Transaction local · Trap: animate whole tree · Prod: lists
Portfolio reuse	Protocols + theming hooks · Trap: fork per app · Prod: S10
SDUI leaf id	Server-stable id · Trap: array index · Prod: S3
Test SwiftUI	Model unit tests first · Trap: only UITest · Prod: S9

DSA stack/queue/LL (`dsa`) · Day 13

Front	Back
Stack use	LIFO matching/undo · Trap: use for BFS · Prod: nav nested
Queue use	FIFO BFS / waiters · Trap: Array.removeFirst hot · Prod: refresh waiters

Monotonic stack	Next greater O(n) · Trap: O(n²) scan · Prod: —
Two-stack queue	Amortized O(1) · Trap: claim strict O(1) always · Prod: —
Floyd cycle	Slow/fast O(1) space · Trap: only hash set · Prod: —
Reverse list	Iterative 3 pointers · Trap: lose next · Prod: —
Merge lists	Dummy head · Trap: null edges · Prod: —
Array as queue cost	removeFirst O(n) · Trap: ignore · Prod: Swift
Coding agenda	Restate→edges→complexity→code · Trap: code immediately · Prod: interviews
Deque window max	Maintain mono deque · Trap: heap each slide only · Prod: —
Min stack	Aux mins · Trap: O(n) scan · Prod: —
LL in Swift apps	Rare vs Array · Trap: force LL UI · Prod: honesty
k-group reverse	Count-reverse-reconnect · Trap: leftover mishandle · Prod: —
Ring buffer	Head/tail mod · Trap: empty/full ambiguity · Prod: audio-ish
Week 2 catch-up	Drill weak day Qs · Trap: only DSA forever · Prod: Mock #2

Mock meta (`mock`) · Day 14

Front	Back
Week 2 one-liner	Layers→network→SDUI→UI identity · Trap: isolated facts · Prod: narrative
Ads 5-min agenda	POP·lifecycle·URLSession pin · Trap: no agenda · Prod: S1/S4
SDUI 5-min agenda	Schema·registry·fallback·prod · Trap: skip fail-soft · Prod: S3/S12
Refresh stampede	Single-flight · Trap: N refreshes · Prod: Day 09
Pin outage	Rotation/runbook · Trap: pinning only · Prod: S4
Unknown SDUI node	Skip+metric · Trap: crash · Prod: crash-free
Dual nav stacks	One owner · Trap: sync forever · Prod: S13
UUID in body	Resets state · Trap: SwiftUI bug · Prod: Day 12
Sheet metric	30%+ fewer pushes · Trap: vague UX · Prod: S6
AI seniority	Envelope+review · Trap: AI wrote app · Prod: S9
Search cancel	Task cancel silent · Trap: error toast · Prod: S3
HeroWidget	Pause on invisible · Trap: play forever · Prod: S1
Stories SDK	Public API+inject · Trap: host coupled · Prod: S10
Queue pitfall	removeFirst O(n) · Trap: ignore · Prod: Day 13
Mock discipline	Timer + retro gotchas · Trap: skip retro · Prod: growth

---

Anki sample: anki-import.csv · DSA: ../coding/dsa-track.md

Week 3 Flashcards — Modularization, Images, Perf, Crashes, Security, Deeplinks/CI

Candidate: **Muhammed Shahid** · BMS / District / Raw
Resume metrics only: **30L+ DAU**, **99.95%+ CFS**, **30%+ fewer full-screen navs**.

Source days: `weeks/week-03/day-15/README.md` … `day-21` .

≈108 cards · Tags: `architecture` · `images` · `perf` · `crashes` · `security` · `deeplinks` · `ci` · `mock`

Modularization (`architecture`) · Day 15

Front	Back
Interface vs Impl module	Interface = protocols/DTOs; Impl = UI+logic. Cross-feature deps only on Interface · Trap: Impl→Impl · Prod: Stories public API
Composition root	App wires DI; features don't resolve globals · Trap: Swinject everywhere · Prod: host injects Stories providers
Why SPM	Native, parallel, no Ruby · Trap: ignore legacy Pod constraints · Prod: new modules SPM-first
Circular dependency fix	Depend on Interface; App registers builders · Trap: import Impl "just once" · Prod: portfolio SDK boundary
Static vs dynamic	Prefer static internal; dynamic for app↔extension share · Trap: 40 dynamic frameworks · Prod: launch dyld cost
Needle-style DI	Typed dependency protocols + component tree · Trap: runtime locator · Prod: CheckoutDependency pattern
Stories SDK lesson	Reuse = API + isolation + versioning · Trap: copy views per app · Prod: S10 portfolio adoption
Core junk drawer	Shared kernel only for true shared types · Trap: everything in Core · Prod: feature DTOs stay local
Forbidden import	FeatureAImpl must not import FeatureBImpl · Trap: convenience import · Prod: graph as architecture
Module testability	Mock Interface protocols in unit tests · Trap: only UITests on full app · Prod: S9 test discipline
Binary size gate	CI budget + thinning · Trap: modules magically shrink binary · Prod: watch duplicates
Singleton rule	OK rarely at process boundary; never hidden in features · Trap: NetworkManager.shared in VM · Prod: S2 shared-state API
First module to extract	Leaf with clear API (design system / networking / Stories) · Trap: split biggest mess first · Prod: S10
Build Timing Summary	Measure before/after splits · Trap: split blindly · Prod: xcodebuild flag
SDK host config	Theme, analytics, loader via protocols · Trap: hardcode Heat branding · Prod: multi-app Stories

Images / media (`images`) · Day 16

Front	Back
Decoded image math	w×h×4 bytes · Trap: think JPEG size · Prod: downsample always
L1/L2/L3	NSCache / disk / network · Trap: URLCache = UIImage cache · Prod: cheatsheet pipeline

Thumbnail API	CGImageSourceCreateThumbnailAtIndex · Trap: draw full UIImage then scale · Prod: ImageIO
Cache key	url + targetSize · Trap: URL only · Prod: avatar vs hero
Deduplication	One in-flight per key, N observers · Trap: N downloads · Prod: feed posters
Cancellation	Token + ignore stale · Trap: late setImage · Prod: cell reuse
NSCache benefit	Evicts under pressure · Trap: deterministic LRU · Prod: memory warnings
HeroWidget rule	Pause offscreen / on disappear · Trap: fire-and-forget AVPlayer · Prod: S1 ads
prepareForReuse	Stop player, clear image, cancel loads · Trap: only clear text · Prod: revenue ads
Prefetch footgun	Decode storm on fling · Trap: prefetch everything · Prod: cancel + cap
Aces audio	Session + interruptions · Trap: treat like image TTL · Prod: S12
Splash metric	Time-to-interactive · Trap: only TTFF vanity · Prod: S12 SDUI splash
Write-around images	Decode then memory; disk async · Trap: sync disk on main · Prod: hitch budget
Memory warning	Trim L1, pause media · Trap: wipe user data · Prod: /Caches OK to lose
GIF scope	Separate path / call out of scope · Trap: same as JPEG pipeline · Prod: SD interviews
Inject loader in SDK	Protocol from host · Trap: hardcode dependency · Prod: S10 Stories

Performance (`perf`) · Day 17

Front	Back
Perf loop	Symptom → attribute → fix → p50/p90 verify · Trap: vibe optimise · Prod: S5
Why p90	Tail = real pain at scale · Trap: averages · Prod: BMS journeys
Time Profiler	CPU hotspots · Trap: find network waits · Prod: main-thread JSON
MetricKit	Daily OS aggregates / hangs · Trap: replaces custom traces · Prod: fleet truth
Firebase Performance	Custom journey traces · Trap: only FPS · Prod: listing/checkout/search
Hitch	Missed frame deadline · Trap: low CPU = smooth · Prod: decode on main
Cold start phases	Pre-main → init → first frame → TTI · Trap: only TTFF · Prod: S12 splash
Hang detector	Ping main; threshold capture · Trap: ignore overhead · Prod: APM doc
Lab vs field	Instruments vs MetricKit/Firebase · Trap: one device = fleet · Prod: 30L DAU
Defer SDK init	Launch budget · Trap: init all in didFinish · Prod: start regression
Signposts	Custom intervals · Trap: unguided profiling · Prod: journey spans
LE sheet as perf	Less navigation cost · Trap: only micro-CPU · Prod: S6 30%+ flows
Network-bound p90	Fix API/cache · Trap: micro-optimise UI · Prod: S5 + backend
Scroll budget	~16ms @60fps · Trap: 100ms OK · Prod: hitch tools
APM overhead	Budget + batch + sample · Trap: sync upload always · Prod: APM NFR

Crashes / reliability (`crashes`) · Day 18

Front	Back
CFS	Crash-free sessions; BMS 99.95%+ · Trap: vanity without process · Prod: S8
Signal safety	No alloc/lock/ObjC in handler · Trap: log normally · Prod: mmap writer
dSYM	Symbolicate UUID-matched builds · Trap: forget CI upload · Prod: unreadable stacks
Breadcrumbs	Last N events, scrub PII · Trap: log tokens · Prod: ring buffer
OOM vs crash	Jetsam heuristics vs fatal signal · Trap: one bucket · Prod: memory tools
IMOC	Owner, blast radius, mitigate, comms · Trap: hero debug only · Prod: S8 P0/P1
Mitigate first	Flag/pause rollout · Trap: wait for perfect RCA · Prod: peak events
Sync dictionaries	Serialise shared map access · Trap: sprinkle locks · Prod: S2
Non-fatals	Useful ≠ CFS · Trap: alert on all · Prod: sample/group
Launch order	Crash SDK early + fast · Trap: after all analytics · Prod: launch crashes
Postmortem	Blameless actions · Trap: finger-point · Prod: S8 write-up
Hang gap	CFS can look fine · Trap: ignore freezes · Prod: MetricKit hangs
Upload timing	Next launch, not in-signal · Trap: network in handler · Prod: crash SDK
Severity matrix	Rare×critical still P0 · Trap: % only · Prod: payments
Symbolicate fail	Missing dSYM · Trap: “Swift bug” · Prod: fix pipeline
30L DAU	Scale context for reliability · Trap: startup metrics only · Prod: resume

Security / storage (`security`) · Day 19

Front	Back
ATS	HTTPS/TLS baseline · Trap: equals pinning · Prod: no cleartext
SPKI pin	Hash public key · Trap: pin leaf cert only · Prod: S4
Pin rotation	Dual pins + ship early · Trap: rotate server first · Prod: S4 lesson
Domain allowlist	Only known hosts · Trap: pin world · Prod: BMS Ads
Alamofire→URLSession	Ownership of trust path · Trap: cargo-cult · Prod: S4
Keychain vs UD	Secrets vs prefs · Trap: token in UD · Prod: auth
Caches vs App Support	Purgeable vs user data · Trap: media in UD · Prod: FileManager
SQLite WAL	Fast writes, concurrent readers · Trap: main-thread queries · Prod: cheatsheet
Core Data use	Object graph needs · Trap: three booleans · Prod: decision tree
NSCache	Session images · Trap: secrets · Prod: Day 16
Biometric truth	Unlocks Keychain item · Trap: store face data · Prod: LAContext
Break-glass pins	Remote disable carefully · Trap: unauthenticated toggle · Prod: ops
Hard-fail pin	Prefer for sensitive APIs · Trap: silent insecure fallback · Prod: ads/pay
SQLCipher	Encrypted DB · Trap: plaintext PII · Prod: threat model

Obfuscation	Not security · Trap: base64 token · Prod: Keychain
Decision tree opener	“Depends on sensitivity + query needs…” · Trap: one tool · Prod: cheatsheet

Deeplinks / push / CI (`deeplinks ci`) · Day 20

Front	Back
Universal Links	HTTPS + AASA association · Trap: equal to schemes · Prod: S13
AASA	Well-known JSON appIDs/paths · Trap: HTTP OK · Prod: HTTPS only
Cold-start queue	Hold link until nav ready · Trap: route instantly · Prod: race bugs
Deferred link	Post-install route · Trap: infinite retention · Prod: expiry
One router	UL + push share table · Trap: duplicate switches · Prod: consistency
APNs token	Register → provider · Trap: hardcode · Prod: refresh
Airship	Engagement layer on APNs · Trap: replaces understanding APNs · Prod: Grizzlies
Payload safety	Defensive parse · Trap: force unwrap · Prod: CFS
GH Actions→TF	Build/lint/upload automation · Trap: no gates · Prod: BMS
Release train	Cadence + phased % · Trap: big-bang 100% · Prod: pause on CFS
dSYM in CI	Upload every build · Trap: only local · Prod: Day 18
Hybrid deeplink	Coordinator owns stack · Trap: ad hoc UIHostingController · Prod: S13
Secure checkout link	Auth + server truth · Trap: trust query price · Prod: payments
AI PR review	Assist, don’t own · Trap: “AI approved” · Prod: S9
Signing secrets	CI secrets, not git · Trap: commit p12 · Prod: rotate
Mixpanel vs Airship	Analytics vs push/engagement · Trap: same tool · Prod: Grizzlies

System-design mock meta (`mock`) · Day 21

Front	Back
45-min spine	Clarify 5 / HLD 10 / API 10 / Dive 15 / Ops 5 · Trap: dive first · Prod: cheatsheet
Agenda opener	Propose plan; confirm · Trap: silent drawing · Prod: staff signal
Scale line	30L+ DAU from resume · Trap: invent QPS · Prod: BMS
CFS in ops	99.95%+ constraint · Trap: omit reliability · Prod: S8
SDUI deep dives	Versioning, fallbacks, actions · Trap: only widgets · Prod: S3
Networking deep dives	Refresh, pin rotation, cache · Trap: list every verb · Prod: S4
Unknown component	Fallback + metric · Trap: crash · Prod: schema lesson
Pin rotation	Dual pins + break-glass · Trap: server-first rotate · Prod: S4
Ops closer	Failures + SLIs + rollout · Trap: end on boxes · Prod: Days 17–20
Resume discipline	Only real metrics · Trap: inflate · Prod: story bank

Negotiate scope	Primary + optional · Trap: both full systems · Prod: T3
Module mention	Protocols at boundaries · Trap: ignore packaging · Prod: Day 15
Image callout	Downsample math if media · Trap: ignore memory · Prod: Day 16
p50/p90	Journey traces · Trap: averages · Prod: S5
Checkpoint habit	Ask “OK to proceed?” · Trap: monologue · Prod: T8

Anki sample: anki-import.csv

Week 4 Flashcards — DSA, On-device AI, Behavioral, Mock Tips

Candidate: **Muhammed Shahid** · BMS / District / Raw
Resume metrics only: **30L+ DAU**, **99.95%+ CFS**, **30%+ fewer full-screen navs**.

Source days: `weeks/week-04/day-22/README.md` … `day-28`.

≈70 cards · Tags: `dsa` · `ai` · `behavioral` · `machine` · `mock` · `bms`

Trees BFS/DFS (`dsa`) · Day 22

Front	Back
BFS vs DFS trigger	Levels/shortest unweighted → BFS; path/subtree/LCA/structure → DFS · Trap: using DFS for “level averages” · Prod: layout pass vs path resolve
Level-size BFS idiom	`for i in 0..<queue.count` each wave · Trap: mixing levels · Prod: batch UI updates by depth
BST validate	Bounds or inorder sorted · Trap: only compare parent · Prod: schema validation analogy
LCA binary tree	Postorder both-sides found → node · Trap: assuming BST walk · Prod: nearest shared nav ancestor
Tree complexity script	O(n) time; O(h)/O(w) space; worst O(n) · Trap: saying O(1) recursive · Prod: skewed CMS tree
Diameter one-pass	DFS height + global left+right max · Trap: root-only sum · Prod: longest dependency chain metaphor
Serialize need	Explicit nulls or count headers · Trap: values-only preorder · Prod: SDUI versioning
Inorder use	BST sorted ops, k-th · Trap: inorder on non-BST for “sorted” · Prod: —
Say this first	Clarify → brute → optimize → complex → edges → code · Trap: silent coding · Prod: senior signal
Symmetric tree	Mirror DFS or BFS pair queue · Trap: only check root children · Prod: mirrored layout QA

HashMap / Heap / mixed (`dsa`) · Day 23

Front	Back
Two Sum default	Hash complement O(n) · Trap: sort loses indices · Prod: id→model maps
Top-K heap size	Min-heap size K → O(n log k) · Trap: max-heap confusion for Kth largest stream · Prod: top similar items
Prefix sum + hash	Subarray sum K with negatives · Trap: two pointer · Prod: analytics rollups

Window + counts	Shrink when invariant breaks · Trap: forget remove at 0 · Prod: debounce coalescing keys
Merge K lists	Heap heads O(N log K) · Trap: full sort only · Prod: merge paginated cursors metaphor
Hash complexity say	Expected O(1); space O(n) · Trap: claiming worst O(1) always · Prod: synchronised dict access ≠ hash math
Unknown pattern 90s	Clarify brute classify pick · Trap: code silently · Prod: senior signal
Bucket top-K freq	O(n) when freq ≤ n · Trap: always heap · Prod: —
Heap wrong when	Need full order / keyed delete · Trap: heap for everything · Prod: —
Clone graph	Hash old→new + DFS/BFS · Trap: no map infinite loop · Prod: object graph copy

On-device AI (`ai`) · Day 24

Front	Back
On-device RAG	Local retrieve → prompt → generate · Trap: “fine-tune on user data” · Prod: FinTrack BM25
Fail-soft AI	Rules/TF-IDF/capability flags · Trap: blank error only · Prod: S15/S16
Quantization why	Fit RAM/ANE; INT8/INT4 · Trap: ignore quality · Prod: GymFlow MiniLM
Hybrid router inputs	Thermal, battery, tokens, capability · Trap: cloud always · Prod: doc HybridAIRouter
Privacy script	Local-first; minimize; consent cloud · Trap: HTTPS = privacy · Prod: FinTrack no sync
BM25 vs MiniLM	Lexical vs semantic · Trap: one-size · Prod: FinTrack vs GymFlow
Hallucinated money	DB is source of truth · Trap: trust tokens · Prod: FinTrack
KV-cache	Faster decode; RAM cost; drop on warning · Trap: ignore Jetsam · Prod: doc model manager
Token stream UI	Background + main append + cancel · Trap: sync main inference · Prod: —
AI tooling vs product AI	Accelerator ≠ on-device architecture · Trap: tool worship · Prod: District vs FinTrack
30L DAU caution	Kill switch, sample, budgets · Trap: run LLM on all sessions · Prod: BMS judgment
Empty retrieval	No invent; generic + CTA · Trap: hallucinate · Prod: coach empty state

Machine round (`machine`) · Day 25

Front	Back
3-hr phases	Clarify 15 → slice → depth → tests → buffer · Trap: polish first · Prod: shipping bias
Vertical slice	One E2E happy path · Trap: all layers half-done · Prod: —
Pagination guard	In-flight + stale token · Trap: double append · Prod: search lists
Cache SWR	Show stale then refresh · Trap: empty until network · Prod: perceived perf
SDUI unknown type	Fallback not crash · Trap: force unwrap · Prod: S3/S12
Tests enough	3 meaningful · Trap: 0 or 40 flaky · Prod: District AI judgment
Cut line	Say what you skip · Trap: hide gaps · Prod: senior honesty
Protocol repo	Fake first then live · Trap: URLSession in VM · Prod: Clean/MVVM

Proctor agenda	Speak plan at 0:15 · Trap: silent · Prod: timing guide
Perfectionism flag	Styling while core broken · Trap: rename churn · Prod: —

Behavioral (`behavioral` `bms`) · Day 26

Front	Back
STAR timing	20 / 90 / 30 / 20 · Trap: 5-min Action · Prod: all stories
Conflict router	S6 or S8 · Trap: vague drama · Prod: 30% nav / IMOC
Incident lesson	Stabilize → RCA → prevent · Trap: blame · Prod: 99.95% CFS
Mentorship signal	Checklist, SDK, standards · Trap: “I’m nice” · Prod: S10/S14
AI tooling	Context + review · Trap: tool-worship · Prod: S9
Product vs process AI	S15/S16 vs S9 · Trap: conflate · Prod: Day 24
Metrics allowed	30L DAU, 99.95%, 30% nav · Trap: invent · Prod: resume
Failure STAR	Real miss + process change · Trap: humblebrag fail · Prod: —
Senior why	Scope + ownership + judgment · Trap: years only · Prod: —
Quick picker	Tags → IDs in story bank · Trap: search mid-answer · Prod: —

Full-mock tips (`mock`) · Days 27–28

Front	Back
Coding first 3 min	Clarify brute optimize complex edges · Trap: silent code · Prod: —
SD 45 clock	5–10–10–15–5 · Trap: deep dive forever · Prod: cheatsheet
Deep dive budget	45s / 120s / 5 min arch · Trap: lecture · Prod: timing guide
Proof trio	30L DAU / 99.95% / 30% nav · Trap: invent · Prod: resume
Protect ops	Always last 5 SD · Trap: cut metrics · Prod: kill switch
Blank recovery	Assumption + draw + brute · Trap: apology loop · Prod: —
Mock breaks	10–15 min real · Trap: cram mid-loop · Prod: —
Debrief cap	≤3 fix-forwards · Trap: rewrite life · Prod: Day 28
AI pivot	Fail-soft + privacy · Trap: tool worship · Prod: S15/S9
Score target	≥3.5 segments · Trap: ignore timing · Prod: —
Metric trio	30L+ DAU / 99.95% CFS / 30%+ nav · Trap: invent · Prod: resume
No new topics	Flash + stories + sleep · Trap: one more LC · Prod: —
Fail-soft one-liner	Degrade usefully; don’t crash · Trap: blank error · Prod: S15/S16
SD protect ops	Last 5 min failure/metrics · Trap: skip · Prod: —
STAR cut	3 Action bullets if long · Trap: 5 min story · Prod: —

Anki sample: anki-import.csv · DSA track: ../coding/dsa-track.md · Timing: ../timing/answer-timing-guide.md